

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Гневшев А. Ю. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**РАЗРАБОТКА И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА RAG-СИСТЕМЫ
ВОПРОС – ОТВЕТ ПО КОРПУСУ НОРМАТИВНЫХ ПРАВОВЫХ
АКТОВ РФ С ОБОСНОВАНИЕМ ОТВЕТОВ ЦИТАТАМИ ИЗ
ПЕРВОИСТОЧНИКОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Мягкий Александр Сергеевич, 221–3210 /
подпись *ФИО, группа*

Руководитель ВКР: _____ / Перепелкина Юлианна Вячеславовна,
подпись канд. физ.-мат. наук /
ФИО, уч. звание и степень

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Тема ВКР	Разработка и экспериментальная оценка RAG-системы вопрос-ответ по корпусу нормативных правовых актов РФ с обоснованием ответов цитатами из первоисточников
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Назначение системы — предоставить пользователю возможность задать юридический вопрос на естественном языке, автоматически найти релевантные фрагменты нормативных правовых актов, сформировать ответ с использованием найденного контекста и показать цитаты из первоисточников, на которых основан ответ.
Основные функции	1. Подготовка локального корпуса нормативных правовых актов РФ из .doc-экспортов. 2. Парсинг .doc/HTML-экспорта нормативных актов в структурированный JSON. 3. Извлечение и нормализация метаданных нормативного акта: вид акта, название, номер, дата, редакция, статус, источник. 4. Разбиение нормативных актов на фрагменты с сохранением юридической структуры документа. 5. Формирование текстового представления фрагмента, включающего название акта, структурный путь и текст нормы. 6. Загрузка нормативных актов и фрагментов в базу данных PostgreSQL. 7. Хранение актов, фрагментов, пользовательских запросов, ответов, цитат и служебных метаданных. 8. Построение полнотекстового поиска по фрагментам нормативных актов средствами PostgreSQL. 9. Расчёт векторных представлений фрагментов с

	использованием модели BAAI/bge-m3. 10. Хранение и поиск векторных представлений фрагментов с использованием расширения pgvector. 11. Выполнение полнотекстового поиска релевантных фрагментов. 12. Выполнение векторного поиска релевантных фрагментов. 13. Выполнение гибридного поиска, объединяющего результаты полнотекстового и векторного поиска. 14. Ограничение объёма контекста, передаваемого языковой модели. 15. Формирование запроса к языковой модели на основе вопроса пользователя и найденных фрагментов. 16. Генерация ответа через GigaChat с опорой на найденный контекст. 17. Получение от языковой модели списка использованных фрагментов. 18. Проверка того, что использованные фрагменты входят в переданный модели контекст. 19. Формирование цитат по использованным фрагментам нормативных актов. 20. Отображение пользователю ответа, цитат и сведений об источниках. 21. Отображение страницы источника с реквизитами нормативного акта и процитированными фрагментами. 22. Сохранение успешных пользовательских запросов, ответов, цитат и параметров поиска в базе данных. 23. Поддержка экспериментального сравнения полнотекстового, векторного и гибридного поиска на наборе тестовых вопросов. 24. Формирование отчёта по результатам экспериментальной оценки качества извлечения релевантных фрагментов.
Используемые технологии и платформы	Python, SQL, HTML, CSS, JSON, YAML; FastAPI, Uvicorn, Jinja2; PostgreSQL, PostgreSQL Full-Text Search, pgvector; SentenceTransformers, BAAI/bge-m3; LLMProvider, GigaChat API; Docker Compose; локальное веб-приложение и серверная среда для подготовки корпуса, поиска, генерации ответа и экспериментальной оценки.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Проанализировать предметную область вопросно-ответного поиска по нормативным правовым актам Российской Федерации; 2. Рассмотреть теоретические основы RAG-подхода, методов извлечения релевантных фрагментов и генерации ответов; 3. Проанализировать существующие юридические справочно-поисковые и интеллектуальные системы и определить требования к разрабатываемому прототипу; 4. Спроектировать архитектуру RAG-системы и модель данных для хранения нормативных актов, фрагментов, запросов, ответов и цитат; 5. Подготовить корпус нормативных правовых актов Российской Федерации для использования в вопросно-ответной системе; 6. Реализовать программный прототип RAG-системы, обеспечивающий поиск релевантных фрагментов, генерацию ответа и отображение источников; 7. Разработать пользовательский сценарий работы с прототипом в формате «вопрос — ответ — цитаты — источник»; 8. Провести экспериментальное сравнение полнотекстового, векторного и гибридного извлечения релевантных фрагментов на наборе тестовых вопросов; 9. Проанализировать результаты разработки и экспериментальной оценки, определить ограничения текущей реализации и направления дальнейшего развития.
Состав технической документации	1. Техническое задание на разработку прототипа RAG-системы; 2. Программа и методика испытаний прототипа; 3. Руководство по локальному запуску и настройке системы; 4. Отчёт по результатам экспериментальной оценки методов извлечения релевантных фрагментов.
Состав графической части	1. Рисунок 1 – Блок-схема критического пользовательского сценария «вопрос – ответ – цитаты – источник»; 2. Рисунок 2 – Общая архитектура прототипа RAG-

	системы вопросно-ответного поиска по нормативным правовым актам Российской Федерации; 3. Рисунок 3 – Инфологическая модель базы данных прототипа RAG-системы; 4. Рисунок 4 – Блок-схема гибридного извлечения релевантных фрагментов нормативных правовых актов; 5. Рисунок 5 – Блок-схема формирования ответа и цитат; 6. Рисунок 6 – Схема экспериментальной оценки извлечения релевантных фрагментов; 7. Рисунок 7 – Схема проверки генерации ответа и формирования цитат; 8. Рисунок 8 – Сравнение Hit@10 для методов извлечения релевантных фрагментов; 9. Рисунок 9 – Сравнение Recall@10 для методов извлечения релевантных фрагментов; 10. Рисунок 10 – Сравнение медианного времени извлечения релевантных фрагментов; 11. Рисунок 11 – Страница ответа с цитатами в веб-прототипе; 12. Рисунок 12 – Страница источника с процитированным фрагментом нормативного акта; 13. Рисунок 13 – Пример сообщения о недостаточности контекста.
--	---

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Гневшев Александр Юрьевич /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Перепелкина Юлианна Вячеславовна,
подпись канд. физ.-мат. наук /
ФИО, уч. звание и степень

СТУДЕНТ:

«__»____2026, _____ / Мягкий Александр Сергеевич, 221–3210 /
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке прототипа RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации с формированием проверяемых ответов и цитат из первоисточников. Цель работы — разработать программный прототип веб-ориентированной системы, позволяющий по вопросу пользователя находить релевантные фрагменты нормативных правовых актов, формировать ответ с использованием найденного контекста и отображать цитаты, на которых основан ответ.

Для достижения цели выполнен анализ предметной области и существующих подходов к построению RAG-систем, сформированы требования к прототипу, подготовлен локальный корпус нормативных актов, спроектированы структура данных и архитектура приложения, реализованы этапы поиска, генерации ответа, отображения источников, авторизации пользователя и сбора обратной связи.

Практическим результатом является локальный веб-прототип системы, включающий подготовку корпуса из .doc/HTML-экспортов нормативных актов, парсинг документов в структурированный JSON, разбиение актов на фрагменты с сохранением юридической структуры, загрузку данных в PostgreSQL, полнотекстовый поиск средствами PostgreSQL, векторный поиск с использованием pgvector и модели BAAI/bge-m3, гибридное извлечение релевантных фрагментов, генерацию ответа через GigaChat, формирование цитат по проверенным идентификаторам использованных фрагментов, регистрацию и вход пользователя, профиль, историю вопросов и обратную связь по ответу.

Работа состоит из введения, трёх глав, заключения, списка использованных источников и семи приложений: А, Б, В, Г, Д, Е, Ж. В первой главе рассматриваются предметная область, подход RAG, методы извлечения релевантных фрагментов, проблема проверяемости ответов и аналоги юридических систем. Во второй главе описаны требования, архитектура, база данных и реализация прототипа. В третьей главе приведены методика и результаты экспериментальной оценки, проверка критического пользовательского сценария и анализ ограничений реализации.

Список использованных источников включает 56 наименований. Промышленное внедрение системы не проводилось; апробация выполнялась в форме локального запуска прототипа и экспериментальной оценки на тестовых вопросах.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	12
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И МЕТОДОВ ПОСТРОЕНИЯ RAG-СИСТЕМ	17
1.1 Предметная область вопросно-ответного поиска по нормативным правовым актам Российской Федерации	17
1.2 Проблема проверяемости ответов в юридических информационных системах	18
1.3 Подход Retrieval-Augmented Generation и его применимость к юридическим текстам	19
1.4 Методы извлечения релевантных фрагментов: полнотекстовый, векторный и гибридный поиск	20
1.5 Векторные представления текстов и векторный поиск в реляционной базе данных	21
1.6 Методы оценки RAG-систем: качество извлечения, согласованность ответа с контекстом и риск галлюцинаций	22
1.7 Анализ аналогов юридических справочно-поисковых и интеллектуальных систем	23
1.8 Выводы по первой главе	25
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРОТОТИПА RAG-СИСТЕМЫ	26
2.1 Назначение системы, роли пользователей и критический пользовательский сценарий	26
2.2 Функциональные и нефункциональные требования к системе	28
2.3 Общая архитектура прототипа и схема обработки пользовательского вопроса	30
2.4 Подготовка локального корпуса нормативных правовых актов	31
2.5 Разбиение нормативных актов на фрагменты и формирование текстового представления фрагмента	32

2.6	Проектирование базы данных и хранение корпуса в PostgreSQL .	33
2.7	Реализация полнотекстового поиска	36
2.8	Расчет векторных представлений и реализация векторного поиска через pgvector	36
2.9	Реализация гибридного извлечения релевантных фрагментов . . .	37
2.10	Генерация ответа через GigaChat и формирование цитат	39
2.11	Веб-прототип пользовательского сценария «вопрос — ответ — цитаты — источник»	41
2.12	Выводы по второй главе	42
3	ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОТОТИПА	44
3.1	Цель, гипотезы и ограничения экспериментальной оценки	44
3.2	Подготовка набора тестовых вопросов и эталонных релевантных фрагментов	47
3.3	Метрики оценки извлечения релевантных фрагментов	49
3.4	Методика оценки согласованности ответа с найденным контекстом	50
3.5	Экспериментальная конфигурация полнотекстового извлечения .	52
3.6	Экспериментальная конфигурация векторного извлечения	53
3.7	Экспериментальная конфигурация гибридного извлечения	54
3.8	Сравнительный анализ результатов полнотекстового, векторного и гибридного извлечения	55
3.9	Проверка критического пользовательского сценария	59
3.10	Анализ ошибок и устойчивости прототипа	63
3.11	Выводы по третьей главе	65
	ЗАКЛЮЧЕНИЕ	67
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	69
	ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	75
	ПРИЛОЖЕНИЕ Б. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ	83
	ПРИЛОЖЕНИЕ В. ОПИСАНИЕ СХЕМЫ БАЗЫ ДАННЫХ	88

ПРИЛОЖЕНИЕ Г. ПРИМЕРЫ ПОДГОТОВЛЕННЫХ ФРАГМЕНТОВ НОРМАТИВНЫХ АКТОВ	93
ПРИЛОЖЕНИЕ Д. МАТЕРИАЛЫ ЭКСПЕРИМЕНТАЛЬНОЙ ОЦЕНКИ .	100
ПРИЛОЖЕНИЕ Е. ПРИМЕРЫ ЗАПРОСОВ И ОТВЕТОВ СИСТЕМЫ . .	106
ПРИЛОЖЕНИЕ Ж. ЛИСТИНГИ КЛЮЧЕВЫХ ФРАГМЕНТОВ ПРОГРАММНОГО КОДА	111

ВВЕДЕНИЕ

Нормативные правовые акты Российской Федерации образуют большой и формально организованный массив юридической информации. При решении практического вопроса пользователю обычно недостаточно найти документ целиком: необходимо определить применимую статью, часть или пункт, проверить реквизиты акта и убедиться, что вывод действительно следует из текста источника. Поэтому для вопросно-ответной системы по законодательству важны не только скорость поиска, но и проверяемость ответа по нормативным правовым актам.

Языковые модели способны формировать связные ответы на естественном языке, однако при генерации без опоры на внешний корпус возникает риск недостоверных утверждений. В исследованиях это явление описывается как галлюцинация, то есть появление в ответе сведений, не подтвержденных входными данными или внешними фактами [1]. Для юридической области данный риск особенно существен: неточная формулировка о праве, обязанности, сроке или исключении может привести пользователя к неверному практическому выводу. Отдельные исследования юридических систем на основе искусственного интеллекта также показывают, что ответы таких систем требуют проверки по источникам [2, 3].

Одним из подходов к снижению указанного риска является Retrieval-Augmented Generation, далее RAG. В этом подходе языковая модель формирует ответ не только по вопросу пользователя, но и по контексту, предварительно извлеченному из внешнего корпуса документов [4, 5]. Для извлечения релевантных фрагментов могут применяться полнотекстовые методы поиска, векторный поиск по смысловой близости и гибридные схемы, объединяющие оба подхода [6, 7, 8]. В юридической системе этот подход позволяет связать итоговый ответ с конкретными фрагментами нормативных актов и вывести пользователю цитаты.

Актуальность темы выпускной квалификационной работы обусловлена противоречием между потребностью пользователя быстро получить ответ на правовой вопрос и необходимостью сохранять юридическую проверяемость результата. Справочно-правовые системы хорошо решают задачу доступа к документам, но пользователь часто самостоятельно анализирует найденные нормы. Генеративная система, напротив, может представить ответ в удобной форме, но без явной связи с первоисточниками такой ответ нельзя считать достаточным. Поэтому практический интерес представляет прототип, который объединяет извлечение релевантных фрагментов, генерацию ответа и отображение источников.

Проблема исследования состоит в необходимости разработать и оценить программный прототип RAG-системы, который по вопросу пользователя извлекает релевантные фрагменты нормативных правовых актов Российской Федерации, формирует ответ на основе найденного контекста и предоставляет цитаты для проверки. При этом система должна учитывать особенности юридических текстов: иерархическую структуру актов, значимость реквизитов, необходимость точной ссылки на статью или пункт, а также ограничение актуальности локального корпуса.

Объектом исследования являются технологии интеллектуального поиска и генерации ответов по юридическим текстам.

Предметом исследования являются методы подготовки корпуса нормативных правовых актов, разбиения документов на фрагменты, полнотекстового, векторного и гибридного извлечения релевантного контекста, а также методы генерации ответа с опорой на найденные источники.

Цель выпускной квалификационной работы — разработать и экспериментально оценить программный прототип веб-ориентированной RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации, обеспечивающий извлечение релевантных фрагментов, генерацию ответа и отображение цитат по использованным источникам.

Для достижения поставленной цели необходимо решить следующие задачи:

1. проанализировать предметную область вопросно-ответного поиска по нормативным правовым актам Российской Федерации и проблеме проверяемости юридических ответов;
2. исследовать подход RAG, методы полнотекстового, векторного и гибридного извлечения, а также подходы к оценке качества таких систем;
3. проанализировать существующие справочно-правовые и интеллектуальные юридические системы и определить отличия разрабатываемого прототипа;
4. сформулировать функциональные и нефункциональные требования к системе и определить критический пользовательский сценарий;
5. подготовить локальный корпус нормативных правовых актов к поиску и генерации ответа;
6. спроектировать хранение нормативных актов, фрагментов, векторных представлений, пользовательских вопросов, ответов и цитат;
7. реализовать полнотекстовое, векторное и гибридное извлечение релевантных фрагментов;
8. реализовать генерацию ответа через GigaChat и формирование цитат на основе использованных фрагментов;
9. разработать веб-прототип сценария «вопрос — ответ — цитаты — источник» с регистрацией, входом, профилем пользователя, историей вопросов и обратной связью;
10. провести экспериментальное сравнение методов извлечения и проверить работоспособность критического пользовательского пути.

Практическая часть работы основана на локальном корпусе нормативных правовых актов Российской Федерации. В корпус включаются кодексы Российской Федерации, Конституция Российской Федерации, федеральные законы и федеральные конституционные законы, полученные из *.doc*-экспортов, содержащих HTML-представление документов. Документы пре-

образуются в структурированный JSON, разбиваются на фрагменты по юридически значимым границам, загружаются в PostgreSQL и снабжаются векторными представлениями. Для полнотекстового поиска используются средства PostgreSQL [7], для векторного поиска — расширение pgvector [9], для расчета векторных представлений — модель BAAI/bge-m3 [10, 11]. Генерация ответа реализуется через GigaChat API [12].

Исследовательская часть работы направлена на сравнение трех вариантов извлечения релевантного контекста: полнотекстового, векторного и гибридного. Качество извлечения оценивается по набору тестовых вопросов и эталонных релевантных фрагментов, а качество ответа — с учетом согласованности с найденным контекстом, корректности цитирования и риска галлюцинаций. Такой подход соответствует современным работам по оценке RAG-систем, где отдельно рассматриваются качество извлечения, использование контекста и качество генерации [13].

Практическая значимость работы заключается в создании локального веб-прототипа информационной системы, который позволяет авторизованному пользователю задать вопрос на русском языке, получить ответ, проверить его по цитатам из нормативных актов, вернуться к истории своих вопросов и оставить обратную связь. Прототип используется как основа для исследования юридических RAG-систем и сравнения методов поиска по нормативным текстам.

Работа состоит из введения, трех глав, заключения, списка использованных источников и семи приложений: А, Б, В, Г, Д, Е, Ж. В первой главе рассматриваются предметная область, подход RAG, методы извлечения релевантных фрагментов, проблема проверяемости, методы оценки и аналоги юридических систем. Во второй главе описываются требования, архитектура и прототип: подготовка корпуса, разбиение актов на фрагменты, хранение данных, полнотекстовый и векторный поиск, гибридное извлечение, генерация ответа, цитаты и веб-прототип. В третьей главе приводятся методика

и результаты экспериментальной оценки, проверка критического пользовательского сценария и анализ ошибок.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И МЕТОДОВ ПОСТРОЕНИЯ RAG-СИСТЕМ

1.1 Предметная область вопросно-ответного поиска по нормативным правовым актам Российской Федерации

Нормативные правовые акты Российской Федерации являются официальными источниками общеобязательных правил поведения. К ним относятся Конституция Российской Федерации, федеральные конституционные законы, федеральные законы, кодексы и иные акты. Конституция Российской Федерации имеет высшую юридическую силу и прямое действие [14], а федеральные конституционные законы и федеральные законы подлежат официальному опубликованию [15]. Для доступа к официальным правовым актам используется государственная система правовой информации, в том числе Официальный интернет-портал правовой информации [16, 17].

Для вопросно-ответного поиска важно, что нормативный акт является не только текстом, но и источником юридически значимой информации; особенности работы с правовой информацией рассматриваются в информационном праве [18]. Юридический текст отличается от обычного информационного текста. Нормативный акт имеет реквизиты, статус, редакцию и внутреннюю иерархию: разделы, главы, статьи, части, пункты и подпункты. Смысл отдельной нормы зависит не только от ее текста, но и от положения в структуре документа, от отсылок к другим нормам и от действующей редакции. Поэтому вопросно-ответная система по законодательству должна сохранять не только текст фрагмента, но и название акта, структурный путь, номер статьи, диапазон пунктов и сведения об источнике.

Запрос пользователя обычно формулируется языком жизненной ситуации, а не языком закона. Пользователь может спросить о сроке, обязанности или праве, тогда как в нормативном акте соответствующая норма выражена специальной юридической лексикой. Следовательно, в такой системе недо-

статочно только буквального совпадения слов: необходимо поддерживать как полнотекстовый поиск по терминам и реквизитам, так и смысловой поиск по близости вопроса и фрагмента.

В рамках работы рассматривается локальный корпус нормативных правовых актов Российской Федерации. В него включаются кодексы Российской Федерации, Конституция Российской Федерации, федеральные законы и федеральные конституционные законы, полученные из *.doc*-экспортов, содержащих HTML-представление документов. Такой корпус позволяет реализовать и исследовать полный конвейер подготовки данных, поиска, генерации ответа и формирования проверяемых цитат.

1.2 Проблема проверяемости ответов в юридических информационных системах

Проверяемость является ключевым требованием к юридической вопросно-ответной системе. В правовой области пользователь должен понимать, из какой нормы следует ответ, к какому акту она относится и в какой редакции была использована. Ошибка в словах «вправе», «обязан», «не допускается» или «за исключением» может изменить правовой смысл, поэтому ответ без источников не является достаточным.

В традиционных справочно-правовых системах проверяемость обеспечивается через доступ к тексту документа, реквизитам, редакциям и ссылкам между актами. Пользователь сам анализирует найденные документы. В системах с языковой моделью появляется дополнительный риск: модель может сформулировать убедительный, но неподтвержденный источниками ответ. В литературе такие ошибки рассматриваются как галлюцинации генеративных моделей [1, 19]. Для юридической области этот риск подтверждается специальными исследованиями надежности юридических систем на основе искусственного интеллекта [2, 3].

Для разрабатываемой системы проверяемость понимается как прослеживаемость ответа до найденных фрагментов корпуса. Это означает, что каждый ответ должен быть построен по переданному контексту, содержать идентификаторы использованных фрагментов и сопровождаться цитатами. Дополнительно система должна явно показывать ограничение: ответ строится только по тем редакциям актов, которые загружены в локальный корпус.

1.3 Подход Retrieval-Augmented Generation и его применимость к юридическим текстам

Retrieval-Augmented Generation, далее RAG, объединяет два этапа: извлечение релевантных фрагментов из внешнего корпуса и генерацию ответа на основе найденного контекста [4]. В отличие от генерации только по параметрам модели, RAG позволяет использовать обновляемую базу документов и связывать ответ с источниками. Поэтому подход хорошо подходит для областей, где важны фактическая точность и возможность проверки.

Для юридических текстов применимость RAG определяется несколькими факторами. Во-первых, нормативные акты можно заранее подготовить как корпус фрагментов с метаданными. Во-вторых, пользовательский вопрос можно сопоставлять не только с документом целиком, но и с конкретной статьей или пунктом. В-третьих, найденные фрагменты можно передавать языковой модели как ограниченный контекст, а затем формировать цитаты по фактически использованным фрагментам.

При этом RAG не устраняет все риски автоматически. Если этап извлечения вернул нерелевантные или неполные фрагменты, модель может сформировать неполный ответ. Если фрагменты противоречат друг другу или относятся к разным редакциям, требуется дополнительная проверка. Поэтому качество RAG-системы зависит от подготовки корпуса, способа разбиения документов, метода извлечения, ограничения контекста, инструкции для модели и проверки ответа.

В данной работе RAG рассматривается не как замена юридической экспертизы, а как архитектурный подход к проверяемому вопросно-ответному поиску. Пользователь получает краткий ответ, но также видит фрагменты нормативных актов, по которым этот ответ можно проверить.

1.4 Методы извлечения релевантных фрагментов: полнотекстовый, векторный и гибридный поиск

Первый класс методов — полнотекстовый поиск. Он опирается на лексическое совпадение между запросом и документом: термины запроса сопоставляются с терминами фрагментов, после чего результаты ранжируются по степени соответствия. Общие основы информационного поиска, лексического сопоставления и ранжирования представлены в классической работе по информационному поиску [20]. Модель BM25 и близкие вероятностные подходы являются классической основой информационного поиска [6]. В PostgreSQL полнотекстовый поиск реализуется через типы и функции для построения поискового вектора, запроса и ранжирования результатов [7, 21]. Для юридических текстов такой поиск полезен при точных терминах, названиях актов, номерах статей и устойчивых формулировках.

Второй класс методов — векторный поиск. В этом случае вопрос и фрагменты документов преобразуются в векторные представления, а релевантность определяется по близости векторов. Подход плотного извлечения был широко исследован в задаче открытого вопросно-ответного поиска, в частности в работе Dense Passage Retrieval [8]. Развитие векторных представлений предложений рассматривается также в работе по Sentence-BERT [22]. Для прототипа важна возможность находить фрагменты, которые близки вопросу по смыслу, даже если в них нет полного совпадения слов.

Третий класс — гибридный поиск. Он объединяет результаты лексического и векторного поиска. Такой подход полезен для юридического корпуса, потому что пользовательский вопрос может содержать как точные правовые

термины, так и бытовые формулировки. Одним из простых и воспроизводимых способов объединения списков является Reciprocal Rank Fusion, при котором учитываются позиции документа в нескольких ранжированных списках [23].

Для ВКР важно сравнить эти методы экспериментально. Полнотекстовый поиск может выигрывать на точных запросах, векторный — на переформулированных вопросах, а гибридный — на смешанных ситуациях. Поэтому исследовательская часть работы должна оценить не только итоговый ответ, но и качество извлечения релевантных фрагментов.

1.5 Векторные представления текстов и векторный поиск в реляционной базе данных

Векторное представление текста — это числовой вектор, в котором близкие по смыслу тексты должны располагаться ближе друг к другу в векторном пространстве. Такие представления используются для смыслового поиска, кластеризации, повторного ранжирования и других задач обработки естественного языка. В современных системах они обычно строятся нейросетевыми моделями, обученными сопоставлять близкие тексты.

В работе используется модель BAAI/bge-m3. В статье о BGE M3-Embedding модель описывается как многоязычная, многофункциональная и пригодная для разных гранулярностей текста [10]. Карточка модели BAAI/bge-m3 указывает ее применение для задач извлечения и смыслового поиска [11]. Для практического расчета таких представлений может использоваться библиотека SentenceTransformers [24]. Для корпуса нормативных актов это важно, поскольку фрагменты могут быть как короткими статьями, так и более длинными частями статей.

Хранение векторных представлений можно организовать в отдельной векторной базе данных или в реляционной базе. В прототипе выбран второй вариант: PostgreSQL хранит нормативные акты, фрагменты, пользователь-

ские вопросы, ответы и цитаты, а расширение pgvector добавляет тип данных для векторов и операции поиска ближайших соседей [9]. Такой подход упрощает связь между найденным векторным результатом и юридическими метаданными фрагмента.

Для юридической RAG-системы реляционная база данных также удобна тем, что обеспечивает ограничения целостности, внешние ключи, индексы и возможность воспроизводимой загрузки корпуса. Это позволяет хранить в одном месте текст, реквизиты акта, структурную ссылку, векторное представление, результат поиска и цитаты, использованные при ответе.

1.6 Методы оценки RAG-систем: качество извлечения, согласованность ответа с контекстом и риск галлюцинаций

Оценка RAG-системы должна учитывать несколько уровней. Первый уровень — качество извлечения. На этом уровне проверяется, попали ли релевантные фрагменты в первые результаты поиска. Для этого применяются метрики информационного поиска: полнота на первых позициях, средняя обратная позиция, nDCG и другие показатели ранжирования [25, 26, 27]. В юридической задаче особенно важна полнота среди первых фрагментов, потому что именно они передаются языковой модели.

Второй уровень — качество ответа. Здесь проверяется, отвечает ли система на вопрос, не искажает ли правовой смысл, указывает ли условия и исключения. Для RAG-систем отдельно оценивается согласованность ответа с контекстом: утверждения в ответе должны следовать из найденных фрагментов. В работах по RAGAS выделяются связанные показатели качества извлечения, использования контекста и генерации [13]. Риск галлюцинаций и фактической несогласованности генеративных моделей рассматривается в общих работах по этой проблеме и в исследованиях надежности юридических систем на основе искусственного интеллекта [1, 19, 2, 3].

Третий уровень — корректность цитирования. Для юридической системы недостаточно показать случайный источник рядом с ответом. Цитата должна соответствовать тем фрагментам, которые реально использованы при генерации. Поэтому система должна проверять идентификаторы использованных фрагментов и формировать цитаты программно, а не доверять модели произвольные ссылки.

Автоматические метрики полезны для сравнения методов, но требуют ручной правовой проверки результатов. При подготовке юридических тестов можно учитывать опыт правовых наборов данных и соревнований, предназначенных для оценки RAG-систем, правового рассуждения и юридического поиска [28, 29, 30]. Для сравнения ранжированных списков дополнительно применимы метрики *simulated gain* и *rank-biased precision* [26, 27]. В экспериментальной оценке используются тестовые вопросы, эталонные релевантные фрагменты и разбор ошибок: пропуск нормы, выбор неправильной редакции, нерелевантная цитата, неполный ответ и неподтвержденное утверждение.

1.7 Анализ аналогов юридических справочно-поисковых и интеллектуальных систем

На российском рынке существуют развитые справочно-правовые системы, например КонсультантПлюс, ГАРАНТ и Контур.Норматив [31, 32, 33]. Они предоставляют доступ к нормативным актам, поиску, редакциям, справочным материалам и профессиональной навигации. Их сильная сторона — полнота и редакционная поддержка правовой базы. При этом они не являются открытым исследовательским прототипом, в котором можно управлять способом разбиения корпуса, методом извлечения и параметрами генерации.

Отдельную группу составляют интеллектуальные юридические помощники, например Нейроюрист Яндекса, Harvey и CoCounsel Legal [34, 35, 36]. Они развивают сценарий диалога на естественном языке и применения языковых моделей в юридических задачах. Однако для целей ВКР су-

ществены воспроизводимость корпуса, прозрачность методов извлечения и возможность экспериментального сравнения, а внутренняя реализация таких систем обычно недоступна.

Таблица 1 – Сравнение разрабатываемого прототипа с аналогами

Решение	Основное назначение	Сильные стороны	Ограничение относительно цели ВКР
Консультант-Плюс	Справочно-правовая система	Большая база документов, редакции, поиск, комментарии	Закрытая внутренняя реализация поиска и обработки документов
ГАРАНТ	Справочно-правовая система	Нормативная база, аналитика, профессиональная навигация	Внутренние алгоритмы поиска и обработки недоступны для исследования
Контур.-Норматив	Онлайн-база нормативных документов	Доступ к актам и поиску по базе	Основной результат — документ или список документов, а не проверяемый ответ RAG
Нейроюрист Яндекса	Интеллектуальный юридический помощник	Диалоговый сценарий, работа с естественным языком, источники	Состав корпуса и методика поиска релевантных фрагментов закрыты для внешнего исследования
Harvey	Платформа искусственного интеллекта для юридических команд	Работа с документами и исследовательскими задачами юристов	Ориентирован на корпоративное применение, внутренняя реализация закрыта
CoCounsel Legal	Юридический помощник на основе искусственного интеллекта	Поддержка юридических исследований и работы с документами	Ориентирован на зарубежную юридическую практику и корпоративные сценарии
Разрабатываемый прототип	RAG-система по локальному корпусу НПА РФ	Воспроизводимая подготовка корпуса, поиск, генерация, цитаты	Локальный корпус и параметры эксперимента задают границы применимости результатов

Анализ аналогов показывает, что существующие системы решают близкие пользовательские задачи, а исследовательская задача ВКР требует собственного прототипа. В нем можно контролировать корпус, разбиение на

фрагменты, полнотекстовое и векторное извлечение, генерацию ответа и механизм цитирования.

1.8 Выводы по первой главе

В первой главе рассмотрена предметная область вопросно-ответного поиска по нормативным правовым актам Российской Федерации. Установлено, что юридические тексты имеют формальную структуру, зависят от редакции, содержат перекрестные ссылки и требуют точной связи ответа с источником.

Показано, что центральным требованием к юридической RAG-системе является проверяемость. Ответ должен быть построен по найденному контексту, сопровождаться цитатами и содержать явные ограничения применимости. Наличие языковой модели без контроля источников не устраняет риск галлюцинаций.

Рассмотрены полнотекстовый, векторный и гибридный методы извлечения релевантных фрагментов. Полнотекстовый поиск полезен для точных терминов и реквизитов, векторный — для смысловой близости вопроса и нормы, гибридный — для объединения преимуществ обоих подходов. Сравнение методов выполняется экспериментально.

Также рассмотрены векторные представления текстов, хранение векторов в PostgreSQL через pgvector, методы оценки RAG-систем и аналоги юридических систем. Сделан вывод, что для целей ВКР нужен воспроизводимый прототип, позволяющий контролировать все этапы: подготовку корпуса, извлечение, генерацию, цитирование и оценку качества.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРОТОТИПА RAG-СИСТЕМЫ

2.1 Назначение системы, роли пользователей и критический пользовательский сценарий

Прототип предназначен для вопросно-ответного поиска по собственному локальному корпусу нормативных правовых актов Российской Федерации. Пользователь вводит вопрос на естественном языке, система извлекает релевантные фрагменты актов, формирует ответ на основе найденного контекста и показывает цитаты, по которым можно проверить результат. Область применимости результата определяется составом локального корпуса и редакциями документов, загруженными в базу данных.

В системе выделяются четыре роли. Неавторизованный посетитель может открыть главную страницу и ознакомиться с назначением веб-прототипа, однако для отправки вопроса должен пройти вход или регистрацию. Авторизованный пользователь задает вопросы, получает ответы, открывает источники, просматривает историю своих вопросов в профиле и оставляет обратную связь по ответу. Разработчик подготавливает корпус, запускает загрузку, расчет векторных представлений, поиск, генерацию и отладку. Исследователь качества запускает эксперименты, анализирует метрики извлечения, результаты генерации и ошибки.

Критический пользовательский сценарий состоит из следующих действий. Посетитель открывает главную страницу, система проверяет наличие пользовательской сессии перед отправкой вопроса. Если пользователь не вошел в систему, ему предлагается регистрация или вход. После успешного входа пользователь вводит вопрос и отправляет форму. Серверная часть выполняет извлечение релевантных фрагментов выбранным методом, проверяет достаточность контекста, ограничивает объем передаваемого контекста, передает вопрос и контекст в GigaChat через слой поставщика языковой мо-

дели, проверяет JSON-ответ и список *used_chunk_ids*, формирует цитаты по сохраненным фрагментам, сохраняет результат и показывает страницу ответа. Затем пользователь может открыть страницу источника, отправить оценку ответа, перейти в профиль и просмотреть историю своих вопросов. Последовательность действий в этом сценарии приведена на рисунке 1.

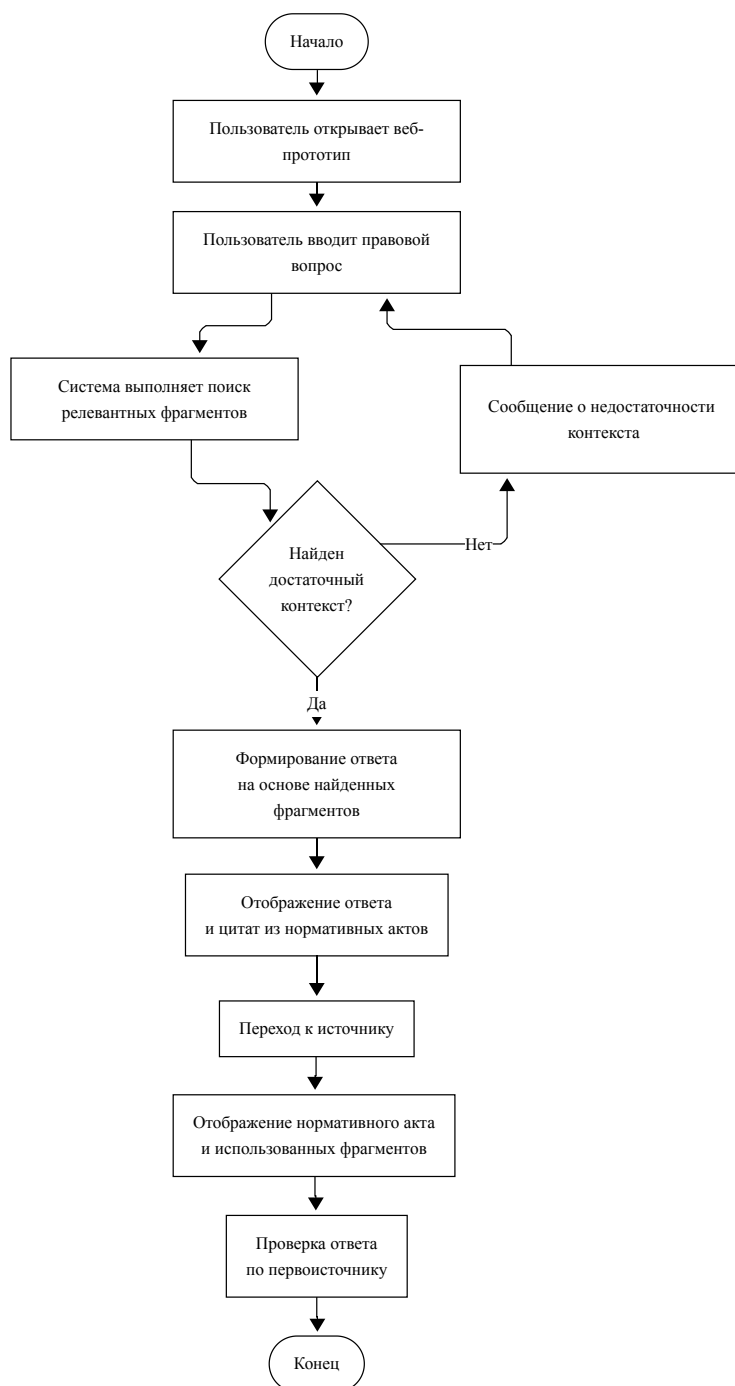


Рисунок 1 – Блок-схема критического пользовательского сценария «вопрос – ответ – цитаты – источник»

2.2 Функциональные и нефункциональные требования к системе

Требования к прототипу сформулированы как проверяемые и прослеживаемые положения. Такой подход соответствует общим принципам инженерии требований, согласно которым требования должны быть недвусмысленными, выполнимыми и пригодными для проверки [37]. В соответствии с подходом С. Куликова требование также должно быть полным, атомарным и контролируемым через наблюдаемое поведение системы [38].

В основной части работы требования рассматриваются на уровне ключевых групп, необходимых для проектирования архитектуры, базы данных, веб-сценария и экспериментальной оценки. Полный перечень функциональных и нефункциональных требований, оформленный в составе технического задания, приведен в приложении А.

Ключевые функциональные направления требований обобщены в таблице 2. Таблица отражает не полный формальный перечень требований, а аналитическую группировку функций, определяющих состав прототипа.

Таблица 2 – Сводка ключевых функциональных направлений требований

Направление	Содержание требований	Проверяемый результат
Подготовка корпуса	Обработка локальных <i>.doc</i> , <i>.html</i> и <i>.htm</i> HTML-экспортов нормативных актов, извлечение метаданных, сохранение структуры документа и разбиение актов на фрагменты.	Создаются структурированные JSON-файлы, файлы подготовленного корпуса и фрагменты с юридически значимыми ссылками.
Загрузка и хранение данных	Загрузка актов и фрагментов в PostgreSQL, хранение полнотекстовых и векторных представлений, сохранение вопросов, ответов, цитат и обратной связи.	В базе заполнены актуальные таблицы приложения; связи между пользователем, вопросом, ответом, цитатами и обратной связью сохраняются.
Извлечение релевантных фрагментов	Выполнение полнотекстового, векторного и гибридного поиска по локальному корпусу с учетом параметров конфигурации.	По вопросу пользователя возвращается ранжированный набор фрагментов, пригодный для передачи в контекст языковой модели.
Формирование проверяемого ответа	Передача вопроса и ограниченного контекста в GigaChat через слой поставщика языковой модели, проверка JSON-ответа и <i>used_chunk_ids</i> , программное формирование цитат.	Пользователь получает ответ с цитатами из базы либо безопасное сообщение о недостаточности контекста.

Продолжение таблицы 2

Направление	Содержание требований	Проверяемый результат
Веб-сценарий пользователя	Регистрация, вход и выход пользователя, отправка вопроса, просмотр ответа и источника, профиль с историей вопросов, отправка обратной связи.	Авторизованный пользователь проходит сценарий «вопрос – ответ – цитаты – источник» и может просмотреть свои сохраненные ответы.
Экспериментальная оценка	Запуск оценки извлечения релевантных фрагментов и проверки генерации на тестовом наборе вопросов.	Результаты оценки сохраняются в файловом контуре проекта в форматах JSONL, CSV и YAML.

Нефункциональные требования определяют ограничения качества и эксплуатации локального прототипа. Они связаны не только с корректностью результата, но и с воспроизводимостью подготовки корпуса, безопасной обработкой пользовательских данных и отделением экспериментальных материалов от рабочей схемы базы данных. Сводка ключевых нефункциональных характеристик приведена в таблице 3.

Таблица 3 – Сводка ключевых нефункциональных характеристик прототипа

Характеристика	Содержание ограничения	Способ контроля
Проверяемость ответа	Ответ должен быть связан с фрагментами нормативных актов, переданными модели в контексте; цитаты формируются программно, а не генерируются моделью.	Проверка списка <i>used_chunk_ids</i> и отображение цитат с реквизитами акта.
Воспроизводимость обработки	Подготовка корпуса, загрузка данных, расчет векторных представлений и экспериментальная оценка должны запускаться повторно.	Использование скриптов, конфигурации <i>settings.yaml</i> и сохранение результатов оценки в файлах.
Сохранение юридической структуры	Фрагмент должен сохранять связь с актом, статьей, структурной ссылкой и при наличии диапазоном пунктов.	Контроль состава полей фрагмента при парсинге, разбиении и загрузке в базу данных.
Защита пользовательского сценария	Пароль не хранится в открытом виде, сохраненные ответы доступны владельцу, обратная связь связана с конкретным пользователем и ответом.	Проверка схемы таблиц <i>users</i> , <i>answers</i> и <i>feedback</i> , а также маршрутов авторизации и доступа.
Диагностика ошибок	Пустой вопрос, недостаточный контекст, ошибка внешней модели или некорректный JSON не должны приводить к неподтвержденному правовому ответу.	Отказ от сохранения неподтвержденного ответа и вывод диагностического сообщения без раскрытия секретных параметров.
Границы локального прототипа	Система работает с локальным корпусом и не описывается как промышленная правовая база или промышленная система управления доступом.	В тексте работы и интерфейсе фиксируются ограничения корпуса, локального запуска и необходимости проверки актуальной редакции по официальному источнику.

Проверяемость требований обеспечивается связью с позитивными и негативными сценариями. Ключевой позитивный сценарий описывает путь авторизованного пользователя от вопроса к ответу, цитатам, источнику, профилю и обратной связи. Дополнительно проверяются сценарии подготовки корпуса и экспериментальной оценки. Негативные сценарии включают пустой вопрос, недостаточный контекст, невалидный ответ внешней модели, недопустимые идентификаторы фрагментов и попытку открыть ответ другого пользователя.

2.3 Общая архитектура прототипа и схема обработки пользовательского вопроса

Архитектура прототипа разделена на два контура: подготовку корпуса и обработку пользовательского вопроса. Первый контур выполняется заранее. Он получает *.doc/HTML*-экспорты нормативных актов, преобразует их в структурированный JSON, разбивает на фрагменты, загружает данные в PostgreSQL и рассчитывает векторные представления. Второй контур запускается при вопросе пользователя: выполняется извлечение фрагментов, формируется контекст, вызывается GigaChat, проверяется ответ и создаются цитаты. Общая архитектура прототипа с разделением на подготовку корпуса и обработку пользовательского вопроса показана на рисунке 2.

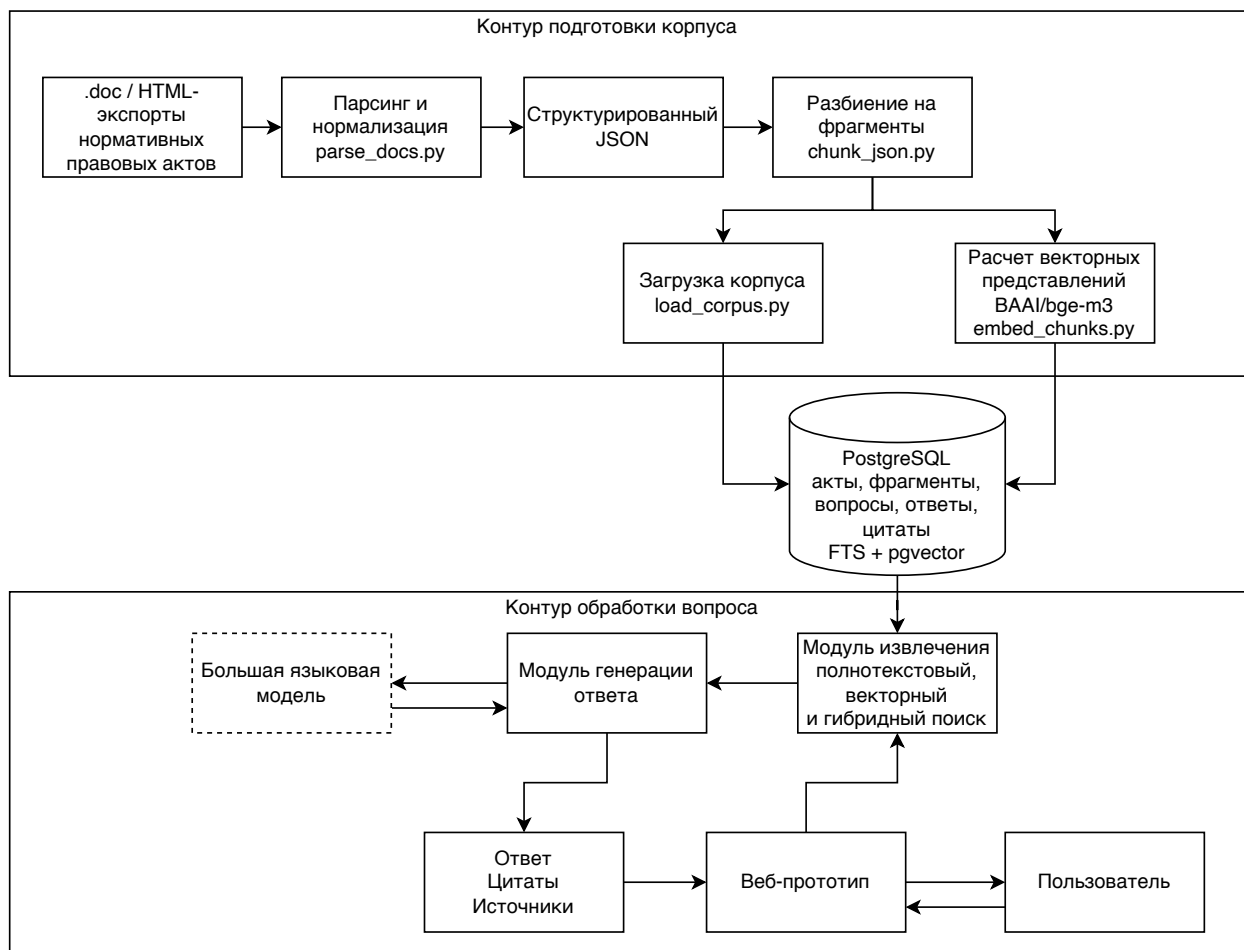


Рисунок 2 – Общая архитектура прототипа RAG-системы вопросно-ответного поиска по нормативным правовым актам Российской Федерации

Основные настройки вынесены в *settings.yaml*. В нем задаются модель векторных представлений *BAAI/bge-m3*, размерность 1024, параметры разбиения, метод извлечения, количество возвращаемых фрагментов, веса гибридного поиска и параметры генерации. Такой подход позволяет менять экспериментальную конфигурацию без изменения программного кода.

2.4 Подготовка локального корпуса нормативных правовых актов

Подготовка корпуса начинается со сценария *scripts/pipeline/parse_docs.py*. Он обрабатывает локальные файлы *.doc*, *.html* и *.htm*, содержащие HTML-представление нормативных актов, и создает нормализованные JSON-файлы. Временные и скрытые файлы, например *~\$*.doc* и файлы с на-

чальной точкой в имени, пропускаются. Если валидных входных файлов нет, сценарий завершается диагностическим сообщением, что предотвращает создание пустого корпуса. Корпус является собственным локальным корпусом из *.doc/HTML*-экспортов и не является автоматически обновляемой промышленной правовой базой.

У каждого итогового JSON есть два верхнеуровневых поля: *act* и *nodes*. Поле *act* содержит извлеченные метаданные акта: ключ, вид акта, тип документа, название, номер, дату, сведения о редакции, статус, исходный файл и систему-источник. Массив *nodes* хранит упорядоченный поток структурных и текстовых узлов: разделы, главы, статьи, преамбулу и абзацы. Узлы сохраняют порядок в исходном документе, структурную ссылку, номер статьи, при наличии номер пункта и исходный HTML-якорь.

Парсер извлекает метаданные не только по фиксированным позициям строк, а по содержанию. Например, дата и номер документа определяются по строке вида «Кодекс Российской Федерации от DD.MM.YYYY № ...», а сведения о редакции — по словам «Редакция», «актуаль» и близким формулировкам. Текст нормализуется: схлопываются лишние пробелы, восстанавливаются номера статей, удаляются технические редакционные элементы, сохраняется нумерация пунктов в *raw_text*. Для разбора HTML используется BeautifulSoup [39]. При нестандартной структуре HTML применяются технические резервные правила извлечения отдельных строк и заголовков; поэтому в работе не предполагается, что все метаданные любых входных документов извлекаются без исключений.

2.5 Разбиение нормативных актов на фрагменты и формирование текстового представления фрагмента

Разбиение выполняет сценарий *scripts/pipeline/chunk_json.py*. Главное правило выбрано исходя из юридической проверяемости: одна статья становится одним фрагментом, если помещается в заданный размер. Если статья

слишком длинная, она делится по целым абзацам. Короткие соседние статьи не объединяются, потому что точная ссылка на статью важнее полного заполнения каждого фрагмента.

Если отдельный абзац превышает заданный размер фрагмента, применяется техническое разбиение по токенам с сохранением границ символов. Такой случай рассматривается как исключение, необходимое для соблюдения ограничения объема контекста.

Текст фрагмента формируется из трех частей: названия акта, структурного пути и текста нормы. Например, фрагмент может включать название кодекса, главу, заголовок статьи и ее пункты. Такой формат используется и для поиска, и для передачи контекста языковой модели, поэтому фрагмент содержит не только текст нормы, но и минимальные сведения, необходимые для проверки источника.

Результатом этапа являются файлы *acts.jsonl*, *chunks.jsonl* и *chunk_manifest.json*. В записях фрагментов сохраняются ключ акта, номер фрагмента, текст, структурная ссылка, номер статьи, диапазон пунктов, якоря источника, границы узлов, число токенов, хеш и диагностические признаки. Параметры размера фрагмента и перекрытия задаются в *settings.yaml*.

2.6 Проектирование базы данных и хранение корпуса в PostgreSQL

Подготовленные данные загружаются в PostgreSQL сценарием *scripts/pipeline/load_corpus.py*. Перед записью сценарий проверяет обязательные поля, уникальность ключей, отсутствие дублей, ссылочную связность актов и фрагментов. При повторной загрузке актов старые фрагменты заменяются, а старые векторные представления сбрасываются, если текст изменился.

Используемая в прототипе схема базы данных включает таблицы *users*, *acts*, *chunks*, *queries*, *answers*, *answer_citations* и *feedback*. Параметры запусков и результаты экспериментальной оценки сохраняются не в базе данных, а

в файловом контуре проекта в виде JSONL- и CSV-файлов. Такой способ выбран для воспроизводимости: файлы можно приложить к работе, повторно обработать скриптами и использовать для построения таблиц и диаграмм.

В таблице *users* хранятся *id*, пользовательский идентификатор *external_uid*, *password_hash*, *display_name*, время создания и время последнего входа. В таблице *acts* хранятся реквизиты нормативных актов, включая стабильный ключ, вид акта, тип документа, название, номер, дату, сведения о редакции, статус, исходный файл и систему-источник. В таблице *chunks* хранятся фрагменты, структурные ссылки, номер статьи, диапазон пунктов, служебные якоря, границы исходных узлов, число токенов, полнотекстовый поисковый вектор, векторное представление, имя модели и хеш текста.

Таблица *queries* хранит пользовательский вопрос, нормализованное представление вопроса, идентификатор пользователя и время создания. Таблица *answers* хранит текст ответа, модель языковой модели, версию шаблона запроса, признак *needs_clarification*, метод извлечения, задержку выполнения, списки *retrieved_chunk_ids* и *dropped_chunk_ids*, а также служебные метаданные ответа. Таблица *answer_citations* связывает ответ с использованными фрагментами и хранит ранг цитаты, оценку релевантности и текст цитаты. Таблица *feedback* связывает *answer_id* и *user_id*, хранит оценку *rating*, комментарий, время создания и имеет уникальное ограничение на пару ответ—пользователь; повторная отправка обратной связи обновляет существующую запись. На уровне предметных сущностей модель данных представлена на рисунке 3.

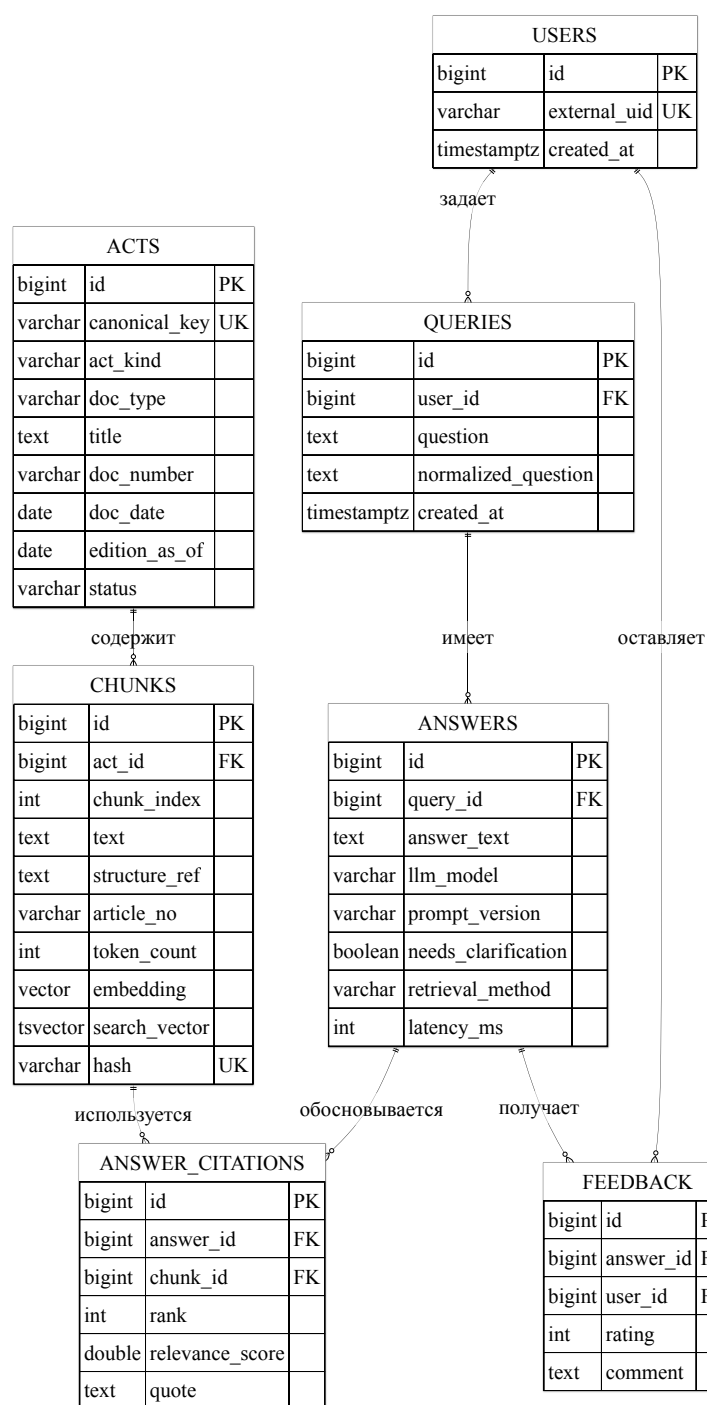


Рисунок 3 – Инфологическая модель базы данных прототипа RAG-системы

Для целостности используются первичные ключи, внешние ключи, ограничения *CHECK*, уникальные ограничения и индексы. Для полнотекстового поиска применяется индекс GIN по *search_vector*, а для векторного поиска — индекс по полю *embedding* через pgvector. Работа с PostgreSQL из программного кода выполняется через Psycopg [40]. Подключение к базе настраивается через переменные окружения и файл *.env*; для локального запуска используется Docker Compose [41].

2.7 Реализация полнотекстового поиска

Полнотекстовый поиск реализован средствами PostgreSQL. В таблице *chunks* поле *search_vector* создается как вычисляемый *tsvector* на основе текста фрагмента с русской конфигурацией. Для поиска используется *websearch_to_tsquery('russian', question)*, что позволяет обрабатывать пользовательский запрос в более привычной форме. Ранжирование выполняется через *ts_rank_cd*, результаты сортируются по убыванию оценки релевантности [7, 21].

Теоретически метод полнотекстового извлечения *sparse* должен быть полезен для запросов с точными юридическими терминами, номерами статей, названиями актов и устойчивыми формулировками. Дополнительно в реализации предусмотрены фильтры по названию акта и номеру статьи. Его качество может снижаться, когда пользователь описывает ситуацию словами, отличающимися от текста закона; это предположение дополнительно проверяется в экспериментальной главе.

2.8 Расчет векторных представлений и реализация векторного поиска через pgvector

Векторные представления рассчитываются сценарием *scripts/pipeline/embed_chunks.py* после загрузки корпуса в базу. Сценарий выбирает только фрагменты, у которых *embedding IS NULL*, поэтому повторный запуск может продолжить расчет без пересчета уже заполненных векторов. Тексты кодируются через *SentenceTransformer*, векторы нормализуются, размерность проверяется по конфигурации, после чего результат записывается в поля *embedding* и *embedding_model* [24]. Модель и размерность берутся из *settings.yaml*; в конфигурации используется *BAAI/bge-m3* и размерность 1024 [10, 11].

Метод векторного извлечения *dense* реализован через *pgvector*. Для вопроса пользователя рассчитывается вектор той же моделью, затем в PostgreSQL выбираются ближайшие фрагменты по косинусному расстоя-

нию [9]. В запросе есть фильтр по *embedding_model*, чтобы вектор вопроса сравнивался только с фрагментами, рассчитанными той же моделью векторных представлений.

Преимущество векторного поиска состоит в возможности находить смыслово близкие фрагменты при несовпадении слов. Ограничение состоит в зависимости результата от качества модели, способа разбиения документов и полноты корпуса. Поэтому векторный поиск не рассматривается как единственный метод, а сравнивается с полнотекстовым и гибридным извлечением.

2.9 Реализация гибридного извлечения релевантных фрагментов

Гибридное извлечение реализовано методом *weighted_hybrid*. Сначала система получает кандидатов из полнотекстового и векторного извлечения. Эти ветви соответствуют методам *sparse* и *dense*, описанным выше. Схема их объединения показана на рисунке 4.

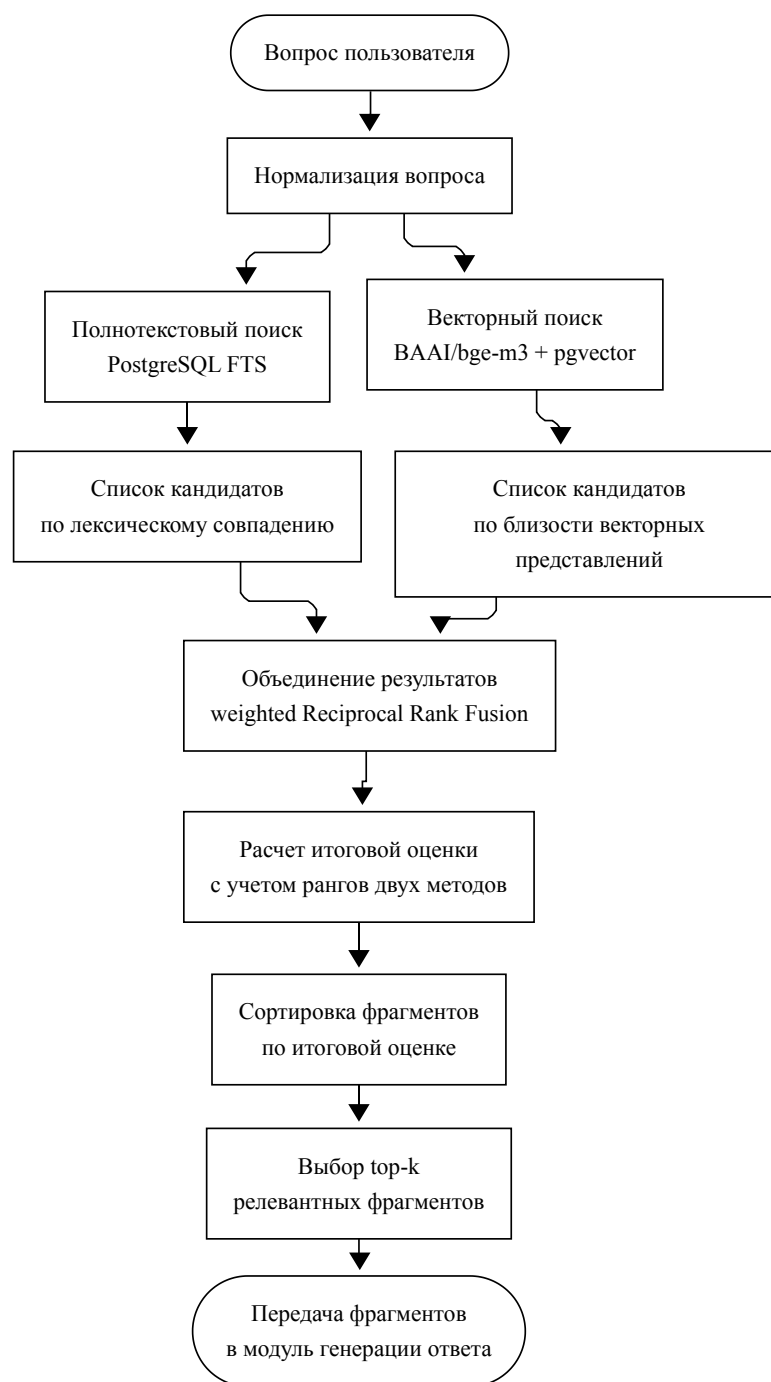


Рисунок 4 – Блок-схема гибридного извлечения релевантных фрагментов нормативных правовых актов

После получения списков кандидатов они объединяются по взвешенной формуле Reciprocal Rank Fusion [23]:

$$score = sparse_weight / (rrf_k + sparse_rank) + dense_weight / (rrf_k + dense_rank)$$

Если фрагмент найден только одним методом, в оценке участвует только соответствующая часть формулы. После объединения берутся первые *top_*

k фрагментов. В конфигурации используются параметры *candidate_limit*, *rrf_k*, *sparse_weight* и *dense_weight*, вынесенные в *settings.yaml*.

Гибридный поиск выбран в качестве основной конфигурации для проверки генерации не как универсально лучший метод, а как метод, который в проведенном эксперименте показал лучшие значения Hit@10 и Recall@10. При этом векторный поиск может быть лучше по Hit@1 и MRR, а гибридный — по полноте в первых десяти результатах в рамках данного набора вопросов.

2.10 Генерация ответа через GigaChat и формирование цитат

Генерация ответа выполняется общим модулем *law_qa_rag.generate_answer*. Он получает вопрос, извлекает фрагменты выбранным методом, применяет ограничение объема контекста, формирует шаблон запроса *answer_v1* и обращается к языковой модели через слой поставщика *LLMProvider*. Текущим внешним поставщиком является GigaChat; документация GigaChat API используется как источник описания внешнего сервиса [12].

Инструкция для языковой модели требует отвечать только по переданному контексту, не использовать внешние знания и вернуть JSON без дополнительного оформления. Ожидаемая структура включает поля *answer*, *used_chunk_ids* и *needs_clarification*. После ответа JSON проверяется на стороне приложения через Pydantic [42]. Идентификаторы *used_chunk_ids* сверяются с фрагментами, которые действительно были переданы в контекст. Если модель указала несуществующий или не переданный фрагмент, результат отклоняется.

Цитаты формируются не языковой моделью, а программно. Для каждого идентификатора из *used_chunk_ids* система берет соответствующий фрагмент из выбранного контекста и сохраняет полный текст фрагмента, название акта, номер, дату, структурную ссылку, статью, диапазон пунктов и оценку релевантности. В текущей версии цитата соответствует полному фрагменту,

а не минимальному предложению внутри него. Если модель возвращает содержательный ответ без *used_chunk_ids*, система заменяет его сообщением о недостаточности контекста и не показывает неподтвержденный правовой ответ. Последовательность формирования ответа и цитат показана на рисунке 5.

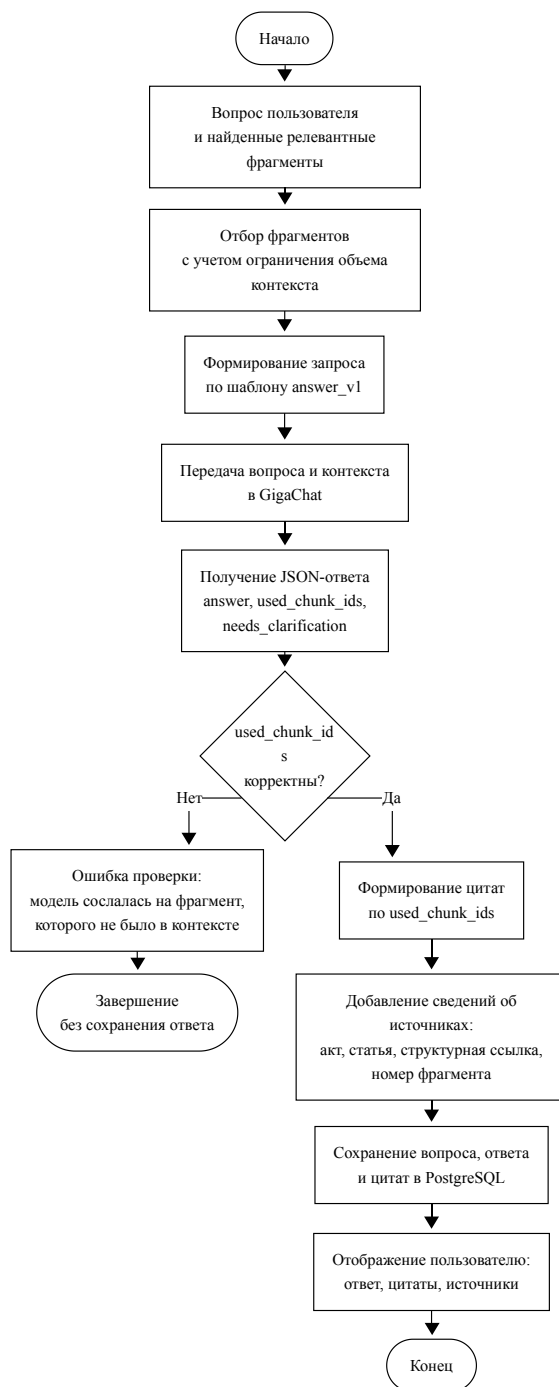


Рисунок 5 – Блок-схема формирования ответа и цитат

2.11 Веб-прототип пользовательского сценария «вопрос — ответ — цитаты — источник»

Веб-прототип реализован в модуле *law_qa_rag.web.app*. Серверная часть построена на FastAPI [43]. HTML-страницы формируются на серверной стороне с использованием шаблонов Jinja2 [44], а клиентская часть содержит только необходимые элементы для отправки вопроса, отображения ответа и работы с формами. Локальный запуск веб-приложения выполняется через ASGI-сервер Uvicorn [45].

Основной веб-сценарий использует регистрацию и вход пользователя. Пароль пользователя не сохраняется в открытом виде: в базе хранится результат хеширования PBKDF2-SHA256 с солью. Такой подход соответствует общим рекомендациям по безопасному хранению паролей [46], а расчет хеша и генерация криптографически значимых случайных значений выполняются средствами стандартной библиотеки Python [47, 48]. Пользовательская сессия используется для сохранения состояния входа между HTTP-запросами. В локальном веб-прототипе это реализуется средствами промежуточного слоя сессий Starlette [49]. Механизм авторизации относится к границам локальной минимально работоспособной версии; промышленная система потребовала бы дополнительной настройки HTTPS, управления секретами, сроков жизни сессий, политики доступа и других мер защиты [50].

В прототипе реализованы следующие страницы и маршруты: главная страница /, отправка вопроса /ask, страница ответа /answers/{answer_id}, страница источника /sources/{act_id}, страница авторизации /auth, регистрация /auth/register, вход /auth/login, выход /auth/logout, профиль /profile и отправка оценки ответа /answers/{answer_id}/feedback. Главная страница показывает форму вопроса и примеры вопросов. Если неавторизованный посетитель пытается отправить вопрос, система предлагает выполнить вход или регистрацию.

Страница ответа показывает вопрос, текст ответа, метод извлечения, модель, версию шаблона запроса, задержку выполнения и список цитат. На этой же странице авторизованный пользователь может отправить обратную связь: оценку и комментарий. Повторная отправка обратной связи обновляет запись для той же пары ответ–пользователь. Страница источника показывает реквизиты нормативного акта и фрагменты, процитированные в ответе; при переходе из ответа процитированные фрагменты выделяются. Профиль показывает данные пользователя и историю его вопросов с переходами к сохраненным ответам. Доступ к странице ответа и к источнику в контексте конкретного ответа ограничивается владельцем ответа.

На страницах отображается предупреждение о том, что ответ строится по редакциям документов, загруженным в локальный корпус, а актуальную редакцию необходимо проверять по официальному источнику. Это связывает пользовательский результат с границами применимости корпуса и снижает риск неверного практического вывода.

Для проверки веб-прототипа подготовлены автоматизированные проверки маршрутов. Они подтверждают отображение формы и примеров вопросов, необходимость входа для отправки вопроса, регистрацию, вход, выход, отображение профиля, загрузку истории, отказ при пустом вопросе, отображение метаданных цитаты, ограничение доступа владельцем ответа, сохранение обратной связи и выделение процитированного фрагмента на странице источника. Эти проверки подтверждают работоспособность критического пользовательского сценария на уровне веб-интерфейса и связаны с требованиями ФТ-10–ФТ-16, НФТ-01 и НФТ-09–НФТ-12.

2.12 Выводы по второй главе

Во второй главе были определены роли пользователей и критический сценарий «вопрос — ответ — цитаты — источник», а также обобщены ключевые группы функциональных и нефункциональных требований к прото-

типу RAG-системы. Полный формальный перечень требований сохранен в техническом задании приложения А.

Описана архитектура прототипа, включающая два контура: подготовку корпуса и обработку пользовательского вопроса. Реализованы парсинг *.doc*/HTML-экспортов в структурированный JSON, разбиение актов на фрагменты по юридически значимым границам, загрузка данных в PostgreSQL и расчет векторных представлений.

Спроектирована схема базы данных, которая хранит пользователей, акты, фрагменты, полнотекстовые и векторные поисковые представления, вопросы, ответы, цитаты и обратную связь. Результаты экспериментальной оценки вынесены в JSONL- и CSV-файлы, что отделяет рабочую схему приложения от материалов оценки. Для извлечения релевантных фрагментов реализованы полнотекстовый, векторный и гибридный методы. Параметры поиска вынесены в конфигурацию, что позволяет сравнивать режимы извлечения без изменения программного кода.

Генерация ответа реализована через GigaChat с требованием вернуть структурированный JSON. Цитаты формируются на стороне приложения по проверенным идентификаторам фрагментов. Разработанный веб-прототип поддерживает регистрацию, вход, выход, профиль пользователя, историю вопросов, обратную связь и основной пользовательский путь «вопрос — ответ — цитаты — источник».

3 ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОТОТИПА

3.1 Цель, гипотезы и ограничения экспериментальной оценки

Целью экспериментальной оценки является проверка того, насколько реализованный прототип RAG-системы способен находить релевантные фрагменты нормативных правовых актов Российской Федерации и формировать по ним проверяемый ответ с цитатами. В отличие от простой демонстрации пользовательского интерфейса, экспериментальная оценка позволяет измерить качество отдельных этапов работы системы: извлечения релевантных фрагментов, выбора фрагментов для ответа и формирования ответа языковой моделью.

В рамках эксперимента оценивается не юридическая консультация как профессиональная услуга, а работоспособность программного прототипа в заданном сценарии: пользователь задает вопрос на русском языке, система находит фрагменты локального корпуса, передает их языковой модели, получает структурированный ответ и формирует цитаты по использованным фрагментам. Такой подход соответствует назначению разработанного прототипа, так как ключевым требованием к системе является проверяемость ответа по первоисточникам.

Экспериментальная оценка была разделена на два уровня. На первом уровне оценивалось извлечение релевантных фрагментов. Для этого сравнивались полнотекстовое, векторное и гибридное извлечение по набору тестовых вопросов и заранее размеченных эталонных фрагментов. На втором уровне проверялась генерация ответа: анализировалось, может ли система на основе найденного контекста сформировать структурированный ответ, выбрать использованные фрагменты и отобразить цитаты.

В ходе работы проверялись следующие гипотезы. Первая гипотеза состоит в том, что полнотекстовое извлечение должно быть полезно для во-

просов с точными юридическими терминами, но может быть недостаточно устойчивым для вопросов, сформулированных бытовым языком. Вторая гипотеза состоит в том, что векторное извлечение должно лучше работать с переформулированными вопросами, так как оно опирается на смысловую близость вопроса и фрагмента. Третья гипотеза состоит в том, что гибридное извлечение должно давать наиболее устойчивый результат в основном RAG-сценарии, поскольку объединяет преимущества полнотекстового и векторного поиска. Четвертая гипотеза состоит в том, что для генерации ответа важнее попадание релевантного фрагмента в первые десять результатов, чем обязательное нахождение его на первой позиции. Поэтому для выбора основной конфигурации особое значение имеют Hit@10 и Recall@10.

Эксперимент имеет ряд ограничений. Корпус является локальным и включает только загруженные редакции нормативных правовых актов. В него входят Конституция Российской Федерации, кодексы Российской Федерации, федеральные законы и федеральные конституционные законы, подготовленные из локальных *.doc*/HTML-экспортов. В корпус не входят подзаконные акты, судебная практика, региональные нормативные акты и разъяснения органов власти. Поэтому вопросы, требующие таких источников, не рассматривались в качестве основных тестовых случаев.

Еще одно ограничение связано с оценкой генерации ответа. Автоматические метрики позволяют проверить попадание эталонных фрагментов в найденный контекст, однако они не доказывают юридическую правильность итогового ответа. Поэтому для генерации была проведена ручная оценка на наборе показательных примеров. В ходе этой оценки проверялась не профессиональная применимость ответа в конкретной правовой ситуации, а его согласованность с найденными фрагментами и корректность цитирования.

Экспериментальные результаты сохранялись в файловом контуре проекта в виде JSONL- и CSV-файлов. Такой способ выбран для воспроизводимости: файлы можно повторно обработать скриптами, приложить к работе и использовать для построения таблиц и диаграмм. Основные результаты на-

ходятся в каталоге *eval/results*. Схема экспериментальной оценки извлечения релевантных фрагментов показана на рисунке 6, а схема проверки генерации ответа и формирования цитат — на рисунке 7.

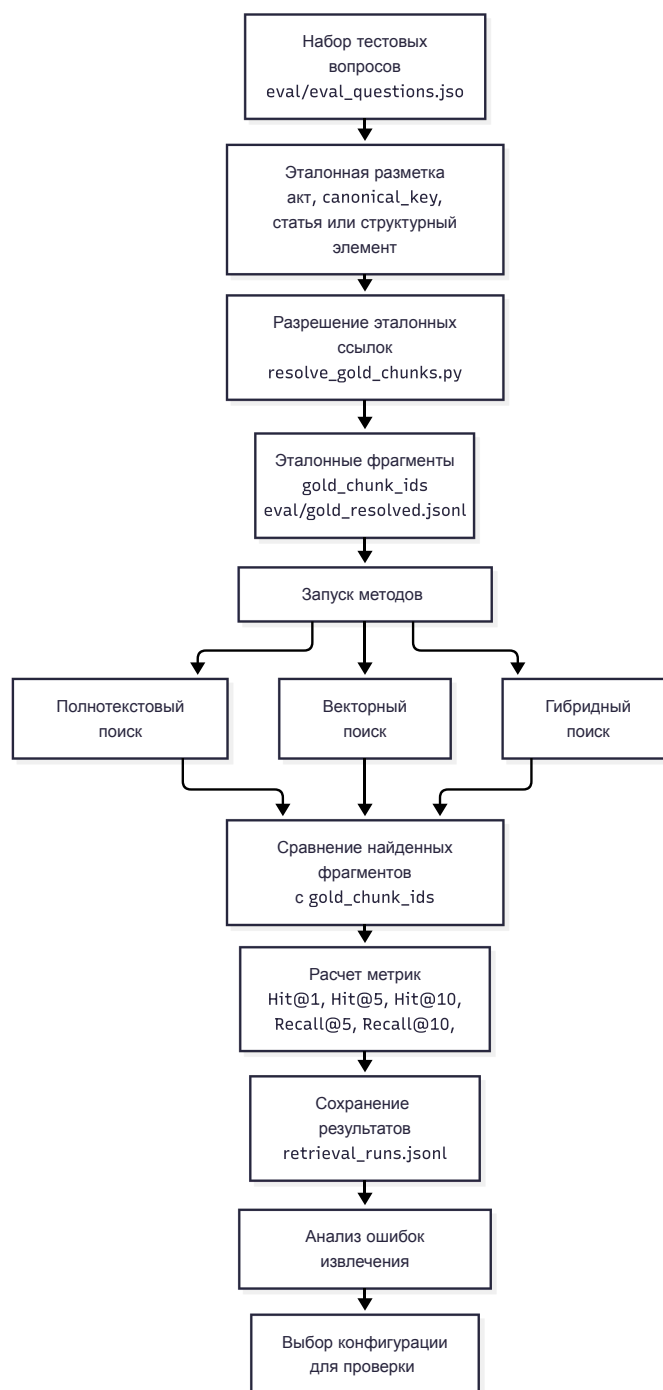


Рисунок 6 – Схема экспериментальной оценки извлечения релевантных фрагментов

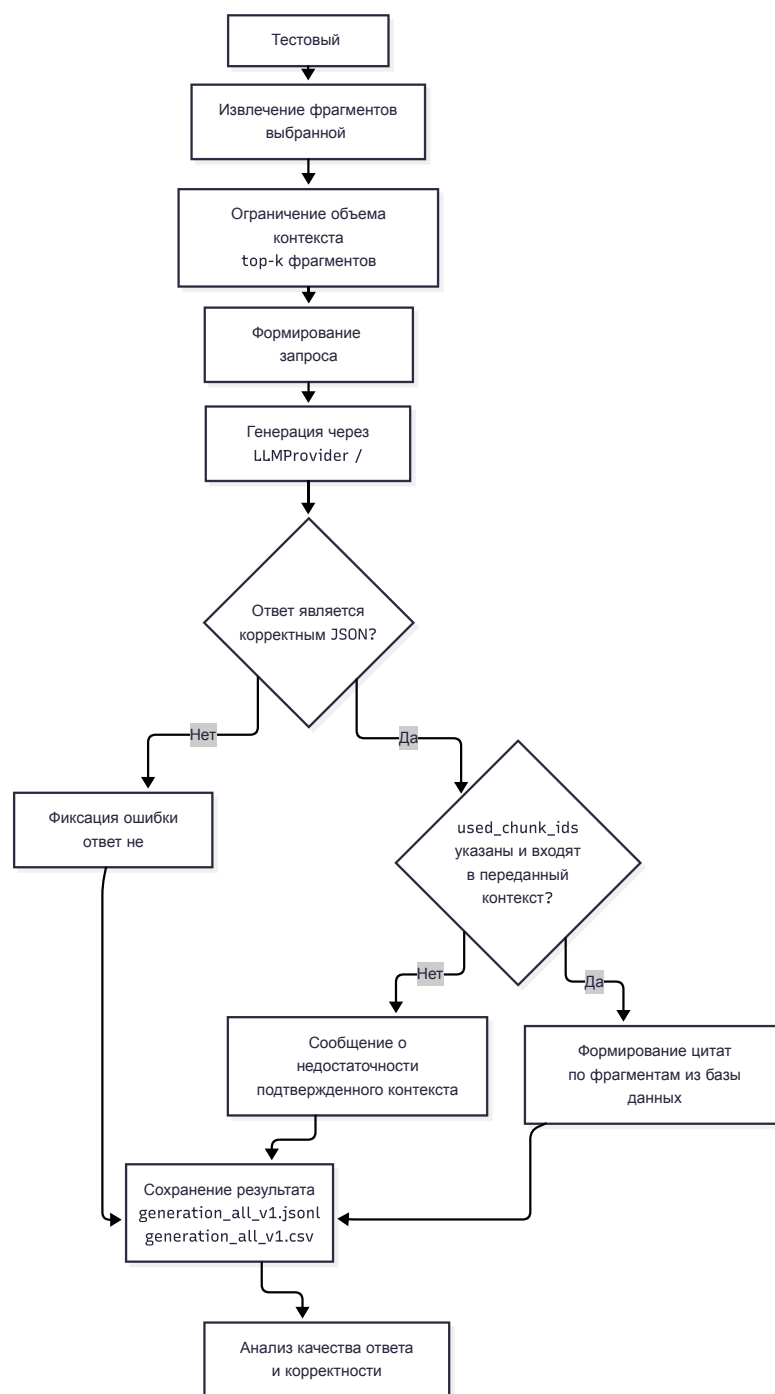


Рисунок 7 – Схема проверки генерации ответа и формирования цитат

3.2 Подготовка набора тестовых вопросов и эталонных релевантных фрагментов

Для оценки был подготовлен набор из 50 тестовых вопросов на русском языке. Вопросы ориентированы на реалистичный пользовательский сценарий юридической вопросно-ответной системы. Они охватывают нормы Конституции Российской Федерации, кодексов Российской Федерации, феде-

ральных законов и федеральных конституционных законов. При составлении набора не включались вопросы, для ответа на которые заведомо требуются подзаконные акты, судебная практика или региональное законодательство.

Набор вопросов включает несколько категорий. Категория *term_exact* содержит вопросы с точными юридическими терминами. Категория *paraphrase* содержит вопросы, сформулированные бытовым языком без точного совпадения с текстом закона. Категория *mixed* объединяет вопросы, в которых есть и бытовая формулировка, и отдельные юридические термины. Категория *multi_fragment* включает вопросы, для ответа на которые требуется несколько релевантных фрагментов. Категория *clarification_needed* содержит вопросы с недостаточным контекстом или вопросами, требующими уточнения условий. Состав тестового набора приведен в таблице 4.

Таблица 4 – Состав тестового набора вопросов

Категория вопросов	Количество вопросов	Назначение категории
<i>term_exact</i>	14	Проверка поиска по точным юридическим терминам и формулировкам
<i>paraphrase</i>	17	Проверка устойчивости к бытовым формулировкам пользователя
<i>mixed</i>	12	Проверка смешанных вопросов, где есть юридические и обычные слова
<i>multi_fragment</i>	5	Проверка сценариев, требующих нескольких нормативных фрагментов
<i>clarification_needed</i>	2	Проверка неоднозначных вопросов и ситуаций недостаточности контекста
Всего	50	Полный набор экспериментальной оценки

Эталонная разметка выполнялась не по идентификаторам фрагментов базы данных, а по устойчивым признакам нормативной нормы: названию акта, *canonical_key* и номеру статьи или структурного элемента. Такой подход выбран потому, что *chunk_id* назначается базой данных при загрузке корпуса и может измениться при повторной загрузке. Поэтому исходная разметка хранит ожидаемый акт и номер статьи, а реальные *chunk_id* определяются отдельным скриптом после загрузки корпуса в PostgreSQL.

Для разрешения эталонных ссылок был использован скрипт *scripts/eval/resolve_gold_chunks.py*. Он сопоставляет записи набора вопросов с таблицами *acts* и *chunks* по *canonical_key*, названию акта и номеру статьи. В результате был сформирован файл *eval/gold_resolved.jsonl*, в котором для каждого вопроса заполнен список *gold_chunk_ids*. Все 50 вопросов были успешно разрешены в реальные фрагменты базы данных, нерешенных случаев не осталось.

В ряде случаев один эталонный вопрос связан с несколькими *chunk_id*. Это связано с правилом разбиения корпуса: обычно одна статья становится одним фрагментом, однако длинные статьи могут быть разделены по абзацам. Такое поведение не является ошибкой, так как сохраняет юридическую структуру текста и позволяет передавать языковой модели фрагменты допустимого размера.

Материалы оценки хранятся в файловом контуре проекта. Набор вопросов находится в *eval/eval_questions.jsonl*, эталонная разметка после разрешения ссылок — в *eval/gold_resolved.jsonl*, результаты оценки извлечения — в каталоге *eval/results/retrieval*, а результаты проверки генерации — в каталоге *eval/results/generation*.

3.3 Метрики оценки извлечения релевантных фрагментов

Оценка извлечения релевантных фрагментов выполнялась как задача ранжирования. Для каждого вопроса система возвращала список из первых десяти найденных фрагментов. Этот список сравнивался с множеством эталонных *gold_chunk_ids*. Если хотя бы один эталонный фрагмент был найден в первых *K* результатах, вопрос считался успешным по метрике *Hit@K*.

Метрика *Hit@K* показывает долю вопросов, для которых хотя бы один эталонный фрагмент присутствует среди первых *K* результатов. В работе использовались значения *Hit@1*, *Hit@5* и *Hit@10*. *Hit@1* показывает качество первого результата, а *Hit@10* отражает пригодность метода для RAG-

сценария, так как именно несколько первых фрагментов передаются далее в контекст языковой модели.

Метрика Recall@K показывает, какая доля всех эталонных фрагментов была найдена среди первых K результатов. Она особенно важна для вопросов, требующих нескольких норм. Например, если вопрос требует одновременно статью о подаче заявления, статью об оформлении увольнения и статью о сроках расчета, то нахождение только одного фрагмента не полностью покрывает потребность пользователя.

Метрика MRR , или средняя обратная позиция первого релевантного результата, показывает, насколько высоко в ранжированном списке расположен первый эталонный фрагмент. Чем ближе релевантный фрагмент к началу списка, тем выше значение MRR . Для пользовательского поиска это важно, потому что фрагменты с более высоким рангом имеют больше шансов попасть в ограниченный контекст языковой модели.

Дополнительно измерялось время выполнения поиска. Для каждого метода фиксировались среднее и медианное время. Значения времени использовались как эксплуатационная характеристика прототипа. При интерпретации времени учитывалось, что эксперимент выполнялся в локальном окружении, поэтому полученные значения не являются промышленным показателем производительности. Используемые метрики относятся к стандартным метрикам оценки ранжированных списков в информационном поиске [20, 25].

3.4 Методика оценки согласованности ответа с найденным контекстом

Оценка генерации ответа выполнялась после выбора основной конфигурации извлечения. Для генерации использовалась гибридная конфигурация с равными весами полнотекстового и векторного поиска, так как она показала наилучшие значения Hit@10 и Recall@10 . На этом этапе проверялся не толь-

ко текст ответа, но и полный пользовательский сценарий: найденные фрагменты, структурированный ответ языковой модели, выбранные *used_chunk_ids* и сформированные цитаты.

В реализованной системе языковая модель не генерирует текст цитат самостоятельно. После извлечения релевантных фрагментов в контекст передаются *chunk_id*, текст фрагмента и его юридические метаданные. Языковая модель должна вернуть JSON-объект, содержащий текст ответа, список *used_chunk_ids* и признак *needs_clarification*. После этого система проверяет, что все идентификаторы из *used_chunk_ids* действительно присутствовали в переданном контексте. Цитаты формируются программно по указанным *used_chunk_ids*.

Такой подход снижает риск выдуманных ссылок. Модель может брать только те фрагменты, которые были переданы ей в контексте, а текст цитаты берется из базы данных. В данной версии цитата соответствует полному фрагменту, а не минимальному предложению внутри него. Это является ограничением прототипа, однако оно сохраняет проверяемость, так как фрагменты сформированы по юридически значимым границам: обычно одна статья соответствует одному фрагменту, а длинные статьи делятся по абзацам.

Для предотвращения неподтвержденных юридических ответов в систему было добавлено правило безопасности: если языковая модель возвращает содержательный ответ, но не указывает ни одного *used_chunk_id*, система заменяет такой ответ сообщением о недостаточности контекста. Это правило необходимо, потому что правовой вывод без цитат противоречит назначению разрабатываемой системы.

Ручная оценка генерации выполнялась по нескольким критериям. Проверялось, найден ли релевантный контекст, правильно ли выбраны *used_chunk_ids*, следует ли ответ из процитированных фрагментов, нет ли в ответе утверждений вне контекста и сохранен ли юридический смысл. Для текста ВКР были подробно проанализированы 15 показательных случаев, включа-

ющих успешные, частично успешные и проблемные ответы. Полный CSV-файл *eval/results/generation/generation_all_v1.csv* подготовлен как машинно-читаемый материал для последующей разметки и проверки. Критерии ручной оценки приведены в таблице 5.

Таблица 5 – Критерии ручной оценки генерации ответа

Критерий	Смысл критерия
Наличие релевантного контекста	Проверяется, попал ли эталонный фрагмент в найденные <i>retrieved_chunk_ids</i>
Корректность <i>used_chunk_ids</i>	Проверяется, выбрала ли модель фрагменты, на которые действительно опирается ответ
Поддержка ответа цитатами	Проверяется, следует ли текст ответа из процитированных фрагментов
Отсутствие лишних утверждений	Проверяется, не добавлены ли сведения, отсутствующие в контексте
Сохранение правового смысла	Проверяется, не искажены ли права, обязанности, сроки и условия
Итоговая оценка	Пример относится к успешным, частично успешным или неуспешным

3.5 Экспериментальная конфигурация полнотекстового извлечения

Полнотекстовое извлечение реализовано средствами PostgreSQL Full-Text Search. Для каждого фрагмента в таблице *chunks* хранится поисковый вектор, сформированный по тексту фрагмента. При обработке пользовательского вопроса система строит полнотекстовый запрос и ранжирует найденные фрагменты средствами PostgreSQL.

В эксперименте полнотекстовое извлечение обозначено как *sparse*. Для него использовались те же значения *top_k* и *candidate_limit*, что и для остальных методов, чтобы сравнение было воспроизводимым. Основная задача этой конфигурации — проверить, насколько хорошо работает лексическое совпадение между вопросом пользователя и текстом нормы.

Полнотекстовый метод должен быть особенно полезен в вопросах с точными юридическими терминами, названиями актов или устойчивыми

формулировками. Однако он может работать хуже в ситуациях, когда пользователь задает вопрос бытовым языком, а соответствующая норма сформулирована специальной юридической лексикой. Такое ограничение особенно важно для вопросно-ответной системы, так как пользователь не всегда знает точную формулировку закона. Параметры полнотекстового извлечения приведены в таблице 6.

Таблица 6 – Параметры полнотекстового извлечения

Параметр	Значение
Метод	<i>sparse</i>
Средство реализации	PostgreSQL Full-Text Search
Количество возвращаемых фрагментов	<i>top_k = 10</i>
Количество кандидатов	<i>candidate_limit = 50</i>
Поле поиска	<i>chunks.search_vector</i>
Назначение в эксперименте	Базовый лексический метод для сравнения

3.6 Экспериментальная конфигурация векторного извлечения

Векторное извлечение основано на сопоставлении векторного представления пользовательского вопроса с векторными представлениями фрагментов корпуса. Векторные представления фрагментов были рассчитаны моделью *BAAI/bge-m3*. Размерность вектора составляет 1024. В PostgreSQL векторы хранятся с использованием расширения *pgvector*.

В эксперименте векторное извлечение обозначено как *dense*. Для каждого вопроса рассчитывалось его векторное представление той же моделью, что использовалась для фрагментов корпуса. Затем выполнялся поиск ближайших фрагментов в векторном пространстве. При поиске дополнительно учитывается имя модели векторных представлений, чтобы сравнивались только совместимые векторы.

Векторный метод должен быть полезен для вопросов, сформулированных не теми словами, которые используются в нормативном тексте. Например, пользователь может спросить: «Когда работодатель должен выплатить

расчет при увольнении?»), тогда как в законе используется формулировка о выплате сумм, причитающихся работнику. В такой ситуации смысловая близость вопроса и нормы может оказаться важнее буквального совпадения слов. Параметры векторного извлечения приведены в таблице 7.

Таблица 7 – Параметры векторного извлечения

Параметр	Значение
Метод	<i>dense</i>
Модель векторных представлений	<i>BAAI/bge-m3</i>
Размерность вектора	1024
Средство хранения и поиска	PostgreSQL + pgvector
Количество возвращаемых фрагментов	<i>top_k = 10</i>
Устройство расчета вопроса	<i>cpu</i> в экспериментальном запуске
Назначение в эксперименте	Проверка смыслового поиска по фрагментам нормативных актов

3.7 Экспериментальная конфигурация гибридного извлечения

Гибридное извлечение объединяет полнотекстовый и векторный методы. В реализованном прототипе используется *weighted Reciprocal Rank Fusion*. Сначала система получает кандидатов от полнотекстового и векторного поиска, после чего объединяет ранжированные списки с учетом весов. Итоговый результат зависит от позиции фрагмента в каждом списке и от заданных весов полнотекстового и векторного компонентов.

В эксперименте сравнивались три варианта гибридного извлечения: с весами 0,4 и 0,6, с весами 0,5 и 0,5, а также с весами 0,3 и 0,7 для полнотекстового и векторного компонентов соответственно. Это позволило проверить, какой баланс между лексическим и смысловым поиском лучше подходит для корпуса нормативных правовых актов.

Гибридный метод важен для юридической вопросно-ответной системы, потому что пользовательские вопросы часто содержат одновременно юридические термины и бытовые формулировки. Полнотекстовый компонент

помогает при совпадении терминов, а векторный компонент помогает при смысловом сходстве без точного совпадения слов. Параметры гибридного извлечения приведены в таблице 8.

Таблица 8 – Параметры гибридного извлечения

Параметр	Значение
Метод	<i>weighted_hybrid</i>
Способ объединения	weighted Reciprocal Rank Fusion
<i>top_k</i>	10
<i>candidate_limit</i>	50
<i>rrf_k</i>	60
Вариант 1	<i>sparse_weight = 0.4, dense_weight = 0.6</i>
Вариант 2	<i>sparse_weight = 0.5, dense_weight = 0.5</i>
Вариант 3	<i>sparse_weight = 0.3, dense_weight = 0.7</i>

3.8 Сравнительный анализ результатов полнотекстового, векторного и гибридного извлечения

Оценка извлечения была выполнена на полном наборе из 50 вопросов. Для каждого вопроса запускались пять конфигураций: полнотекстовое извлечение, векторное извлечение и три варианта гибридного извлечения. Всего было выполнено 250 запусков извлечения релевантных фрагментов.

Сводные результаты приведены в таблице 9. Значения Hit@K, Recall@K и MRR находятся в диапазоне от 0 до 1. Чем выше значение, тем лучше качество извлечения. Время указано в миллисекундах.

Таблица 9 – Сводные результаты оценки извлечения релевантных фрагментов

Конфигурация	Hit@1	Hit@5	Hit@10	Recall@5	Recall@10	MRR	Среднее время, мс	Медианное время, мс
<i>sparse</i>	0,16	0,18	0,18	0,1525	0,1525	0,1640	49,3	24,0
<i>dense</i>	0,82	0,88	0,88	0,8169	0,8436	0,8467	364,0	216,5
<i>weighted_hybrid_s0.4_d0.6</i>	0,66	0,86	0,88	0,7827	0,8302	0,7479	324,7	145,0
<i>weighted_hybrid_s0.5_d0.5</i>	0,64	0,88	0,92	0,8027	0,8652	0,7391	286,1	136,0
<i>weighted_hybrid_s0.3_d0.7</i>	0,66	0,86	0,88	0,7852	0,8336	0,7479	192,4	132,0

Полнотекстовое извлечение показало наименее устойчивый результат. Значение Hit@10 составило 0,18, то есть эталонный фрагмент попадал в первые десять результатов только для 18 % вопросов. Это показывает, что чисто-

го лексического совпадения недостаточно для пользовательских вопросов на естественном языке. Особенно это заметно для вопросов, где пользователь формулирует ситуацию бытовыми словами, а текст закона использует юридическую лексику.

Векторное извлечение показало лучший результат по Hit@1 и MRR. Hit@1 составил 0,82, а MRR — 0,8467. Это означает, что векторный метод часто помещал релевантный фрагмент на первую позицию. Такой результат подтверждает применимость модели *BAAI/bge-m3* для смыслового сопоставления вопросов и фрагментов нормативных актов.

Гибридная конфигурация с равными весами полнотекстового и векторного поиска показала лучший результат по Hit@10 и Recall@10. Значение Hit@10 составило 0,92, а Recall@10 — 0,8652. Для RAG-сценария эти показатели имеют особое значение, так как языковой модели передается не только первый найденный фрагмент, а несколько первых фрагментов контекста. Поэтому именно эта конфигурация была выбрана для дальнейшей проверки генерации ответа.

Графическое сравнение ключевых результатов приведено на рисунках 8, 9 и 10: отдельно показаны Hit@10, Recall@10 и медианное время извлечения релевантных фрагментов.

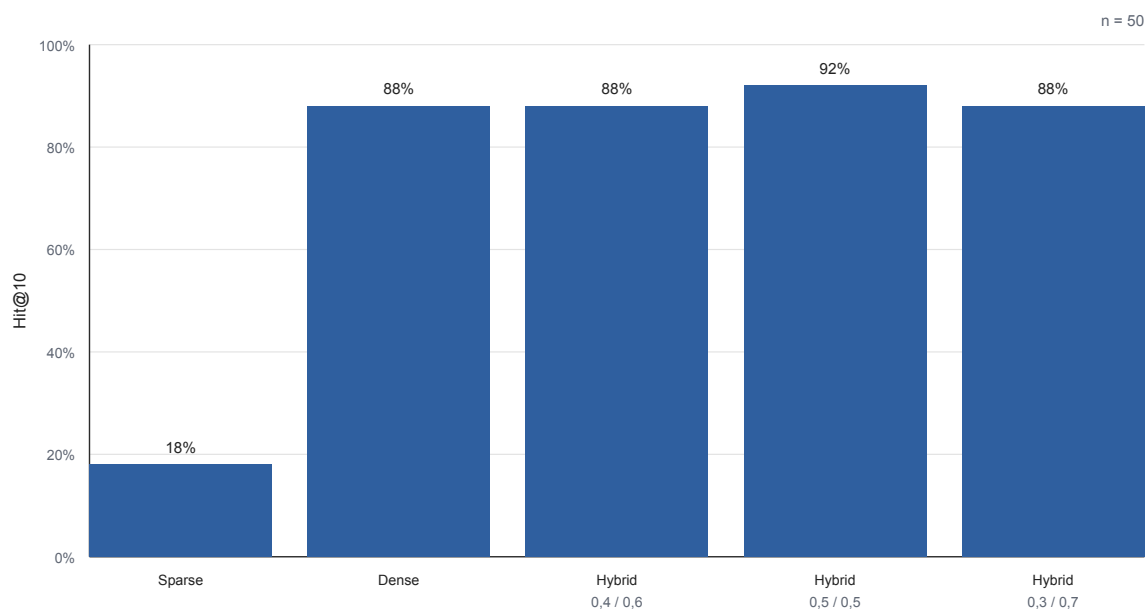


Рисунок 8 – Сравнение Hit@10 для методов извлечения релевантных фрагментов

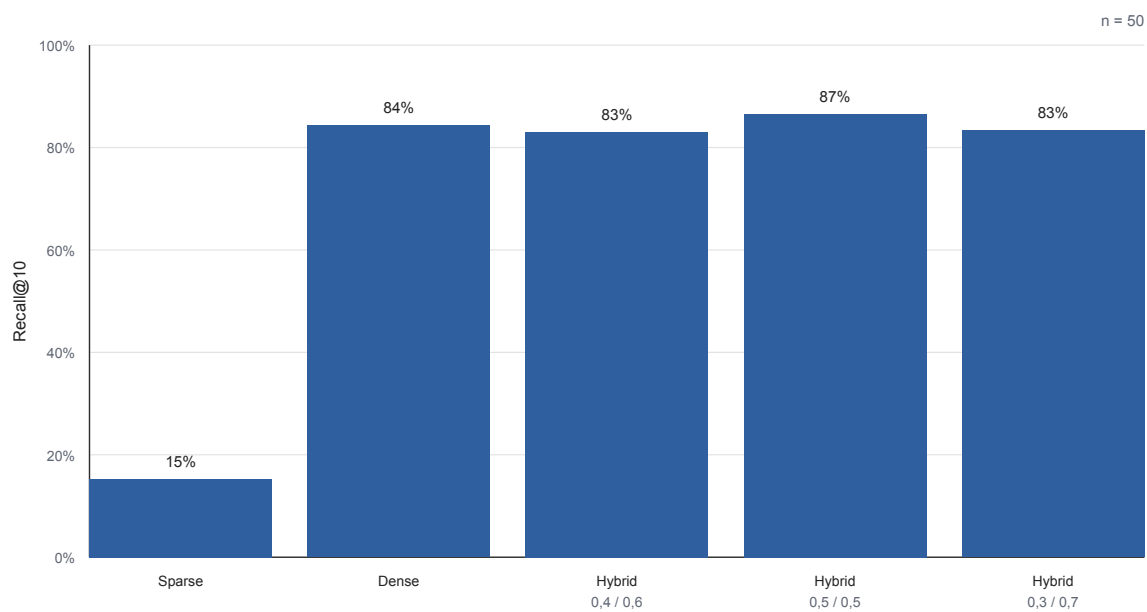


Рисунок 9 – Сравнение Recall@10 для методов извлечения релевантных фрагментов

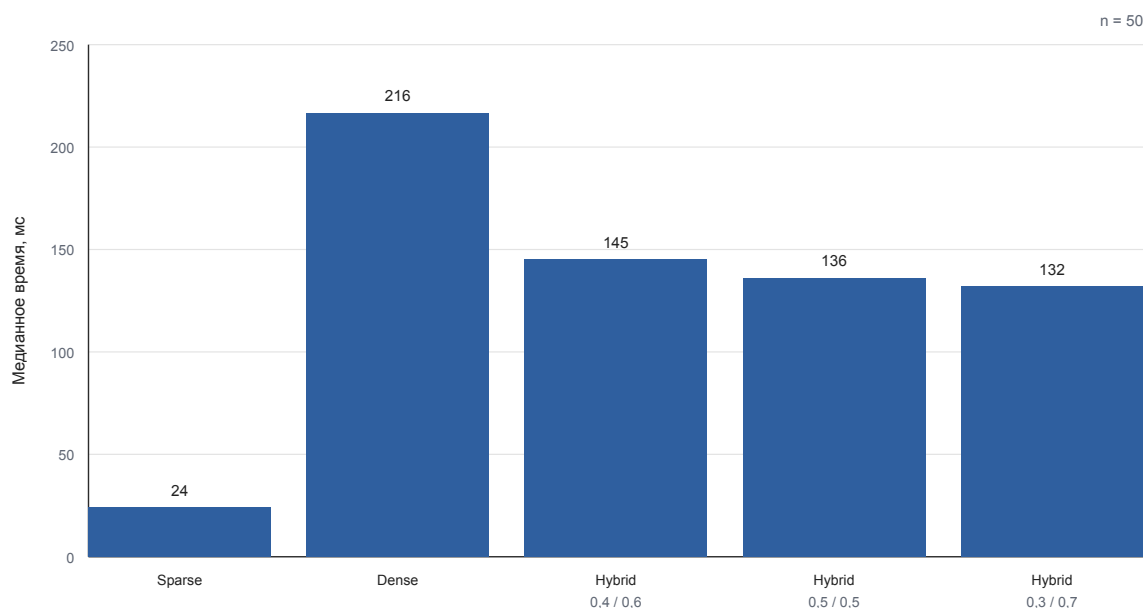


Рисунок 10 – Сравнение медианного времени извлечения релевантных фрагментов

Дополнительно был рассмотрен результат гибридной конфигурации с равными весами, выбранной для проверки генерации по значениям Hit@10 и Recall@10. Данные по категориям вопросов приведены в таблице 10.

Таблица 10 – Результаты *weighted_hybrid_s0.5_d0.5* по категориям вопросов

Категория	Количество вопросов	Hit@10	Recall@10	MRR	Вопросы без попадания эталона в top-10
<i>paraphrase</i>	17	1,00	1,00	0,8791	Нет
<i>mixed</i>	12	1,00	0,9107	0,8036	Нет
<i>multi_fragment</i>	5	1,00	0,90	1,00	Нет
<i>clarification_needed</i>	2	1,00	0,75	0,6667	Нет
<i>term_exact</i>	14	0,7143	0,6667	0,4310	q003, q021, q024, q025

Наиболее сложной оказалась категория *term_exact*. Это может показаться неожиданным, так как точные юридические термины должны помогать полнотекстовому поиску. Однако часть таких вопросов была слишком короткой и содержала общие юридические понятия, например «сделка», «договор купли-продажи» и «договор возмездного оказания услуг». Для таких вопросов в корпусе находится много близких по смыслу или тематически связанных фрагментов, и эталонная статья может уступить место другим нормам.

Результаты подтверждают необходимость отдельной оценки извлечения и генерации. Если релевантная статья не попала в найденный контекст,

языковая модель не должна использовать ее при ответе. Следовательно, ошибка итогового ответа в таких случаях является прежде всего ошибкой извлечения релевантных фрагментов.

3.9 Проверка критического пользовательского сценария

После выбора основной конфигурации извлечения была выполнена проверка полного пользовательского сценария. Для генерации использовалась конфигурация *weighted_hybrid* с весами 0,5 и 0,5. Проверка была выполнена на полном наборе из 50 вопросов. Результаты сохранены в файлах *eval/results/generation/generation_all_v1.jsonl* и *eval/results/generation/generation_all_v1.csv*.

В ходе проверки для каждого вопроса фиксировались найденные *retrieved_chunk_ids*, итоговый ответ, выбранные моделью *used_chunk_ids*, сформированные цитаты, признак *needs_clarification*, задержка выполнения и статус обработки. Это позволило проверить, что система не только находит фрагменты, но и формирует проверяемый ответ с привязкой к источникам.

Сводные результаты проверки генерации приведены в таблице 11.

Таблица 11 – Результаты проверки генерации ответа на 50 вопросах

Показатель	Значение
Количество вопросов	50
Успешно обработано как структурированный ответ	49
Ошибки внешнего поставщика языковой модели	1
Эталонный фрагмент был среди найденных <i>retrieved_chunk_ids</i> для успешных запусков	45 из 49
Эталонный фрагмент был выбран в <i>used_chunk_ids</i> для успешных запусков	43 из 49
Ответы с безопасным сообщением о недостаточности контекста	2
Средняя полная задержка	23,2 с
Медианная полная задержка	20,8 с
Средняя задержка вызова языковой модели	4,2 с
Медианная задержка вызова языковой модели	3,8 с

Из 50 вопросов в 49 случаях система получила структурированный ответ, пригодный для дальнейшей обработки. В одном случае внешний постав-

щик языковой модели вернул не JSON-ответ, а ограничительное сообщение. Система отклонила такой результат как невалидный. Это поведение соответствует требованию не показывать пользователю неподтвержденный правовой ответ без корректных цитат.

В 45 из 49 успешных запусков эталонный фрагмент присутствовал среди найденных фрагментов. В 43 из 49 успешных запусков эталонный фрагмент был также выбран моделью в *used_chunk_ids*. Это показывает, что при наличии релевантного контекста система в большинстве случаев способна сформировать ответ, связанный с правильными источниками.

Для ручной оценки были выбраны 15 показательных примеров. В таблице 12 представлены успешные, частично успешные и проблемные случаи. Оценка выполнялась по тому, найден ли эталонный фрагмент, выбран ли он в *used_chunk_ids*, следует ли ответ из цитат и можно ли считать пример пригодным для демонстрации пользовательского сценария.

Таблица 12 – Ручная оценка показательных примеров генерации ответа

ID	Вопрос	Итоговая оценка	Комментарий
q001	Какие гарантии судебной защиты прав и свобод установлены Конституцией?	Успешно	Использована статья 46 Конституции РФ; ответ следует из процитированного фрагмента
q006	Когда работодатель обязан выплатить расчет при увольнении?	Успешно	Использована статья 140 ТК РФ; ответ корректно отражает срок расчета
q007	Работник хочет уволиться по собственному желанию: как подать заявление, оформить увольнение и когда выплатят деньги?	Частично успешно	Использованы статьи 80, 127 и 140 ТК РФ; ответ покрывает заявление и расчет, но вопрос об оформлении увольнения раскрыт неполно
q014	Когда организация может обрабатывать персональные данные и когда нужно согласие субъекта?	Частично успешно	Использованы статьи 6 и 9 закона о персональных данных; ответ отражает общий смысл, но не перечисляет все основания и условия
q018	В какой срок должны рассмотреть обращение гражданина?	Успешно	Использована статья 12 закона о порядке рассмотрения обращений граждан; ответ содержит общий срок и возможное продление
q023	Я передумал исполнять договор. Можно ли просто отказаться от обязательства в одностороннем порядке?	Частично успешно	Использована статья 310 ГК РФ и связанные нормы; вопрос неоднозначный, поэтому ответ требует осторожной интерпретации
q031	Какие права есть у человека, в отношении которого ведется дело об административном правонарушении?	Успешно	Использована статья 25.1 КоАП РФ; ответ соответствует процитированной норме

Продолжение таблицы 12

ID	Вопрос	Итоговая оценка	Комментарий
q036	Что нужно написать в исковом заявлении в гражданский суд и какие документы приложить?	Успешно	Использованы статьи 131 и 132 ГПК РФ; ответ покрывает содержание заявления и документы
q039	Что должно быть в исковом заявлении в арбитражный суд и какие документы к нему приложить?	Успешно	Использованы статьи 125 и 126 АПК РФ; ответ связан с релевантными фрагментами
q049	Как проверить электронный документ: какие бывают подписи и когда он равнозначен бумажному документу?	Успешно	Использованы статьи 5 и 6 закона об электронной подписи; дополнительные статьи 11 и 12 делают цитирование избыточным, но не противоречат ответу
q003	Какие вопросы относятся к совместному ведению Российской Федерации и субъектов?	Неуспешно	Эталонная статья 72 Конституции РФ не попала в найденный контекст; ответ сформирован по другому акту
q021	Что такое сделка по Гражданскому кодексу?	Безопасный отказ	Эталонная статья 153 ГК РФ не была найдена; система вернула сообщение о недостаточности контекста
q024	Что такое договор купли-продажи?	Безопасный отказ	Эталонная статья 454 ГК РФ не была найдена; система не стала формировать ответ без цитат
q025	Что такое договор возмездного оказания услуг?	Неуспешно	Вместо статьи 779 ГК РФ использован фрагмент закона о защите прав потребителей; цитата не подтверждает точное определение договора
q050	Нужно ли согласие пациента на медицинское вмешательство, если ситуация срочная?	Ошибка поставщика модели	Внешняя языковая модель вернула неструктурированное ограничительное сообщение вместо JSON; результат отклонен системой

Проверка показала, что критический пользовательский сценарий работает для большинства выбранных примеров. В успешных случаях пользователь получает ответ, список цитат и возможность перейти к источнику. В частично успешных случаях релевантные фрагменты найдены, однако ответ требует ручной проверки полноты или учета условий. В неуспешных случаях основная причина связана с отсутствием эталонного фрагмента в найденном контексте или с внешним ограничением языковой модели. Пример страницы ответа с цитатами показан на рисунке 11, страница источника с процитированным фрагментом — на рисунке 12, а пример сообщения о недостаточности контекста — на рисунке 13.

случаях требуется ручная проверка, так как ответ может быть частично верным, но его цитатная база недостаточно точна.

Третья группа связана с неподтвержденными ответами. В ходе отладки было выявлено, что языковая модель иногда могла возвращать содержательный ответ без указания *used_chunk_ids*. Для юридической системы такое поведение недопустимо, потому что пользователь получает правовой вывод без проверяемого источника. Поэтому в систему было добавлено правило: если *used_chunk_ids* пустой, содержательный ответ не принимается, а пользователю возвращается сообщение о недостаточности контекста.

Четвертая группа связана с внешним поставщиком языковой модели. В вопросе q050 модель вернула не JSON-ответ, а ограничительное сообщение. Система обработала такой случай как ошибку структурированного ответа и не сформировала неподтвержденные цитаты. Это показывает, что прототип должен учитывать не только ошибки собственного кода, но и нестабильность внешней языковой модели.

Отдельным ограничением реализации является уровень цитирования. В данной версии прототипа цитата соответствует полному фрагменту, а не минимальному предложению или абзацу внутри него. Такой подход надежен, потому что текст цитаты берется из базы данных, но для длинных статей он может быть менее удобен пользователю. Развитием механизма цитирования может стать выделение релевантного абзаца внутри фрагмента при сохранении ссылки на полный нормативный текст.

Также следует отметить ограничение, связанное с уточняющими вопросами. В прототипе есть признак *needs_clarification*, который позволяет показать, что контекста недостаточно или вопрос требует уточнения. Пользователь может задать новый вопрос, но система не хранит отдельную цепочку диалога вида «исходный вопрос — уточнение — новый ответ». Для минимально работоспособной версии это ограничение допустимо, так как основной сценарий работы системы — один вопрос, один ответ, цитаты и источник.

Основные направления развития представлены в таблице 13.

Таблица 13 – Ограничения реализации и направления развития

Ограничение	Возможное развитие
Ошибки извлечения для коротких терминологических вопросов	Улучшение формулировок запросов, добавление повторного ранжирования, настройка весов гибридного извлечения
Ответ может опираться не на самый точный фрагмент из найденного контекста	Уточнение инструкции для языковой модели и дополнительная проверка <i>used_chunk_ids</i>
Цитата соответствует полному фрагменту	Выделение релевантного абзаца или предложения внутри фрагмента
Нет полноценного уточняющего диалога	Добавление поля уточняющего вопроса и связи между несколькими пользовательскими ходами
Локальный корпус ограничен загруженными актами	Расширение корпуса, контроль актуальности редакций и добавление новых видов источников
Зависимость от внешней языковой модели	Обработка отказов поставщика модели, резервные сценарии и повторный запрос при невалидном формате

В целом устойчивость прототипа можно считать достаточной для минимально работоспособной версии. Система успешно выполняет основной сценарий на большинстве вопросов, а в части проблемных случаев безопасно отказывается от ответа без цитат. При этом результаты эксперимента показывают, что дальнейшее развитие должно быть сосредоточено прежде всего на улучшении извлечения релевантных фрагментов и повышении точности выбора *used_chunk_ids*.

3.11 Выводы по третьей главе

В третьей главе была проведена экспериментальная оценка разработанного прототипа RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации. Для оценки был подготовлен набор из 50 тестовых вопросов и эталонных ссылок на релевантные фрагменты. Эталонная разметка была разрешена в реальные *chunk_id* после загрузки корпуса в PostgreSQL.

Сравнение методов извлечения показало, что полнотекстовое извлечение в текущей конфигурации недостаточно устойчиво для вопросов на естественном языке. Векторное извлечение показало лучший результат по Hit@1 и MRR, то есть чаще помещало релевантный фрагмент на первое место. Гибридная конфигурация с равными весами полнотекстового и векторного поиска показала лучший результат по Hit@10 и Recall@10. Поэтому именно она была выбрана для проверки генерации ответа.

Проверка генерации на 50 вопросах показала, что в 49 случаях система успешно получила структурированный ответ языковой модели. В 45 из 49 успешных запусков эталонный фрагмент присутствовал в найденном контексте, а в 43 из 49 случаев он был выбран моделью в качестве использованного источника. Это подтверждает работоспособность основного сценария «вопрос — ответ — цитаты — источник».

Ручная оценка показательных примеров выявила как успешные, так и проблемные случаи. В успешных случаях ответ следовал из процитированных фрагментов, а пользователь мог проверить его по источнику. В проблемных случаях основная причина была связана с ошибками извлечения, выбором нерелевантного источника или неструктурированным ответом внешней языковой модели. Для предотвращения неподтвержденных юридических ответов в систему добавлено правило безопасного отказа при пустом списке *used_chunk_ids*.

Таким образом, экспериментальная оценка подтвердила, что разработанный прототип реализует заявленную минимально работоспособную версию и позволяет формировать проверяемые ответы по локальному корпусу нормативных правовых актов. При этом результаты показывают направления дальнейшего развития: улучшение извлечения для коротких терминологических вопросов, добавление повторного ранжирования, повышение точности выбора цитируемых фрагментов, развитие механизма уточнения вопроса и расширение корпуса.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы была достигнута поставленная цель: разработан и экспериментально проверен локальный веб-прототип RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации. Подготовлен собственный локальный корпус НПА РФ из .doc/HTML-экспортов, включающий Конституцию Российской Федерации, кодексы Российской Федерации, федеральные законы и федеральные конституционные законы.

В прототипе реализованы парсинг исходных документов в структурированный JSON, разбиение актов на фрагменты с сохранением юридической структуры, загрузка актов и фрагментов в PostgreSQL, полнотекстовый, векторный и гибридный поиск релевантных фрагментов. Векторные представления рассчитываются моделью *BAAI/bge-m3*, а хранение и поиск векторов выполняются через *pgvector*.

Генерация ответа выполняется через слой *LLMProvider*; текущим внешним поставщиком является GigaChat. Ответ языковой модели проверяется как структурированный JSON, а цитаты формируются программно по проверенным *used_chunk_ids*. Такой подход позволяет не доверять модели генерацию текста цитат и связывает ответ с фрагментами локального корпуса. В веб-прототипе реализованы регистрация, вход и выход пользователя, профиль, история вопросов, просмотр собственных ответов, ограничение доступа владельцем ответа и обратная связь по ответу.

Экспериментальная оценка на 50 тестовых вопросах показала, что в данном наборе гибридная конфигурация с равными весами полнотекстового и векторного поиска имеет лучшие значения *Hit@10* и *Recall@10*, тогда как векторный поиск показывает более высокие *Hit@1* и *MRR*. Проверка генерации подтвердила работоспособность основного сценария для большинства вопросов, однако результаты не следует трактовать как гарантию юридической правильности ответа. Система формирует проверяемый ответ по ло-

кальному корпусу и не заменяет профессиональную юридическую консультацию.

Ограничения работы связаны с локальным составом корпуса: в него не включены подзаконные акты, судебная практика, региональное законодательство и разъяснения органов власти. Также сохраняются зависимость от внешней языковой модели, ограничение цитаты уровнем полного фрагмента и отсутствие промышленного защищенного развертывания. Дальнейшее развитие целесообразно направить на обновление корпуса, версионирование редакций актов, добавление повторного ранжирования, выделение более точных цитат внутри фрагмента, расширение тестового набора, повышение устойчивости к ошибкам языковой модели и развитие механизма пользовательской обратной связи.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ji Z., Lee N., Frieske R., Yu T., Su D., Xu Y., Ishii E., Bang Y. J., Madotto A., Fung P. Survey of Hallucination in Natural Language Generation // ACM Computing Surveys. 2023. Vol. 55, no. 12. Article 248. DOI: 10.1145/3571730.
2. Magesh V., Surani F., Dahl M., Suzgun M., Manning C. D., Ho D. E. Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools. arXiv:2405.20362. 2024. URL: <https://arxiv.org/abs/2405.20362> (дата обращения: 11.05.2026).
3. Dahl M., Magesh V., Suzgun M., Ho D. E. Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models // Journal of Legal Analysis. 2024. Vol. 16. P. 64–93. DOI: 10.1093/jla/lae003.
4. Lewis P., Perez E., Piktus A., Petroni F., Karpukhin V., Goyal N., Küttler H., Lewis M., Yih W., Rocktäschel T., Riedel S., Kiela D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems. 2020. Vol. 33. P. 9459–9474. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5f-Abstract.html> (дата обращения: 11.05.2026).
5. Gao Y., Xiong Y., Gao X., Jia K., Pan J., Bi Y., Dai Y., Sun J., Wang M., Wang H. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997. 2023. DOI: 10.48550/arXiv.2312.10997.
6. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond // Foundations and Trends in Information Retrieval. 2009. Vol. 3, no. 4. P. 333–389. DOI: 10.1561/15000000019.
7. PostgreSQL Global Development Group. PostgreSQL Documentation. Chapter 12. Full Text Search. URL: <https://www.postgresql.org/docs/current/textsearch.html> (дата обращения: 11.05.2026).
8. Karpukhin V., Oguz B., Min S., Lewis P., Wu L., Edunov S., Chen D., Yih W. Dense Passage Retrieval for Open-Domain Question Answering // Proceedings

of the 2020 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2020. P. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

9. pgvector. Open-source vector similarity search for Postgres. URL: <https://github.com/pgvector/pgvector> (дата обращения: 11.05.2026).

10. Chen J., Xiao S., Zhang P., Luo K., Lian D., Liu Z. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2402.03216. 2024. DOI: 10.48550/arXiv.2402.03216.

11. BAAI. BAAI/bge-m3 Model Card. Hugging Face. URL: <https://huggingface.co/BAAI/bge-m3> (дата обращения: 11.05.2026).

12. Сбер. GigaChat API. Документация. URL: <https://developers.sber.ru/docs/ru/gigachat/api/overview> (дата обращения: 11.05.2026).

13. Es S., James J., Espinosa-Anke L., Schockaert S. RAGAS: Automated Evaluation of Retrieval Augmented Generation // Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. Stroudsburg: ACL, 2024. P. 150–158. URL: <https://aclanthology.org/2024.eacl-demo.16/> (дата обращения: 11.05.2026).

14. Конституция Российской Федерации: принята всенародным голосованием 12.12.1993. Официальный интернет-портал правовой информации. URL: <https://ips.pravo.gov.ru/api/ips/legislation/document?t?baseid=None&hash=c9693d70b5bc11c97f515e288930b34815497eab8f08a7ad32eaab319b6d262a> (дата обращения: 11.05.2026).

15. Российская Федерация. Федеральный закон от 14.06.1994 № 5-ФЗ «О порядке опубликования и вступления в силу федеральных конституционных законов, федеральных законов, актов палат Федерального Собрания». Официальный интернет-портал правовой информации. URL: <https://ips.pravo.gov.ru/api/ips/legislation/document?baseid=None&hash=e83dd370fd2edeb5519b0a61cc1487cd8143f7828dd5574f3cee5ba4d1ea9291> (дата обращения: 11.05.2026).

16. Российская Федерация. Президент. Указ от 02.04.2014 № 198 «О порядке опубликования законов и иных правовых актов на “Официальном интернет-портале правовой информации” (www.pravo.gov.ru)». URL: <http://publication.pravo.gov.ru/Document/View/0001201404040002> (дата обращения: 11.05.2026).

17. Официальный интернет-портал правовой информации. Государственная система правовой информации. URL: <http://pravo.gov.ru/> (дата обращения: 11.05.2026).

18. Бачило И. Л., Лопатин В. Н., Федотов М. А. Информационное право: учебник / под ред. Б. Н. Топорнина. СПб.: Юридический центр Пресс, 2001. 789 с.

19. Maynez J., Narayan S., Bohnet B., McDonald R. On Faithfulness and Factuality in Abstractive Summarization // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Stroudsburg: ACL, 2020. P. 1906–1919. DOI: 10.18653/v1/2020.acl-main.173.

20. Manning C. D., Raghavan P., Schütze H. Introduction to Information Retrieval. Cambridge: Cambridge University Press, 2008. 482 p. URL: <https://nlp.stanford.edu/IR-book/> (дата обращения: 11.05.2026).

21. PostgreSQL Global Development Group. PostgreSQL Documentation. 12.9. Preferred Index Types for Text Search. URL: <https://www.postgresql.org/docs/current/textsearch-indexes.html> (дата обращения: 11.05.2026).

22. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: ACL, 2019. P. 3982–3992. DOI: 10.18653/v1/D19-1410.

23. Cormack G. V., Clarke C. L. A., Büttcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods // Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. New York: ACM, 2009. P. 758–759. DOI:

10.1145/1571941.1572114.

24. SentenceTransformers Documentation. URL: <https://www.sbert.net/> (дата обращения: 11.05.2026).

25. Voorhees E. M., Harman D. K. TREC: Experiment and Evaluation in Information Retrieval. Cambridge: MIT Press, 2005. 500 p.

26. Järvelin K., Kekäläinen J. Cumulated Gain-Based Evaluation of IR Techniques // ACM Transactions on Information Systems. 2002. Vol. 20, no. 4. P. 422–446. DOI: 10.1145/582415.582418.

27. Moffat A., Zobel J. Rank-Biased Precision for Measurement of Retrieval Effectiveness // ACM Transactions on Information Systems. 2008. Vol. 27, no. 1. Article 2. DOI: 10.1145/1416950.1416952.

28. Pipitone N., Alami M. LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. arXiv:2408.10343. 2024. DOI: 10.48550/arXiv.2408.10343.

29. Guha N., Nyarko J., Ho D. E., Ré C., Chilton A., Chohlas-Wood A., Peters A., Waldon B., Rockmore D. N., Zambrano D., Talisman D., Hoque E., Surani F. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models // Advances in Neural Information Processing Systems. 2023. Vol. 36. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/89e44582fd28ddfea1ea4dcb0b70a397-Abstract-Datasets_and_Benchmarks.html (дата обращения: 11.05.2026).

30. COLIEE. COLIEE 2025. Measuring the Competition Results. URL: <https://coliee.org/COLIEE2025/evaluation> (дата обращения: 11.05.2026).

31. КонсультантПлюс. Официальный сайт справочной правовой системы. URL: <https://www.consultant.ru/> (дата обращения: 11.05.2026).

32. Информационно-правовой портал ГАРАНТ. Официальный сайт. URL: <https://www.garant.ru/> (дата обращения: 11.05.2026).

33. Контур.Норматив. Справочно-правовая система. URL: <https://normativ.kontur.ru/> (дата обращения: 11.05.2026).

34. Yandex Cloud Marketplace. Нейроюрист. URL: <https://yandex.cloud/ru/marketplace/products/yc/neurolegal> (дата обращения: 11.05.2026).
35. Harvey. AI platform for law firms and professional services. URL: <https://www.harvey.ai/> (дата обращения: 11.05.2026).
36. Thomson Reuters. CoCounsel Legal. URL: <https://legal.thomsonreuters.com/en/products/cocounsel-legal> (дата обращения: 11.05.2026).
37. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO, 2018.
38. Куликов С. Тестирование программного обеспечения. Базовый курс. 3-е изд. Версия 3.2.6 от 24.05.2024. URL: https://svyatoslav.biz/software_testing_book/ (дата обращения: 11.05.2026).
39. Richardson L. Beautiful Soup Documentation. Beautiful Soup 4.14.3 documentation. URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> (дата обращения: 11.05.2026).
40. Psycorg. Psycorg 3 documentation. URL: <https://www.psycorg.org/psycorg3/docs/> (дата обращения: 11.05.2026).
41. Docker Inc. Docker Compose documentation. URL: <https://docs.docker.com/compose/> (дата обращения: 11.05.2026).
42. Pydantic. Documentation. URL: <https://pydantic.dev/docs/> (дата обращения: 11.05.2026).
43. FastAPI. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата обращения: 11.05.2026).
44. Pallets Projects. Jinja Documentation. URL: <https://jinja.palletsprojects.com/> (дата обращения: 14.05.2026).
45. Uvicorn Documentation. URL: <https://uvicorn.dev/> (дата обращения: 14.05.2026).

46. OWASP Foundation. Password Storage Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 14.05.2026).

47. Python Software Foundation. hashlib — Secure hashes and message digests. Python documentation. URL: <https://docs.python.org/3/library/hashlib.html> (дата обращения: 14.05.2026).

48. Python Software Foundation. secrets — Generate secure random numbers for managing secrets. Python documentation. URL: <https://docs.python.org/3/library/secrets.html> (дата обращения: 14.05.2026).

49. Starlette. Middleware. SessionMiddleware. URL: <https://www.starlette.io/middleware/> (дата обращения: 14.05.2026).

50. OWASP Foundation. Authentication Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 14.05.2026).

51. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. М.: Издательство стандартов, 1978.

52. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. М.: Стандартинформ, 2019.

53. ГОСТ 19.301-79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. М.: Издательство стандартов, 1979.

ПРИЛОЖЕНИЕ А. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Настоящее техническое задание подготовлено для локального прототипа RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации. Структура приложения приближена к требованиям ЕСПД к техническому заданию [51] и учитывает общие требования к оформлению текстовых документов [52].

1. Общие сведения

Наименование разработки: локальный веб-прототип RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации.

Условное обозначение разработки: *law-qa-rag*.

Разработчик: студент, выполняющий выпускную квалификационную работу по теме «Разработка RAG-системы вопрос–ответ по корпусу нормативных правовых актов Российской Федерации».

Область применения: исследовательский и учебный локальный прототип, предназначенный для проверки сценария «вопрос — ответ — цитаты — источник» на собственном корпусе нормативных правовых актов Российской Федерации. Система не является юридической консультацией и не заменяет проверку актуальной редакции акта по официальному источнику.

В минимально работоспособной версии система работает с локальным корпусом документов, подготовленным из *.doc*, *.html* и *.htm* HTML-экспортов нормативных правовых актов. Корпус включает Конституцию Российской Федерации, кодексы Российской Федерации, федеральные законы и федеральные конституционные законы. В корпус не включаются подзаконные акты, судебная практика, региональные нормативные акты и разъяснения органов власти.

2. Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу по теме «Разработка RAG-системы вопрос–ответ по корпусу нормативных правовых актов Российской Федерации». Разработка выполняется в целях создания программного прототипа, позволяющего по вопросу пользователя извлекать релевантные фрагменты нормативных актов, формировать ответ с использованием найденного контекста и показывать цитаты из первоисточников.

3. Назначение разработки

Назначение системы состоит в предоставлении авторизованному пользователю проверяемого ответа на вопрос по локальному корпусу нормативных правовых актов. Пользователь вводит вопрос на естественном языке, после чего система выполняет полнотекстовое, векторное или гибридное извлечение релевантных фрагментов, формирует ответ через поставщика языковой модели и показывает цитаты, сформированные по использованным фрагментам из базы данных.

В системе предусматриваются следующие роли: неавторизованный посетитель, авторизованный пользователь, разработчик и исследователь качества. Неавторизованный посетитель может открыть веб-прототип и ознакомиться с назначением системы, но для отправки вопроса должен пройти регистрацию или вход. Авторизованный пользователь задает вопросы, получает ответы, открывает источники, просматривает профиль и историю вопросов, а также оставляет обратную связь по ответу. Разработчик подготавливает корпус, выполняет загрузку данных и расчет векторных представлений. Исследователь качества запускает экспериментальную оценку и анализирует результаты в файлах *JSONL*, *CSV* и *YAML*.

4. Требования к системе

Функциональные требования к системе приведены в таблице А.1.

Таблица А.1 – Функциональные требования к системе

идентификатор	требование	приоритет	критерий проверки
ФТ-01	Система должна обрабатывать локальные <i>.doc</i> , <i>.html</i> и <i>.htm</i> HTML-экспорты нормативных правовых актов.	Высокий	Для валидного входного файла создается структурированный JSON с полями <i>act</i> и <i>nodes</i> ; временные и скрытые файлы пропускаются.
ФТ-02	Система должна извлекать и сохранять метаданные нормативного акта.	Высокий	В результате парсинга присутствуют <i>canonical_key</i> , <i>act_kind</i> , <i>doc_type</i> , <i>title</i> , <i>doc_number</i> , <i>doc_date</i> , <i>edition_as_of</i> , <i>status</i> , <i>source_file</i> , <i>source_system</i> .
ФТ-03	Система должна сохранять внутреннюю юридическую структуру акта.	Высокий	В узлах документа сохраняются структурные элементы, статья, пункт, структурная ссылка и исходный якорь при наличии.
ФТ-04	Система должна разбивать акты на фрагменты по юридически значимым границам.	Высокий	Статья формирует один фрагмент, если помещается в заданный размер; длинная статья делится по целым абзацам или технически по токенам при превышении лимита.
ФТ-05	Система должна формировать текстовое представление фрагмента.	Высокий	Текст фрагмента включает название акта, структурный путь и текст нормы, если соответствующие параметры включены в <i>settings.yaml</i> .
ФТ-06	Система должна создавать файлы подготовленного корпуса.	Высокий	После этапа разбиения создаются <i>acts.jsonl</i> , <i>chunks.jsonl</i> и <i>chunk_manifest.json</i> .
ФТ-07	Система должна загружать акты и фрагменты в PostgreSQL.	Высокий	Таблицы <i>acts</i> и <i>chunks</i> заполнены; скрипт проверяет обязательные поля, дубли и ссылочную связность.
ФТ-08	Система должна выполнять расчет векторных представлений фрагментов моделью <i>BAAI/bge-m3</i> .	Высокий	Для фрагментов с <i>embedding IS NULL</i> рассчитываются векторы размерности 1024 и заполняются поля <i>embedding</i> и <i>embedding_model</i> .
ФТ-09	Система должна поддерживать полнотекстовый поиск PostgreSQL.	Высокий	По вопросу пользователя возвращается ранжированный список фрагментов на основе <i>search_vector</i> и <i>websearch_to_tsquery</i> .
ФТ-10	Система должна поддерживать векторный поиск через <i>pgvector</i> .	Высокий	По вектору вопроса возвращаются ближайшие фрагменты с тем же значением <i>embedding_model</i> .
ФТ-11	Система должна поддерживать гибридное извлечение <i>weighted_hybrid</i> .	Высокий	Результаты полнотекстового и векторного поиска объединяются по взвешенной формуле Reciprocal Rank Fusion.
ФТ-12	Система должна ограничивать объем контекста, передаваемого языковой модели.	Средний	Низкоранговые фрагменты, не помещающиеся в бюджет контекста, фиксируются в <i>dropped_chunk_ids</i> .
ФТ-13	Система должна формировать ответ через <i>LLMProvider</i> с текущим поставщиком <i>GigaChat</i> .	Высокий	Модулю генерации передаются вопрос и выбранные фрагменты; результат возвращается в виде структурированного JSON.

Продолжение таблицы А.1

идентификатор	требование	приоритет	критерий проверки
ФТ-14	Система должна проверять структуру ответа языковой модели.	Высокий	Ответ принимается только при наличии полей <i>answer</i> , <i>used_chunk_ids</i> , <i>needs_clarification</i> ; невалидный JSON отклоняется.
ФТ-15	Система должна проверять <i>used_chunk_ids</i> .	Высокий	Каждый идентификатор из <i>used_chunk_ids</i> должен входить в набор фрагментов, переданный модели в контексте.
ФТ-16	Система должна программно формировать цитаты по использованным фрагментам.	Высокий	Цитаты создаются по данным фрагментов из базы, а не по произвольным ссылкам, возвращенным языковой моделью.
ФТ-17	Система должна сохранять успешные вопросы, ответы и цитаты.	Средний	После успешного ответа создаются связанные записи в <i>queries</i> , <i>answers</i> и <i>answer_citations</i> .
ФТ-18	Система должна предоставлять веб-прототип основного пользовательского сценария.	Высокий	Доступны главная страница, отправка вопроса, страница ответа и страница источника.
ФТ-19	Система должна поддерживать регистрацию пользователя.	Высокий	Пользователь может создать учетную запись; пароль не сохраняется в открытом виде.
ФТ-20	Система должна поддерживать вход и выход пользователя.	Высокий	После входа создается пользовательская сессия; после выхода сессия очищается.
ФТ-21	Система должна предоставлять профиль пользователя.	Средний	Авторизованный пользователь видит данные своего профиля и элементы управления выходом.
ФТ-22	Система должна отображать историю вопросов пользователя.	Средний	В профиле отображаются только вопросы текущего пользователя и ссылки на его ответы.
ФТ-23	Система должна поддерживать обратную связь по ответу.	Средний	Пользователь может поставить оценку от 1 до 5 и оставить комментарий; повторная отправка обновляет запись.
ФТ-24	Система должна обрабатывать пустой пользовательский вопрос.	Высокий	При пустом вопросе поиск, генерация и сохранение ответа не выполняются; пользователю выводится диагностическое сообщение.
ФТ-25	Система должна поддерживать экспериментальную оценку извлечения релевантных фрагментов.	Высокий	Скрипты оценки создают <i>retrieval_runs.jsonl</i> , <i>summary_metrics.csv</i> , <i>error_analysis.csv</i> и <i>config_snapshot.yaml</i> .
ФТ-26	Система должна поддерживать проверку генерации ответа на тестовом наборе.	Средний	Скрипт оценки создает <i>generation_all_v1.jsonl</i> и <i>generation_all_v1.csv</i> ; ручные поля в CSV могут заполняться отдельно.

Нефункциональные требования к системе приведены в таблице А.2.

Таблица А.2 – Нефункциональные требования к системе

идентификатор	требование	приоритет	критерий проверки
НФТ-01	Ответ должен быть проверяемым по цитатам.	Высокий	На странице ответа отображаются цитаты с реквизитами акта, структурной ссылкой и текстом фрагмента либо сообщение о недостаточности контекста.

Продолжение таблицы А.2

идентификатор	требование	приоритет	критерий проверки
НФТ-02	Подготовка корпуса должна быть воспроизводимой.	Высокий	Парсинг, разбиение, загрузка и расчет векторных представлений запускаются отдельными скриптами с параметрами из <i>settings.yaml</i> .
НФТ-03	Юридическая структура фрагмента должна сохраняться.	Высокий	Во фрагменте сохраняются название акта, структурная ссылка, статья, диапазон пунктов и исходные якоря при наличии.
НФТ-04	Секретные параметры должны храниться вне программного кода.	Высокий	Пароль PostgreSQL, секрет сессии и учетные данные GigaChat задаются через <i>.env</i> или переменные окружения.
НФТ-05	Пароль пользователя не должен храниться в открытом виде.	Высокий	В таблице <i>users</i> хранится <i>password_hash</i> , сформированный с солью.
НФТ-06	Доступ к сохраненным ответам должен быть ограничен владельцем.	Высокий	При открытии ответа и источника в контексте ответа проверяется <i>user_id</i> владельца.
НФТ-07	Экспериментальная оценка должна быть воспроизводимой.	Высокий	Входные вопросы, разрешенная разметка, результаты и снимки конфигурации сохраняются в <i>JSONL</i> , <i>CSV</i> и <i>YAML</i> .
НФТ-08	Ошибочные ситуации должны сопровождаться диагностическими сообщениями.	Средний	При отсутствии входных файлов, пустом вопросе, ошибке базы данных или невалидном JSON выводится причина без раскрытия секретов.
НФТ-09	Система должна иметь локальный характер минимально работоспособной версии.	Высокий	Запуск выполняется в локальной среде; промышленное развертывание, отказоустойчивость и эксплуатационный мониторинг не входят в состав MVP.
НФТ-10	Параметры поиска и генерации должны изменяться без правки кода.	Средний	Метод извлечения, <i>top_k</i> , веса <i>weighted_hybrid</i> , модель и ограничения контекста задаются в <i>settings.yaml</i> .
НФТ-11	Целостность данных должна обеспечиваться ограничениями базы данных.	Высокий	Схема содержит первичные ключи, внешние ключи, <i>CHECK</i> , <i>UNIQUE</i> и индексы.
НФТ-12	Материалы экспериментов не должны смешиваться с рабочей моделью данных.	Высокий	Актуальная схема не использует таблицы <i>experiments</i> и <i>experiment_runs</i> ; результаты оценки хранятся в <i>eval/results</i> .

5. Требования к составу и параметрам технических средств

Для локального запуска прототипа требуется персональный компьютер или сервер разработчика, способный запускать Python-приложение, контейнер PostgreSQL и модель векторных представлений. Минимальная конфигурация определяется числом фрагментов корпуса и выбранным устройством расчета векторов. В базовом локальном режиме допускается расчет на цен-

тральном процессоре; при наличии CUDA или MPS может быть выбран соответствующий ускоритель.

Для хранения данных требуется дисковое пространство для локального корпуса, файлов *JSONL*, *CSV*, *YAML*, базы PostgreSQL и кэша модели *BAAI/bge-m3*. Для обращения к GigaChat требуется сетевой доступ к внешнему сервису и корректно заданные учетные данные. Промышленная инфраструктура, балансировка нагрузки, резервирование и эксплуатационный мониторинг в состав MVP не входят.

6. Требования к информационному обеспечению

Информационное обеспечение системы включает локальный корпус нормативных правовых актов Российской Федерации, подготовленные промежуточные файлы и базу данных. Исходные данные представляют собой *.doc*, *.html* и *.htm* HTML-экспорты нормативных актов. Подготовленные файлы включают структурированный JSON после парсинга, *acts.jsonl*, *chunks.jsonl* и *chunk_manifest.json* после разбиения.

В базе данных хранятся пользователи, акты, фрагменты, пользовательские вопросы, ответы, цитаты и обратная связь. Актуальная схема включает таблицы *users*, *acts*, *chunks*, *queries*, *answers*, *answer_citations* и *feedback*. Таблицы *experiments* и *experiment_runs* не входят в текущую модель данных и не должны описываться как часть актуальной схемы.

Результаты экспериментальной оценки хранятся в файловом контуре проекта в каталоге *eval/results* в форматах *JSONL*, *CSV* и *YAML*. Такой способ позволяет повторно анализировать результаты без изменения рабочей схемы базы данных.

7. Требования к программному обеспечению

Для локального запуска требуются Python версии не ниже 3.11, PostgreSQL с расширением *pgvector*, Docker Compose для запуска контей-

нера базы данных, а также зависимости, перечисленные в *requirements.txt* и *pyproject.toml*. Серверная часть веб-прототипа реализована с использованием FastAPI, Uvicorn и Jinja2. Клиентская часть представлена HTML-шаблонами, стилями и минимальным JavaScript-кодом для работы форм.

Для подготовки корпуса используются скрипты каталога *scripts/pipeline*. Для экспериментальной оценки используются скрипты каталога *scripts/eval*. Основная программная логика поиска, генерации, сохранения результатов и веб-прототипа размещена в пакете *law_qa_rag*.

Секретные параметры не должны включаться в исходный код и документацию. В инструкциях запуска допускается использовать только шаблоны вида *POSTGRES_PASSWORD=...*, *SESSION_SECRET=...*, *GIGACHAT_CREDENTIALS=...*

8. Стадии и этапы разработки

Разработка выполняется по этапам, связанным с задачами выпускной квалификационной работы. Состав этапов приведен в таблице А.3.

Таблица А.3 – Стадии и этапы разработки

этап	наименование	результат
1	Анализ предметной области	Определение особенностей вопросно-ответного поиска по нормативным правовым актам и требований к проверяемости ответа.
2	Постановка требований	Формирование функциональных и нефункциональных требований, ограничений MVP и критериев проверки.
3	Проектирование архитектуры	Определение контуров подготовки корпуса, извлечения фрагментов, генерации ответа и веб-сценария.
4	Проектирование базы данных	Описание сущностей <i>users</i> , <i>acts</i> , <i>chunks</i> , <i>queries</i> , <i>answers</i> , <i>answer_citations</i> , <i>feedback</i> .
5	Подготовка корпуса	Парсинг исходных документов, разбиение актов на фрагменты и формирование файлов <i>acts.jsonl</i> и <i>chunks.jsonl</i> .
6	Реализация поиска и генерации	Загрузка данных в PostgreSQL, расчет векторов, реализация полнотекстового, векторного и гибридного извлечения, генерация ответа.
7	Реализация веб-прототипа	Регистрация, вход, выход, профиль, история вопросов, страница ответа, страница источника и обратная связь.
8	Экспериментальная оценка	Запуск оценки извлечения релевантных фрагментов и проверки генерации на наборе тестовых вопросов.
9	Документирование	Подготовка пояснительной записки, приложений, программы испытаний и руководства по локальному запуску.

9. Порядок контроля и приемки

Контроль разработки выполняется в локальной среде MVP. Проверка включает контроль подготовленного корпуса, проверку схемы базы данных, запуск модулей поиска, запуск генерации ответа, проверку формирования цитат, проверку веб-маршрутов и запуск экспериментальной оценки.

Приемка прототипа выполняется по программе и методике испытаний, приведенной в приложении Б. Система считается соответствующей заданию, если основные функции подготовки корпуса, поиска, генерации ответа, регистрации пользователя, истории вопросов и обратной связи выполняются в локальной среде, а ответы сопровождаются проверяемыми цитатами либо безопасным сообщением о недостаточности контекста.

10. Ограничения MVP

Минимально работоспособная версия имеет следующие ограничения. Система работает только с собственным локальным корпусом и не выполняет автоматическое обновление редакций актов. Ответ ограничен редакциями документов, загруженными в локальный корпус. Прототип не является юридической консультацией и не заменяет обращение к профессиональному юристу или проверку официального источника.

В корпус не входят подзаконные акты, судебная практика, региональные нормативные акты и разъяснения органов власти. Промышленное развертывание, администрирование ролей, восстановление пароля, подтверждение электронной почты, интеграция с внешними поставщиками авторизации, мониторинг и резервное копирование не входят в MVP. Экспериментальная оценка хранится в файлах *JSONL*, *CSV* и *YAML*; таблицы *experiments* и *experiment_runs* не являются частью актуальной схемы базы данных.

ПРИЛОЖЕНИЕ Б. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Настоящее приложение подготовлено по структуре, приближенной к ЕСПД и требованиям ГОСТ 19.301-79 к программе и методике испытаний [53]. Испытания предназначены для проверки локального прототипа, а не промышленной информационной системы.

1. Объект испытаний

Объектом испытаний является локальный MVP RAG-системы вопросно-ответного поиска по корпусу нормативных правовых актов Российской Федерации. Проверяется программный прототип, включающий подготовку корпуса, загрузку данных в PostgreSQL, полнотекстовое, векторное и гибридное извлечение релевантных фрагментов, генерацию ответа через *LLMProvider* и GigaChat, формирование цитат, веб-прототип, регистрацию, вход, выход, профиль пользователя, историю вопросов и обратную связь.

2. Цель испытаний

Цель испытаний состоит в подтверждении соответствия прототипа функциональным и нефункциональным требованиям, приведенным в приложении А. Отдельно проверяется, что ответ системы может быть прослежен до фрагментов нормативных актов, что *used_chunk_ids* не выходят за пределы переданного контекста, а неподтвержденные ответы не сопровождаются произвольными цитатами.

3. Условия проведения испытаний

Испытания проводятся в локальной среде разработчика. Предполагается наличие установленного Python-окружения, контейнера PostgreSQL с рас-

ширением *pgvector*, заполненного файла *.env*, схемы базы данных и подготовленного корпуса. Для проверок генерации требуется доступ к GigaChat и заполненная переменная окружения *GIGACHAT_CREDENTIALS*. Для проверок без обращения к внешней модели допускается использовать тестовые заглушки на уровне модулей.

Промышленное развертывание, нагрузочное тестирование с реальными пользователями, эксплуатационный мониторинг и юридическая экспертиза всех ответов в состав данных испытаний не входят.

4. Состав проверяемых функций

Проверяются следующие группы функций: подготовка корпуса, парсинг и разбиение документов; загрузка актов и фрагментов в PostgreSQL; расчет векторных представлений; полнотекстовое, векторное и гибридное извлечение; генерация ответа; проверка *used_chunk_ids*; формирование цитат; регистрация и вход пользователя; профиль и история вопросов; обратная связь; открытие страницы источника; запуск экспериментальной оценки извлечения и генерации.

5. Методика проведения испытаний

Испытания выполняются путем запуска соответствующих скриптов, веб-маршрутов и модульных проверок. Для каждого контрольного испытания фиксируются начальные условия, действия, ожидаемый результат и связанные требования. При ошибке фиксируется диагностическое сообщение, входные данные, параметры запуска и файл результата, если он был создан.

6. Критерии приемки

Прототип считается прошедшим испытания, если критические проверки подготовки корпуса, загрузки данных, поиска, генерации, проверки *used_*

chunk_ids, формирования цитат и веб-сценария выполняются без неустранимых ошибок в локальной среде. Для сценариев с недостаточным контекстом допустимым результатом считается безопасное сообщение о недостаточности контекста. Для сценария ошибки внешней модели допустимым результатом считается отказ от сохранения неподтвержденного ответа и диагностическое сообщение.

7. Перечень контрольных испытаний

Таблица Б.1 – Контрольные испытания прототипа

ID	проверяемая функция	начальные условия	действия	ожидаемый результат	связанные требования
T-01	Парсинг валидного .doc/HTML-экспорта	В каталоге входных данных есть валидный HTML-экспорт нормативного акта.	Запустить <i>scripts/pipeline/parse_docs.py</i> для входного каталога.	Создан JSON с <i>act</i> и <i>nodes</i> , запись добавлена в <i>manifest.json</i> .	ФТ-01, ФТ-02, ФТ-03, НФТ-02
T-02	Пропуск временных и скрытых файлов	Во входном каталоге есть файл <i>~\$*.doc</i> или файл с начальной точкой.	Запустить парсинг входного каталога.	Служебный файл пропущен, в журнале есть <i>[SKIP]</i> , пустой акт не создан.	ФТ-01, НФТ-08
T-03	Ошибка при отсутствии валидных входных файлов	Во входном каталоге нет файлов поддерживаемых форматов.	Запустить <i>parse_docs.py</i> .	Скрипт завершает работу с диагностическим сообщением <i>No supported files found</i> .	ФТ-01, НФТ-08
T-04	Разбиение статьи на фрагмент	Есть структурированный JSON с обычной статьей, помещающейся в лимит.	Запустить <i>scripts/pipeline/chunk_json.py</i> .	Для статьи создан один фрагмент с <i>article_no</i> , <i>structure_ref</i> и <i>token_count</i> .	ФТ-04, ФТ-05, ФТ-06, НФТ-03
T-05	Разбиение длинной статьи	Есть статья, превышающая лимит размера фрагмента.	Запустить <i>chunk_json.py</i> с профилем <i>base</i> или <i>large</i> .	Статья разделена на несколько фрагментов без объединения с соседними статьями; сохранены границы узлов.	ФТ-04, НФТ-03
T-06	Загрузка актов и фрагментов в PostgreSQL	Созданы <i>acts.jsonl</i> и <i>chunks.jsonl</i> ; база доступна.	Запустить <i>scripts/pipeline/load_corpus.py data/chunks_json --reset</i> .	Таблицы <i>acts</i> и <i>chunks</i> заполнены; проверены обязательные поля и связи.	ФТ-07, НФТ-11
T-07	Расчет векторных представлений	В таблице <i>chunks</i> есть записи с <i>embedding IS NULL</i> .	Запустить <i>scripts/pipeline/embed_chunks.py --settings settings.yaml --device cpu</i> .	Поля <i>embedding</i> и <i>embedding_model</i> заполнены; размерность соответствует 1024.	ФТ-08, НФТ-02
T-08	Полнотекстовый поиск	Корпус загружен в PostgreSQL.	Запустить <i>scripts/debug/debug_sparse_search.py</i> с контрольным вопросом.	Возвращен ранжированный список фрагментов или диагностически пустой результат.	ФТ-09
T-09	Векторный поиск	Для фрагментов рассчитаны векторные представления.	Запустить <i>scripts/debug/debug_dense_search.py</i> с контрольным вопросом.	Возвращен ранжированный список ближайших фрагментов с учетом <i>embedding_model</i> .	ФТ-10
T-10	Гибридный поиск	Доступны полнотекстовый поиск и векторные представления.	Запустить <i>scripts/debug/debug_hybrid_search.py</i> .	Результаты объединены методом <i>weighted_hybrid</i> ; возвращены итоговые оценки.	ФТ-11, НФТ-10

Продолжение таблицы Б.1

ID	проверяемая функция	начальные условия	действия	ожидаемый результат	связанные требования
T-11	Пустой пользовательский вопрос	Пользователь авторизован, главная страница открыта.	Отправить пустое поле вопроса.	Показано сообщение <i>Введите вопрос</i> ; поиск, генерация и сохранение ответа не выполняются.	ФТ-24, НФТ-08
T-12	Успешная генерация ответа	Корпус загружен, модель доступна, вопрос имеет релевантный контекст.	Отправить вопрос через веб-прототип или <i>scripts/debug/generate_answer.py</i> .	Получен структурированный ответ, заполнены <i>answer</i> , <i>used_chunk_ids</i> , <i>needs_clarification</i> .	ФТ-12, ФТ-13, ФТ-14
T-13	Проверка <i>used_chunk_ids</i>	Модель вернула идентификаторы использованных фрагментов.	Сравнить <i>used_chunk_ids</i> с фрагментами, переданными в контекст.	Все идентификаторы входят в контекст; по ним формируются цитаты.	ФТ-15, ФТ-16, НФТ-01
T-14	Случай <i>used_chunk_ids</i> вне контекста	Тестовая заглушка модели возвращает идентификатор, которого нет в контексте.	Запустить генерацию с такой заглушкой.	Результат отклоняется, выводится ошибка о неизвестном <i>chunk_id</i> .	ФТ-15, НФТ-01, НФТ-08
T-15	Случай пустого <i>used_chunk_ids</i> при содержательном ответе	Тестовая модель возвращает ответ без использованных фрагментов.	Запустить разбор ответа модели.	Система заменяет результат безопасным сообщением о недостаточности контекста.	ФТ-14, ФТ-15, НФТ-01
T-16	Формирование цитат	Ответ модели содержит корректные <i>used_chunk_ids</i> .	Выполнить генерацию и открыть страницу ответа.	Цитаты сформированы по фрагментам базы данных; отображаются акт, статья, структурная ссылка и текст.	ФТ-16, ФТ-17, НФТ-01
T-17	Регистрация пользователя	Пользователь не авторизован.	Заполнить форму регистрации.	Создана запись в <i>users</i> , пароль сохранен как <i>password_hash</i> , открыта сессия.	ФТ-19, НФТ-05
T-18	Вход пользователя	Пользователь зарегистрирован.	Заполнить форму входа корректными учетными данными.	Сессия создана, <i>last_login_at</i> обновлен.	ФТ-20
T-19	Выход пользователя	Пользователь авторизован.	Нажать кнопку выхода.	Сессия очищена, пользователь возвращен на главную страницу.	ФТ-20
T-20	Открытие профиля	Пользователь авторизован.	Перейти на <i>/profile</i> .	Отображаются данные пользователя и история его вопросов.	ФТ-21, ФТ-22, НФТ-06
T-21	Просмотр истории вопросов	У пользователя есть сохраненные ответы.	Открыть профиль и перейти к ответу из истории.	Отображаются только вопросы текущего пользователя.	ФТ-22, НФТ-06
T-22	Отправка <i>feedback</i>	Открыта страница ответа владельца.	Поставить оценку и отправить комментарий.	Создана запись в <i>feedback</i> , пользователь возвращен на страницу ответа.	ФТ-23
T-23	Повторная отправка <i>feedback</i>	Для ответа уже есть оценка текущего пользователя.	Отправить другую оценку или комментарий.	Существующая запись обновлена по паре <i>answer_id</i> , <i>user_id</i> , дубль не создан.	ФТ-23, НФТ-11
T-24	Открытие страницы источника	Открыта страница ответа с цитатами.	Перейти по ссылке на источник.	Отображаются реквизиты акта и процитированные фрагменты; доступ проверяется по владельцу ответа.	ФТ-18, ФТ-16, НФТ-06
T-25	Запуск оценки извлечения релевантных фрагментов	Подготовлен <i>eval/gold_resolved.jsonl</i> , корпус загружен, векторы рассчитаны.	Запустить <i>scripts/eval/run_retrieval_eval.py</i> .	Созданы <i>retrieval_runs.jsonl</i> , <i>summary_metrics.csv</i> , <i>error_analysis.csv</i> , <i>config_snapshot.yaml</i> .	ФТ-25, НФТ-07
T-26	Запуск проверки генерации	Подготовлен <i>gold_resolved.jsonl</i> , доступен GigaChat.	Запустить <i>scripts/eval/run_generation_eval.py</i> .	Созданы <i>generation_all_v1.jsonl</i> и <i>generation_all_v1.csv</i> .	ФТ-26, НФТ-07

8. Ограничения испытаний

Испытания проводятся в локальной среде MVP. Они подтверждают работоспособность реализованного прототипа и воспроизводимость экспериментальных материалов, но не являются промышленными приемочными испытаниями. В испытания не входят юридическая экспертиза всех ответов, проверка актуальности нормативных актов вне локального корпуса, нагрузочное тестирование, проверка отказоустойчивости, аудит информационной безопасности и промышленное развертывание.

ПРИЛОЖЕНИЕ В. ОПИСАНИЕ СХЕМЫ БАЗЫ ДАННЫХ

В приложении приведено текстовое описание актуальной схемы базы данных, сформированной на основе файлов *sql/init/001_schema.sql*, *sql/migrations/001_answers_web_fields.sql* и *sql/migrations/002_auth_feedback.sql*. В актуальную модель данных входят таблицы *users*, *acts*, *chunks*, *queries*, *answers*, *answer_citations* и *feedback*. Таблицы *experiments* и *experiment_runs* не включаются в описание, поскольку результаты экспериментальной оценки хранятся в файловом контуре проекта. Логическая схема базы данных представлена на рисунке В.1.

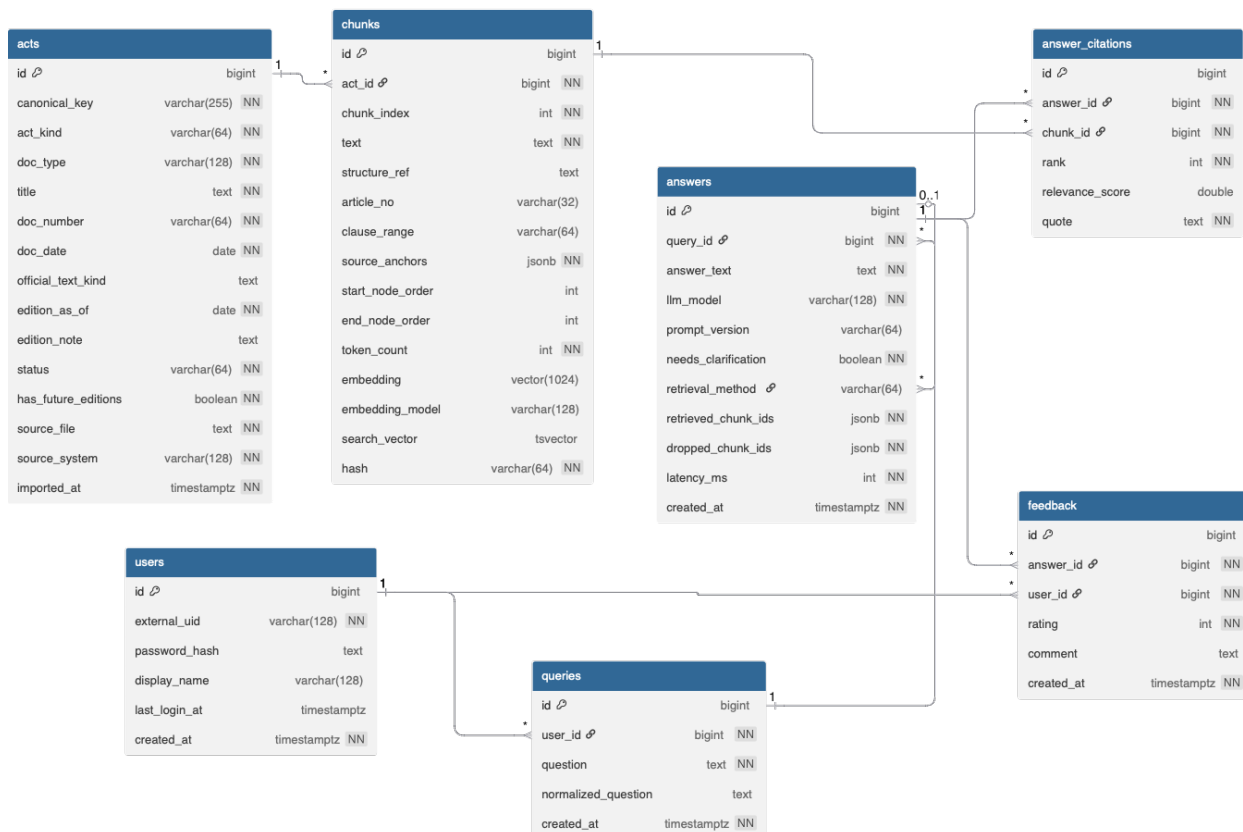


Рисунок В.1 – Логическая схема базы данных прототипа RAG-системы

Таблица В.1 – Назначение таблиц базы данных

таблица	назначение	основные связи
<i>users</i>	Хранение локальных учетных записей пользователей веб-прототипа.	<i>users.id</i> связан с <i>queries.user_id</i> и <i>feedback.user_id</i> .
<i>acts</i>	Хранение метаданных нормативных правовых актов.	<i>acts.id</i> связан с <i>chunks.act_id</i> .
<i>chunks</i>	Хранение подготовленных фрагментов актов, полнотекстового поискового вектора и векторного представления.	<i>chunks.act_id</i> ссылается на <i>acts.id</i> ; <i>chunks.id</i> связан с <i>answer_citations.chunk_id</i> .

Продолжение таблицы В.1

таблица	назначение	основные связи
<i>queries</i>	Хранение пользовательских вопросов.	<i>queries.user_id</i> ссылается на <i>users.id</i> ; <i>answers.query_id</i> ссылается на <i>queries.id</i> .
<i>answers</i>	Хранение ответов системы и служебных параметров извлечения релевантных фрагментов.	<i>answers.query_id</i> ссылается на <i>queries.id</i> ; <i>answer_citations.answer_id</i> и <i>feedback.answer_id</i> ссылаются на <i>answers.id</i> .
<i>answer_citations</i>	Хранение цитат, сформированных по использованным фрагментам.	Связывает <i>answers</i> и <i>chunks</i> .
<i>feedback</i>	Хранение пользовательской оценки ответа.	Связывает <i>answers</i> и <i>users</i> ; действует уникальность пары <i>answer_id, user_id</i> .

Таблица В.2 – Поля таблицы users

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор пользователя.	Первичный ключ.
<i>external_uid</i>	<i>varchar(128)</i>	Логин или внешний идентификатор пользователя.	<i>NOT NULL, UNIQUE</i> .
<i>password_hash</i>	<i>text</i>	Хеш пароля пользователя.	Добавлен в актуальной схеме; пароль не хранится в открытом виде.
<i>display_name</i>	<i>varchar(128)</i>	Отображаемое имя пользователя.	Может быть <i>NULL</i> .
<i>last_login_at</i>	<i>timestampz</i>	Время последнего успешного входа.	Может быть <i>NULL</i> .
<i>created_at</i>	<i>timestampz</i>	Дата и время создания пользователя.	<i>NOT NULL DEFAULT now()</i> .

Таблица В.3 – Поля таблицы acts

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор акта.	Первичный ключ.
<i>canonical_key</i>	<i>varchar(255)</i>	Стабильный ключ акта.	<i>NOT NULL, UNIQUE</i> .
<i>act_kind</i>	<i>varchar(64)</i>	Вид акта.	<i>NOT NULL, CHECK: constitution, codex, federal_law, federal_constitutional_law, law_rf, other</i> .
<i>doc_type</i>	<i>varchar(128)</i>	Тип документа.	<i>NOT NULL</i> .
<i>title</i>	<i>text</i>	Название нормативного акта.	<i>NOT NULL</i> .
<i>doc_number</i>	<i>varchar(64)</i>	Номер документа.	<i>NOT NULL</i> .
<i>doc_date</i>	<i>date</i>	Дата принятия или подписания документа.	<i>NOT NULL</i> .
<i>official_text_kind</i>	<i>text</i>	Сведения о типе официального текста.	Может быть <i>NULL</i> .
<i>edition_as_of</i>	<i>date</i>	Дата редакции, использованной в локальном корпусе.	<i>NOT NULL</i> .
<i>edition_note</i>	<i>text</i>	Пояснение о редакции.	Может быть <i>NULL</i> .
<i>status</i>	<i>varchar(64)</i>	Статус документа.	<i>NOT NULL DEFAULT unknown, CHECK: actual, actual_with_future_editions, inactive, unknown</i> .
<i>has_future_editions</i>	<i>boolean</i>	Признак наличия будущих редакций.	<i>NOT NULL DEFAULT false</i> .

Продолжение таблицы В.3

поле	тип данных	назначение	ограничение или примечание
<i>source_file</i>	<i>text</i>	Имя исходного файла.	<i>NOT NULL</i> .
<i>source_system</i>	<i>varchar(128)</i>	Система-источник HTML-экспорта.	<i>NOT NULL DEFAULT pravo.gov.ru_html_doc</i> .
<i>imported_at</i>	<i>timestampz</i>	Дата и время загрузки в базу.	<i>NOT NULL DEFAULT now()</i> .

Таблица В.4 – Поля таблицы *chunks*

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор фрагмента.	Первичный ключ.
<i>act_id</i>	<i>bigint</i>	Идентификатор акта.	<i>NOT NULL</i> , внешний ключ на <i>acts(id)</i> с <i>ON DELETE CASCADE</i> .
<i>chunk_index</i>	<i>int</i>	Порядковый номер фрагмента внутри акта.	<i>NOT NULL</i> , <i>CHECK (chunk_index >= 0)</i> , уникальность пары <i>act_id</i> , <i>chunk_index</i> .
<i>text</i>	<i>text</i>	Текстовое представление фрагмента.	<i>NOT NULL</i> .
<i>structure_ref</i>	<i>text</i>	Структурная ссылка внутри акта.	Может быть <i>NULL</i> .
<i>article_no</i>	<i>varchar(32)</i>	Номер статьи.	Может быть <i>NULL</i> ; индекс <i>idx_chunks_article_no</i> .
<i>clause_range</i>	<i>varchar(64)</i>	Диапазон пунктов внутри фрагмента.	Может быть <i>NULL</i> .
<i>source_anchors</i>	<i>jsonb</i>	Список исходных HTML-якорей.	<i>NOT NULL DEFAULT []::jsonb</i> .
<i>start_node_order</i>	<i>int</i>	Порядок первого исходного узла.	<i>CHECK (start_node_order >= 0)</i> .
<i>end_node_order</i>	<i>int</i>	Порядок последнего исходного узла.	<i>CHECK (end_node_order >= start_node_order)</i> .
<i>token_count</i>	<i>int</i>	Число токенов во фрагменте.	<i>NOT NULL</i> , <i>CHECK (token_count > 0)</i> .
<i>embedding</i>	<i>vector(1024)</i>	Векторное представление фрагмента.	Может быть <i>NULL</i> ; индекс <i>ivfflat</i> при наличии значения.
<i>embedding_model</i>	<i>varchar(128)</i>	Имя модели векторных представлений.	Может быть <i>NULL</i> ; <i>CHECK</i> согласует заполненность с <i>embedding</i> .
<i>search_vector</i>	<i>tsvector</i>	Полнотекстовый поисковый вектор.	Вычисляемое поле <i>GENERATED ALWAYS AS ... STORED</i> , индекс <i>GIN</i> .
<i>hash</i>	<i>varchar(64)</i>	Хеш фрагмента.	<i>NOT NULL</i> , <i>UNIQUE</i> .

Таблица В.5 – Поля таблицы *queries*

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор вопроса.	Первичный ключ.
<i>user_id</i>	<i>bigint</i>	Идентификатор пользователя, задавшего вопрос.	<i>NOT NULL</i> , внешний ключ на <i>users(id)</i> с <i>ON DELETE CASCADE</i> .
<i>question</i>	<i>text</i>	Исходный текст вопроса.	<i>NOT NULL</i> .
<i>normalized_question</i>	<i>text</i>	Нормализованное представление вопроса.	Может быть <i>NULL</i> .

Продолжение таблицы В.5

поле	тип данных	назначение	ограничение или примечание
<i>created_at</i>	<i>timestampz</i>	Дата и время создания вопроса.	<i>NOT NULL DEFAULT now()</i> .

Таблица В.6 – Поля таблицы *answers*

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор ответа.	Первичный ключ.
<i>query_id</i>	<i>bigint</i>	Идентификатор вопроса.	<i>NOT NULL</i> , внешний ключ на <i>queries(id)</i> с <i>ON DELETE CASCADE</i> .
<i>answer_text</i>	<i>text</i>	Текст ответа системы.	<i>NOT NULL</i> .
<i>llm_model</i>	<i>varchar(128)</i>	Имя модели языковой модели.	<i>NOT NULL</i> .
<i>prompt_version</i>	<i>varchar(64)</i>	Версия шаблона запроса.	Может быть <i>NULL</i> .
<i>needs_clarification</i>	<i>boolean</i>	Признак недостаточности контекста или необходимости уточнения.	<i>NOT NULL DEFAULT false</i> .
<i>retrieval_method</i>	<i>varchar(64)</i>	Метод извлечения релевантных фрагментов.	Может быть <i>NULL</i> ; в текущем запуске используется <i>weighted_hybrid</i> .
<i>retrieved_chunk_ids</i>	<i>jsonb</i>	Список найденных фрагментов.	<i>NOT NULL DEFAULT []::jsonb</i> .
<i>dropped_chunk_ids</i>	<i>jsonb</i>	Список фрагментов, отброшенных из-за ограничения контекста.	<i>NOT NULL DEFAULT []::jsonb</i> .
<i>latency_ms</i>	<i>int</i>	Время генерации ответа в миллисекундах.	<i>NOT NULL, CHECK (latency_ms >= 0)</i> .
<i>created_at</i>	<i>timestampz</i>	Дата и время создания ответа.	<i>NOT NULL DEFAULT now()</i> .

Таблица В.7 – Поля таблицы *answer_citations*

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор цитаты.	Первичный ключ.
<i>answer_id</i>	<i>bigint</i>	Идентификатор ответа.	<i>NOT NULL</i> , внешний ключ на <i>answers(id)</i> с <i>ON DELETE CASCADE</i> .
<i>chunk_id</i>	<i>bigint</i>	Идентификатор использованного фрагмента.	<i>NOT NULL</i> , внешний ключ на <i>chunks(id)</i> с <i>ON DELETE CASCADE</i> .
<i>rank</i>	<i>int</i>	Порядок цитаты в ответе.	<i>NOT NULL, CHECK (rank > 0)</i> .
<i>relevance_score</i>	<i>double precision</i>	Оценка релевантности фрагмента.	<i>CHECK (relevance_score >= 0)</i> , может быть <i>NULL</i> .
<i>quote</i>	<i>text</i>	Текст цитаты.	<i>NOT NULL</i> ; в текущей версии соответствует полному тексту фрагмента.

Таблица В.8 – Поля таблицы *feedback*

поле	тип данных	назначение	ограничение или примечание
<i>id</i>	<i>bigserial</i>	Внутренний идентификатор записи обратной связи.	Первичный ключ.
<i>answer_id</i>	<i>bigint</i>	Идентификатор оцениваемого ответа.	<i>NOT NULL</i> , внешний ключ на <i>answers(id)</i> с <i>ON DELETE CASCADE</i> .
<i>user_id</i>	<i>bigint</i>	Идентификатор пользователя, оставившего оценку.	<i>NOT NULL</i> , внешний ключ на <i>users(id)</i> с <i>ON DELETE CASCADE</i> .
<i>rating</i>	<i>int</i>	Оценка ответа.	<i>NOT NULL</i> , <i>CHECK (rating BETWEEN 1 AND 5)</i> .
<i>comment</i>	<i>text</i>	Комментарий пользователя.	Может быть <i>NULL</i> .
<i>created_at</i>	<i>timestampz</i>	Дата и время создания или обновления оценки.	<i>NOT NULL DEFAULT now()</i> .
<i>UNIQUE (answer_id, user_id)</i>	ограничение	Запрещает несколько оценок одного ответа одним пользователем.	Повторная отправка обратной связи обновляет существующую запись.

В таблице *users* актуальная схема содержит поля *password_hash*, *display_name* и *last_login_at*. В таблице *answers* актуальная схема содержит поля *retrieved_chunk_ids* и *dropped_chunk_ids*. В таблице *feedback* актуальная схема содержит поле *created_at* и ограничение уникальности по паре *answer_id, user_id*.

Результаты экспериментальной оценки не хранятся в таблицах базы данных. Для них используются файлы JSONL, CSV и YAML в каталоге *eval/results*.

ПРИЛОЖЕНИЕ Г. ПРИМЕРЫ ПОДГОТОВЛЕННЫХ ФРАГМЕНТОВ НОРМАТИВНЫХ АКТОВ

В архиве репозитория отсутствует каталог *data/chunks_json* с полным файлом *chunks.jsonl*. Поэтому примеры подготовленных фрагментов в настоящем приложении взяты из актуальных файлов экспериментальной оценки: полный текст фрагментов — из *eval/results/generation/generation_all_v1.jsonl*, а число токенов и часть метаданных — из *eval/gold_resolved.jsonl* и *eval/results/retrieval/retrieval_runs.jsonl*. Эти файлы сформированы на основе той же загрузки базы данных, по которой выполнялись оценочные прогоны.

Таблица Г.1 – Примеры подготовленных фрагментов нормативных актов

ID примера	акт	структурная ссылка	статья или элемент	число токенов	назначение примера
Г.1	Конституция Российской Федерации	РАЗДЕЛ ПЕРВЫЙ / ГЛАВА 2. ПРАВА И СВОБОДЫ ЧЕЛОВЕКА И ГРАЖДАНИНА / Статья 46 / п. 1-3	статья 46	130	пример фрагмента Конституции Российской Федерации
Г.2	Водный кодекс Российской Федерации	Глава 1. Общие положения / Статья 6. Водные объекты общего пользования / п. 1-8	статья 6	533	пример фрагмента кодекса Российской Федерации
Г.3	О персональных данных	Глава 1. Общие положения / Статья 3. Основные понятия, используемые в настоящем Федеральном законе / п. 1-11	статья 3	573	пример фрагмента федерального закона
Г.4	О Конституционном Суде Российской Федерации	РАЗДЕЛ ПЕРВЫЙ. ОРГАНИЗАЦИЯ КОНСТИТУЦИОННОГО СУДА РОССИЙСКОЙ ФЕДЕРАЦИИ И СТАТУС СУДЕЙ / ГЛАВА 1. Общие положения / Статья 3. Полномочия Конституционного Суда Российской Федерации	статья 3	740	пример фрагмента федерального конституционного закона
Г.5	Гражданский процессуальный кодекс Российской Федерации	РАЗДЕЛ II. ПРОИЗВОДСТВО В СУДЕ ПЕРВОЙ ИНСТАНЦИИ / ПОДРАЗДЕЛ II. ИСКОВОЕ ПРОИЗВОДСТВО / Глава 12. ПРЕДЪЯВЛЕНИЕ ИСКА / Статья 131. Форма и содержание искового заявления / п. 1-8	статья 131	788	пример длинной статьи, представленной отдельным фрагментом после разбиения

Пример Г.1. Конституция Российской Федерации, статья 46

Источник данных: *eval/results/generation/generation_all_v1.jsonl*; дополнительные метаданные: *eval/gold_resolved.jsonl* и *eval/results/retrieval/retrieval_runs.jsonl*.

Основные метаданные: *chunk_id* = 13354; *act_title* = Конституция Российской Федерации; *doc_number* = б/н; *doc_date* = 1993-12-12; *article_no* = 46; *clause_range* = 1-3; *token_count* = 130.

Текст фрагмента:

Конституция Российской Федерации
РАЗДЕЛ ПЕРВЫЙ ГЛАВА 2. ПРАВА И СВОБОДЫ ЧЕЛОВЕКА И ГРАЖДАНИНА Статья 46

1. Каждому гарантируется судебная защита его прав и свобод. 2. Решения и действия (или бездействие) органов государственной власти, органов местного самоуправления, общественных объединений и должностных лиц могут быть обжалованы в суд. 3. Каждый вправе в соответствии с международными договорами Российской Федерации обращаться в межгосударственные органы по защите прав и свобод человека, если исчерпаны все имеющиеся внутригосударственные средства правовой защиты.

Комментарий: Пример показывает, что фрагмент Конституции сохраняет структурный путь до главы и статьи, а также пункты статьи, необходимые для проверяемого ответа.

Пример Г.2. Водный кодекс Российской Федерации, статья 6

Источник данных: *eval/results/generation/generation_all_v1.jsonl*; дополнительные метаданные: *eval/gold_resolved.jsonl* и *eval/results/retrieval/retrieval_runs.jsonl*.

Основные метаданные: *chunk_id* = 752; *act_title* = Водный кодекс Российской Федерации; *doc_number* = 74-ФЗ; *doc_date* = 2006-06-03; *article_no* = 6; *clause_range* = 1-8; *token_count* = 533.

Текст фрагмента:

Водный кодекс Российской Федерации
Глава 1. Общие положения Статья 6. Водные объекты общего пользования

1. Поверхностные водные объекты, находящиеся в государственной или муниципальной собственности, являются водными объектами общего пользования, то есть общедоступными водными объектами, если иное не предусмотрено настоящим Кодексом. 2. Каждый гражданин вправе иметь доступ к водным объектам общего пользования и бесплатно использовать их для личных и бытовых нужд, если иное не предусмотрено настоящим Кодексом, другими федеральными законами. 3. Использование водных объектов общего пользования осуществляется с учетом правил использования водных объектов для рекреационных целей, утверждаемых в соответствии со статьей 50 настоящего Кодекса, а также с учетом правил использования водных объектов для личных и бытовых нужд. 4. На водных объектах общего пользования могут быть запрещены забор (изъятие) водных ресурсов для целей питьевого и хозяйственно-бытового водоснабжения, купание, использование маломерных судов, водных мотоциклов и других технических средств, предназначенных для отдыха на водных объектах, водопой, а также установлены иные запреты в случаях, предусмотренных законодательством Российской Федерации и законодательством субъектов Российской Федерации. 5. Информация об ограничении водопользования на водных объектах общего пользования предоставляется гражданам органами местного самоуправления через средства массовой информации и посредством специальных информационных знаков, устанавливаемых вдоль берегов водных объектов. Могут быть также использованы иные способы предоставления такой информации. 6.

[Фрагмент сокращен для включения в приложение.]

Комментарий: Пример демонстрирует фрагмент кодекса, содержащий определение и условия использования объекта общего пользования.

Пример Г.3. О персональных данных, статья 3

Источник данных: *eval/results/generation/generation_all_v1.jsonl*; дополнительные метаданные: *eval/gold_resolved.jsonl* и *eval/results/retrieval/retrieval_runs.jsonl*.

Основные метаданные: *chunk_id* = 19483; *act_title* = О персональных данных; *doc_number* = 152-ФЗ; *doc_date* = 2006-07-27; *article_no* = 3; *clause_range* = 1-11; *token_count* = 573.

Текст фрагмента:

О персональных данных

Глава 1. Общие положения
Статья 3. Основные понятия, используемые в настоящем Федеральном законе

В целях настоящего Федерального закона используются следующие основные понятия: 1) персональные данные - любая информация, относящаяся к прямо или косвенно определенному или определяемому физическому лицу (субъекту персональных данных); 1.1) персональные данные, разрешенные субъектом персональных данных для распространения, - персональные данные, доступ неограниченного круга лиц к которым предоставлен субъектом персональных данных путем дачи согласия на обработку персональных данных, разрешенных субъектом персональных данных для распространения в порядке, предусмотренном настоящим Федеральным законом; 2) оператор - государственный орган, муниципальный орган, юридическое или физическое лицо, самостоятельно или совместно с другими лицами организующие и (или) осуществляющие обработку персональных данных, а также определяющие цели обработки персональных данных, состав персональных данных, подлежащих обработке, действия (операции), совершаемые с персональными данными; 3) обработка персональных данных - любое действие (операция) или совокупность действий (операций), совершаемых с использованием средств автоматизации или без использования таких средств с персональными данными, включая сбор, запись, систематизацию, накопление, хранение,

ние, уточнение (обновление, изменение), извлечение, использование, передачу (распространение, предоставление, доступ), обезличивание, блокирование, удаление, уничтожение персональных данных; 4) автоматизированная обработка персональных данных - обработка персональных данных с помощью средств вычислительной техники;

[Фрагмент сокращен для включения в приложение.]

Комментарий: Пример важен как фрагмент федерального закона с перечнем основных понятий, используемых при ответе на вопрос пользователя.

Пример Г.4. О Конституционном Суде Российской Федерации, статья 3

Источник данных: *eval/results/generation/generation_all_v1.jsonl*; дополнительные метаданные: *eval/gold_resolved.jsonl* и *eval/results/retrieval/retrieval_runs.jsonl*.

Основные метаданные: *chunk_id* = 21143; *act_title* = О Конституционном Суде Российской Федерации; *doc_number* = 1-ФКЗ; *doc_date* = 1994-07-21; *article_no* = 3; *clause_range* = None; *token_count* = 740.

Текст фрагмента:

О Конституционном Суде Российской Федерации
РАЗДЕЛ ПЕРВЫЙ. ОРГАНИЗАЦИЯ КОНСТИТУЦИОННОГО СУДА РОССИЙСКОЙ ФЕДЕРАЦИИ И СТАТУС СУДЕЙ ГЛАВА I. Общие положения
Статья 3. Полномочия Конституционного Суда Российской Федерации
Конституционный Суд Российской Федерации: 1) по запросам Президента Российской Федерации, Совета Федерации Федерального Собрания Российской Федерации (далее - Совет Федерации), Государственной Думы Федерального Собрания Российской Федерации (далее - Государственная Дума), одной пятой сенаторов Российской Федерации или депутатов Государственной Думы, Правительства Российской Федерации, Верховного Суда Российской Федерации, органов законодательной и исполнительной власти субъек-

тов Российской Федерации разрешает дела о соответствии Конституции Российской Федерации: а) федеральных конституционных законов, федеральных законов, нормативных актов Президента Российской Федерации, Совета Федерации, Государственной Думы, Правительства Российской Федерации; б) конституций республик, уставов, а также законов и иных нормативных актов субъектов Российской Федерации, изданных по вопросам, относящимся к ведению органов государственной власти Российской Федерации и совместному ведению органов государственной власти Российской Федерации и органов государственной власти субъектов Российской Федерации; в) договоров между органами государственной власти Российской Федерации и органами государственной власти субъектов Российской Федерации, договоров между органами государственной власти субъектов Российской Федерации; г) не вступивших в силу международных договоров Российской Федерации; 2) по ходатайству Президента Российской Федерации, любого из участвующих в споре органов разрешает споры о компетенции:

[Фрагмент сокращен для включения в приложение.]

Комментарий: Пример показывает обработку федерального конституционного закона и длинной статьи с полномочиями государственного органа.

Пример Г.5. Гражданский процессуальный кодекс Российской Федерации, статья 131

Источник данных: *eval/results/generation/generation_all_v1.jsonl*; дополнительные метаданные: *eval/gold_resolved.jsonl* и *eval/results/retrieval/retrieval_runs.jsonl*.

Основные метаданные: *chunk_id* = 4676; *act_title* = Гражданский процессуальный кодекс Российской Федерации; *doc_number* = 138-ФЗ; *doc_date* = 2002-11-14; *article_no* = 131; *clause_range* = 1-8; *token_count* = 788.

Текст фрагмента:

Гражданский процессуальный кодекс Российской Федерации

РАЗДЕЛ II. ПРОИЗВОДСТВО В СУДЕ ПЕРВОЙ ИНСТАНЦИИ ПОДРАЗДЕЛ II. ИСКОВОЕ ПРОИЗВОДСТВО Глава 12. ПРЕДЪЯВЛЕНИЕ ИСКА
Статья 131. Форма и содержание искового заявления

1. Исковое заявление подается в суд на бумажном носителе или в электронном виде, в том числе в форме электронного документа. 2. В исковом заявлении должны быть указаны: 1) наименование суда, в который подается заявление; 2) сведения об истце: для гражданина - фамилия, имя, отчество (при наличии), дата и место рождения, место жительства или место пребывания, один из идентификаторов (страховой номер индивидуального лицевого счета, идентификационный номер налогоплательщика, серия и номер документа, удостоверяющего личность, серия и номер водительского удостоверения); для организации - наименование, адрес, идентификационный номер налогоплательщика; если заявление подается представителем, - также фамилия, имя, отчество (при наличии) или наименование представителя, адрес для направления судебных повесток и иных судебных извещений, один из идентификаторов представителя (для гражданина); 3) сведения об ответчике: для гражданина - фамилия, имя, отчество (при наличии), дата и место рождения, место жительства или место пребывания, место работы (если известно), один из идентификаторов (страховой номер индивидуального лицевого счета, идентификационный номер налогоплательщика, серия и номер документа, удостоверяющего личность, основной государственный регистрационный номер индивидуального предпринимателя, серия и номер водительского удостоверения); для организации - наименование, адрес, идентификационный номер налогоплательщика и основной государственный регистрационный номер.

[Фрагмент сокращен для включения в приложение.]

Комментарий: Пример отражает длинную статью процессуального кодекса, которая приближается к лимиту размера фрагмента и используется для ответа на составной вопрос.

ПРИЛОЖЕНИЕ Д. МАТЕРИАЛЫ ЭКСПЕРИМЕНТАЛЬНОЙ ОЦЕНКИ

1. Набор тестовых вопросов

Набор экспериментальной оценки включает 50 вопросов. Вопросы разделены на категории *term_exact*, *paraphrase*, *mixed*, *multi_fragment* и *clarification_needed*. В таблице Д.1 приведена устойчивая разметка по ожидаемому акту и статье или структурному элементу. Идентификаторы *gold_chunk_ids* не используются как основная разметка в данной таблице, поскольку они зависят от конкретной загрузки базы данных.

Таблица Д.1 – Набор тестовых вопросов экспериментальной оценки

ID вопроса	категория	вопрос	ожидаемый акт	ожидаемая статья или структурный элемент
q001	paraphrase	Какие гарантии судебной защиты прав и свобод установлены Конституцией?	Конституция Российской Федерации	статья 46
q002	term_exact	Какие вопросы относятся к ведению Российской Федерации?	Конституция Российской Федерации	статья 71
q003	term_exact	Какие вопросы относятся к совместному ведению Российской Федерации и субъектов?	Конституция Российской Федерации	статья 72
q004	term_exact	Какие полномочия имеет Конституционный Суд Российской Федерации?	Федеральный конституционный закон «О Конституционном Суде Российской Федерации»	статья 3
q005	mixed	Каков правовой статус Правительства Российской Федерации?	Федеральный конституционный закон «О Правительстве Российской Федерации»	статья 1
q006	paraphrase	Когда работодатель обязан выплатить расчет при увольнении?	Трудовой кодекс Российской Федерации	статья 140
q007	multi_fragment	Работник хочет уволиться по собственному желанию: как подать заявление, оформить увольнение и когда выплатят деньги?	Трудовой кодекс Российской Федерации; Трудовой кодекс Российской Федерации; Трудовой кодекс Российской Федерации	статья 80; статья 84.1; статья 140
q008	mixed	Какие сведения обязательно должны быть указаны в трудовом договоре?	Трудовой кодекс Российской Федерации	статья 57
q009	paraphrase	Как часто работодатель должен выплачивать заработную плату?	Трудовой кодекс Российской Федерации	статья 136
q010	paraphrase	Сколько дней основного ежегодного оплачиваемого отпуска положено работнику?	Трудовой кодекс Российской Федерации	статья 115
q011	mixed	Какие основные обязанности есть у работника?	Трудовой кодекс Российской Федерации	статья 21
q012	mixed	Какие основные обязанности есть у работодателя?	Трудовой кодекс Российской Федерации	статья 22
q013	term_exact	Что относится к персональным данным?	Федеральный закон «О персональных данных»	статья 3
q014	multi_fragment	Когда организация может обрабатывать персональные данные и когда нужно согласие субъекта?	Федеральный закон «О персональных данных»; Федеральный закон «О персональных данных»	статья 6; статья 9
q015	paraphrase	Какие права есть у человека, чьи персональные данные обрабатываются?	Федеральный закон «О персональных данных»	статья 14
q016	mixed	Какие меры должна принимать организация для защиты персональных данных?	Федеральный закон «О персональных данных»	статья 19

Продолжение таблицы Д.1

ID вопроса	категория	вопрос	ожидаемый акт	ожидаемая статья или структурный элемент
q017	mixed	Какие требования предъявляются к письменному обращению гражданина в государственный орган?	Федеральный закон «О порядке рассмотрения обращений граждан Российской Федерации»	статья 7
q018	paraphrase	В какой срок должны рассмотреть обращение гражданина?	Федеральный закон «О порядке рассмотрения обращений граждан Российской Федерации»	статья 12
q019	paraphrase	Что обязан сделать государственный орган при рассмотрении обращения?	Федеральный закон «О порядке рассмотрения обращений граждан Российской Федерации»	статья 10
q020	term_exact	Что относится к недвижимым вещам по Гражданскому законодательству?	Гражданский кодекс Российской Федерации (часть первая)	статья 130
q021	term_exact	Что такое сделка по Гражданскому кодексу?	Гражданский кодекс Российской Федерации (часть первая)	статья 153
q022	paraphrase	Когда договор считается заключенным?	Гражданский кодекс Российской Федерации (часть первая)	статья 432
q023	clarification_needed	Я передумал исполнять договор. Можно ли просто отказаться от обязательства в одностороннем порядке?	Гражданский кодекс Российской Федерации (часть первая); Гражданский кодекс Российской Федерации (часть первая)	статья 310; статья 450
q024	term_exact	Что такое договор купли-продажи?	Гражданский кодекс Российской Федерации (часть вторая)	статья 454
q025	term_exact	Что такое договор возмездного оказания услуг?	Гражданский кодекс Российской Федерации (часть вторая)	статья 779
q026	paraphrase	Когда нужно вернуть неосновательное обогащение?	Гражданский кодекс Российской Федерации (часть вторая)	статья 1102
q027	mixed	Какие основные обязанности есть у налогоплательщика?	Налоговый кодекс Российской Федерации (часть первая)	статья 23
q028	mixed	Когда обязанность по уплате налога считается исполненной?	Налоговый кодекс Российской Федерации (часть первая)	статья 45
q029	term_exact	Что такое налоговая декларация?	Налоговый кодекс Российской Федерации (часть первая)	статья 80
q030	term_exact	Что такое административное правонарушение?	Кодекс Российской Федерации об административных правонарушениях	статья 2.1
q031	paraphrase	Какие права есть у человека, в отношении которого ведется дело об административном правонарушении?	Кодекс Российской Федерации об административных правонарушениях	статья 25.1
q032	mixed	В какой срок можно обжаловать постановление по делу об административном правонарушении?	Кодекс Российской Федерации об административных правонарушениях	статья 30.3
q033	term_exact	Что такое преступление?	Уголовный кодекс Российской Федерации	статья 14
q034	term_exact	Какие категории преступлений выделяет Уголовный кодекс?	Уголовный кодекс Российской Федерации	статья 15
q035	paraphrase	С какого возраста по общему правилу наступает уголовная ответственность?	Уголовный кодекс Российской Федерации	статья 20
q036	multi_fragment	Что нужно написать в исковом заявлении в гражданский суд и какие документы приложить?	Гражданский процессуальный кодекс Российской Федерации; Гражданский процессуальный кодекс Российской Федерации	статья 131; статья 132
q037	paraphrase	В какой срок суд рассматривает гражданское дело?	Гражданский процессуальный кодекс Российской Федерации	статья 154
q038	mixed	Кто имеет право обратиться в арбитражный суд?	Арбитражный процессуальный кодекс Российской Федерации	статья 4
q039	multi_fragment	Что должно быть в исковом заявлении в арбитражный суд и какие документы к нему приложить?	Арбитражный процессуальный кодекс Российской Федерации; Арбитражный процессуальный кодекс Российской Федерации	статья 125; статья 126
q040	paraphrase	Равны ли права и обязанности родителей в отношении детей?	Семейный кодекс Российской Федерации	статья 61
q041	paraphrase	Кто обязан содержать несовершеннолетних детей?	Семейный кодекс Российской Федерации	статья 80
q042	mixed	Как обычно определяется размер алиментов на несовершеннолетних детей?	Семейный кодекс Российской Федерации	статья 81
q043	paraphrase	Кто обязан платить за жилое помещение и коммунальные услуги?	Жилищный кодекс Российской Федерации	статья 153

Продолжение таблицы Д.1

ID вопроса	категория	вопрос	ожидаемый акт	ожидаемая статья или структурный элемент
q044	paraphrase	До какого числа обычно нужно внести плату за жилье и коммунальные услуги?	Жилищный кодекс Российской Федерации	статья 155
q045	mixed	Какие обязанности есть у собственника земельного участка?	Земельный кодекс Российской Федерации	статья 42
q046	term_exact	Что такое водный объект общего пользования?	Водный кодекс Российской Федерации	статья 6
q047	paraphrase	Может ли гражданин свободно находиться в лесах?	Лесной кодекс Российской Федерации	статья 11
q048	term_exact	Какие виды электронной подписи предусмотрены законом?	Федеральный закон «Об электронной подписи»	статья 5
q049	multi_fragment	Как проверить электронный документ: какие бывают подписи и когда он равнозначен бумажному документу?	Федеральный закон «Об электронной подписи»; Федеральный закон «Об электронной подписи»	статья 5; статья 6
q050	clarification_needed	Нужно ли согласие пациента на медицинское вмешательство, если ситуация срочная?	Федеральный закон «Об основах охраны здоровья граждан в Российской Федерации»	статья 20

2. Разрешение эталонных фрагментов

Файл *eval/gold_resolved.jsonl* содержит результат сопоставления устойчивой разметки по акту и статье с реальными идентификаторами фрагментов текущей загрузки базы данных. В представленном прогоне все 50 вопросов имеют статус *resolved*; предупреждения разрешения отсутствуют. Распределение числа эталонных фрагментов по вопросам показывает, что 33 вопроса имеют один эталонный фрагмент, остальные требуют нескольких фрагментов из-за длинных статей или составного характера вопроса.

Таблица Д.2 – Файлы экспериментальной оценки

файл	формат	назначение
<i>eval/eval_questions.jsonl</i>	JSONL	Набор из 50 тестовых вопросов с категорией, ожидаемым актом и устойчивой разметкой по статье или структурному элементу.
<i>eval/questions_seed.md</i>	Markdown	Человекочитаемая версия набора вопросов, использованная при подготовке разметки.
<i>eval/gold_resolved.jsonl</i>	JSONL	Разрешение эталонных ссылок на реальные <i>chunk_id</i> текущей загрузки базы данных.
<i>eval/results/retrieval/retrieval_runs.jsonl</i>	JSONL	Подробные результаты запусков полнотекстового, векторного и гибридного извлечения по каждому вопросу.
<i>eval/results/retrieval/summary_metrics.csv</i>	CSV	Сводные метрики качества извлечения релевантных фрагментов.
<i>eval/results/retrieval/error_analysis.csv</i>	CSV	Материал для ручного анализа ошибок ранжирования и пропусков эталонных фрагментов.
<i>eval/results/retrieval/config_snapshot.yaml</i>	YAML	Зафиксированная конфигурация запуска оценки извлечения.

Продолжение таблицы Д.2

файл	формат	назначение
<i>eval/results/generation/generation_all_v1.jsonl</i>	JSONL	Полные результаты проверки генерации ответа, включая ответы, использованные фрагменты и цитаты.
<i>eval/results/generation/generation_all_v1.csv</i>	CSV	Таблица для последующей ручной оценки качества генерации и согласованности ответа с контекстом.

3. Сводные результаты извлечения релевантных фрагментов

Сводные метрики взяты из файла *eval/results/retrieval/summary_metrics.csv*. В эксперименте использовались 50 вопросов, $top_k = 10$, $candidate_limit = 50$, $rrf_k = 60$, модель векторных представлений *BAAI/bge-m3* и размерность 1024.

Таблица Д.3 – Сводные метрики извлечения релевантных фрагментов

конфигурация	число вопросов	Hit@1	Hit@5	Hit@10	Recall@5	Recall@10	MRR	среднее время, мс	медианное время, мс
sparse	50	0.16	0.18	0.18	0.1525	0.1525	0.164	49.3	24.0
dense	50	0.82	0.88	0.88	0.816881	0.843571	0.846667	363.96	216.5
weighted_hybrid_s0.4_d0.6	50	0.66	0.86	0.88	0.782714	0.830238	0.747857	324.72	145.0
weighted_hybrid_s0.5_d0.5	50	0.64	0.88	0.92	0.802714	0.865238	0.739079	286.08	136.0
weighted_hybrid_s0.3_d0.7	50	0.66	0.86	0.88	0.785214	0.833571	0.747857	192.42	132.0

По данным таблицы Д.3, конфигурация *weighted_hybrid_s0.5_d0.5* показала максимальное значение *Hit@10* среди проверенных конфигураций и максимальное значение *Recall@10* в данном прогоне. При этом векторное извлечение имеет наилучшие значения *Hit@1* и *MRR*, что важно учитывать при интерпретации результатов.

4. Анализ ошибок извлечения

Примеры ошибок в таблице Д.4 взяты из файла *eval/results/retrieval/error_analysis.csv*. В таблицу включены случаи пропуска эталонного фрагмента в первых десяти результатах, а также показательные ошибки выбора первого результата.

Таблица Д.4 – Примеры ошибок извлечения

ID вопроса	конфигурация	тип ошибки	вопрос	эталонная норма	первый найденный результат	эталонные <i>chunk_id</i>
q003	weighted_hybrid_s0.5_d0.5	miss_at_10	Какие вопросы относятся к совместному ведению Российской Федерации и субъектов?	Статья 72 содержит перечень вопросов совместного ведения.	Об общих принципах организации публичной власти в субъектах Российской Федерации, статья 44	[13381]

Продолжение таблицы Д.4

ID вопроса	конфигурация	тип ошибки	вопрос	эталонная норма	первый найденный результат	эталонные <i>chunk_id</i>
q021	weighted_hybrid_s0.5_d0.5	miss_at_10	Что такое сделка по Гражданскому кодексу?	Статья 153 ГК РФ содержит понятие сделки.	Гражданский кодекс Российской Федерации, статья 134	[9536]
q024	weighted_hybrid_s0.5_d0.5	miss_at_10	Что такое договор купли-продажи?	Статья 454 ГК РФ определяет договор купли-продажи.	О несостоятельности (банкротстве), статья 195	[7097]
q025	weighted_hybrid_s0.5_d0.5	miss_at_10	Что такое договор возмездного оказания услуг?	Статья 779 ГК РФ определяет договор возмездного оказания услуг.	О защите прав потребителей, статья 9	[7422]
q022	dense	miss_at_10	Когда договор считается заключенным?	Статья 432 ГК РФ определяет основные положения о заключении договора.	О контрактной системе в сфере закупок товаров, работ, услуг для обеспечения государственных и муниципальных нужд, статья 51	[9895]
q037	dense	miss_at_10	В какой срок суд рассматривает гражданское дело?	Статья 154 ГПК РФ задает общие сроки, но для отдельных категорий дел возможны специальные правила.	Гражданский процессуальный кодекс Российской Федерации, статья 6.1	[4714]
q032	sparse	miss_at_10	В какой срок можно обжаловать постановление по делу об административном правонарушении?	Статья 30.3 КоАП РФ устанавливает срок обжалования постановления.	Кодекс Российской Федерации об административных правонарушениях, статья 31.1	[11307]
q046	sparse	miss_at_10	Что такое водный объект общего пользования?	Статья 6 ВК РФ регулирует водные объекты общего пользования.	Об общих принципах организации местного самоуправления в единой системе публичной власти, статья 32	[752]

Ошибки показывают несколько типовых причин снижения качества. Для вопросов о совместном ведении система может предпочитать федеральный закон, близкий по тематике, вместо статьи Конституции. Для вопросов о договорах в Гражданском кодексе релевантные статьи могут не попадать в первые десять результатов, если формулировка вопроса совпадает с терминами в других актах. Для полнотекстового поиска характерны пропуски при недостаточном совпадении слов вопроса и текста нормы.

5. Материалы проверки генерации ответа

Проверка генерации выполнена по файлам *eval/results/generation/generation_all_v1.jsonl* и *eval/results/generation/generation_all_v1.csv*. Конфигурация прогона зафиксирована в *eval/results/generation/config_snapshot.yaml*: метод извлечения *weighted_hybrid*, *top_k = 10*, *candidate_limit = 50*, веса *sparse_weight = 0.5* и *dense_weight = 0.5*, поставщик *gigachat*, модель из прогона *GigaChat:2.0.28.2*, версия шаблона запроса *answer_v1*.

Таблица Д.5 – Результаты проверки генерации ответа

показатель	значение	комментарий
Всего вопросов в проверке генерации	50	<i>generation_all_v1.jsonl</i> содержит 50 строк.
Валидные структурированные ответы	49	Для этих строк статус равен <i>ok</i> , ответ прошел разбор и проверку структуры.
Ошибки внешней модели или невалидного ответа	1	В одном случае модель вернула невалидный JSON.

Продолжение таблицы Д.5

показатель	значение	комментарий
Случаи, где эталонный фрагмент найден среди <i>retrieved_chunk_ids</i>	45 (любой эталон), 40 (все эталоны)	Показатель рассчитан по сохраненному JSONL-файлу генерации.
Случаи, где эталонный фрагмент выбран в <i>used_chunk_ids</i>	43 (любой эталон), 35 (все эталоны)	Показывает, что модель сослалась на эталонный фрагмент после получения контекста.
Случаи с пустым <i>used_chunk_ids</i>	3	Два случая являются безопасным отказом о недостаточности контекста; один случай связан с ошибкой невалидного JSON.
Случаи <i>needs_clarification = true</i>	2	Зафиксированы для <i>q021</i> и <i>q024</i> .
Ручная оценка генерации	не заполнена	CSV-файл содержит поля для ручной оценки генерации и используется как материал для последующей проверки согласованности ответа с контекстом.

CSV-файл содержит поля для ручной оценки генерации и используется как материал для последующей проверки согласованности ответа с контекстом. Ручные поля *context_sufficient_manual*, *answer_supported_by_citations_manual*, *no_extra_claims_manual*, *citation_relevance_manual*, *legal_meaning_preserved_manual* в представленном файле не заполнены для всех 50 вопросов. Поэтому результаты генерации следует рассматривать как материал автоматизированной проверки структуры ответа, использования фрагментов и подготовки последующей ручной оценки.

ПРИЛОЖЕНИЕ Е. ПРИМЕРЫ ЗАПРОСОВ И ОТВЕТОВ СИСТЕМЫ

Примеры взяты из файла *eval/results/generation/generation_all_v1.jsonl*. Для каждого примера приведены вопрос, метод извлечения релевантных фрагментов, краткое содержание найденного контекста, ответ системы, *used_chunk_ids*, сформированные цитаты и комментарий по проверяемости. Длинные цитаты сокращены для включения в приложение без изменения смысла.

Пример Е.1. Вопрос *q001*

Таблица Е.1 – Параметры примера Е.1

параметр	значение
ID вопроса	q001
Вопрос пользователя	Какие гарантии судебной защиты прав и свобод установлены Конституцией?
Метод извлечения	weighted_hybrid
Краткое содержание найденного контекста	Контекст содержит фрагменты нормативных актов, перечисленные в <i>retrieved_chunk_ids</i> .
Ответ системы	Конституция РФ устанавливает следующие гарантии судебной защиты прав и свобод: каждому гарантируется судебная защита его прав и свобод; решения и действия органов власти могут быть обжалованы в суд; каждый вправе обращаться в межгосударственные органы по защите прав и свобод человека после исчерпания всех имеющихся внутригосударственных средств правовой защиты.
<i>used_chunk_ids</i>	[13354]
<i>retrieved_chunk_ids</i>	[13353, 13354, 13356, 13325, 13337, 13327, 13360, 13316, 13336, 13391]
Статус	ok
Исключение	нет

Сформированные цитаты:

Таблица Е.2 – Сформированные цитаты примера Е.1

ранг	<i>chunk_id</i>	акт	статья	структурная ссылка	цитата
1	13354	Конституция Российской Федерации	статья 46	РАЗДЕЛ ПЕРВЫЙ / ГЛАВА 2. ПРАВА И СВОБОДЫ ЧЕЛОВЕКА И ГРАЖДАНИНА / Статья 46 / п. 1-3	Конституция Российской Федерации РАЗДЕЛ ПЕРВЫЙ ГЛАВА 2. ПРАВА И СВОБОДЫ ЧЕЛОВЕКА И ГРАЖДАНИНА Статья 46 1. Каждому гарантируется судебная защита его прав и свобод. 2. Решения и действия (или бездействие) органов государственной власти, органов местного самоуправления, общественных объединений и должностных лиц могут быть обжалованы в суд. 3. Каждый вправе в соответствии с международными договорами Российской Федерации обращаться в межгосударственные органы по защите прав и свобод человека, если исчерпаны все имеющиеся внутригосударственные средства правовой защиты.

Комментарий по проверяемости: Пример является успешным ответом с одной цитатой. Ответ проверяется по статье 46 Конституции Российской Федерации.

Пример Е.2. Вопрос *q049*

Таблица Е.3 – Параметры примера Е.2

параметр	значение
ID вопроса	q049
Вопрос пользователя	Как проверить электронный документ: какие бывают подписи и когда он равнозначен бумажному документу?
Метод извлечения	weighted_hybrid
Краткое содержание найденного контекста	Контекст содержит фрагменты нормативных актов, перечисленные в <i>retrieved_chunk_ids</i> .
Ответ системы	Электронный документ признается равнозначным бумажному документу, подписанному собственноручной подписью, если он подписан квалифицированной электронной подписью или простой электронной подписью в случаях, предусмотренных федеральными законами, нормативными правовыми актами или соглашением участников электронного взаимодействия. Существуют различные виды подписей: простая, неквалифицированная и квалифицированная электронная подпись. Сертификат ключа проверки электронной подписи подтверждает принадлежность ключа проверки владельцу сертификата.
<i>used_chunk_ids</i>	[14224, 14225, 14226, 14233, 14234]
<i>retrieved_chunk_ids</i>	[14225, 14226, 14234, 14220, 14224, 14233, 14231, 14221, 14227, 14254]
Статус	ok
Исключение	нет

Сформированные цитаты:

Таблица Е.4 – Сформированные цитаты примера Е.2

ранг	chunk_id	акт	статья	структурная ссылка	цитата
1	14224	Об электронной подписи	статья 5	Статья 5. Виды электронных подписей / п. 1-5	Об электронной подписи Статья 5. Виды электронных подписей 1. Видами электронных подписей, отношения в области использования которых регулируются настоящим Федеральным законом, являются простая электронная подпись и усиленная электронная подпись. Различаются усиленная неквалифицированная электронная подпись (далее - неквалифицированная электронная подпись) и усиленная квалифицированная электронная подпись (далее - квалифицированная электронная подпись). 2. [Фрагмент сокращен для включения в приложение.]
2	14225	Об электронной подписи	статья 6	Статья 6. Условия признания электронных документов, подписанных электронной подписью, равнозначными документам на бумажном носителе, подписанным собственноручной подписью / п. 1-3.1	Об электронной подписи Статья 6. Условия признания электронных документов, подписанных электронной подписью, равнозначными документам на бумажном носителе, подписанным собственноручной подписью 1. Информация в электронной форме, подписанная квалифицированной электронной подписью, признается электронным документом, равнозначным документу на бумажном носителе, подписанному собственноручной подписью, и может применяться в любых правоотношениях в соответствии с законодательством Российской Федерации, кроме случая, если федеральными законами или принимаемыми в соответствии с ними нормативными правовыми актами установлено требование о необходимости [Фрагмент сокращен для включения в приложение.]
3	14226	Об электронной подписи	статья 6	Статья 6. Условия признания электронных документов, подписанных электронной подписью, равнозначными документам на бумажном носителе, подписанным собственноручной подписью / п. 3.1-4	Об электронной подписи Статья 6. Условия признания электронных документов, подписанных электронной подписью, равнозначными документам на бумажном носителе, подписанным собственноручной подписью 3.1. Если федеральными законами, принимаемыми в соответствии с ними нормативными правовыми актами предусмотрено, что документ должен подписываться несколькими лицами, электронный документ должен быть подписан лицами (уполномоченными должностными лицами органа, организации), изготовившими этот документ, тем видом подписи, который установлен законодательством Российской Федерации для подписания изготовленного электронного документа электронной подписью [Фрагмент сокращен для включения в приложение.]

Продолжение таблицы Е.4

ранг	chunk_id	акт	статья	структурная ссылка	цитата
4	14233	Об электронной подписи	статья 11	Статья 11. Признание квалифицированной электронной подписи / п. 1-4	Об электронной подписи Статья 11. Признание квалифицированной электронной подписи Квалифицированная электронная подпись признается действительной до тех пор, пока решением суда не установлено иное, при одновременном соблюдении следующих условий: 1) квалифицированный сертификат создан и выдан аккредитованным удостоверяющим центром, аккредитация которого действительна на день выдачи указанного сертификата; [Фрагмент сокращен для включения в приложение.]
5	14234	Об электронной подписи	статья 12	Статья 12. Средства электронной подписи / п. 1-5	Об электронной подписи Статья 12. Средства электронной подписи 1. Для создания и проверки электронной подписи, создания ключа электронной подписи и ключа проверки электронной подписи должны использоваться средства электронной подписи, которые: 1) позволяют установить факт изменения подписанного электронного документа после момента его подписания; 2) обеспечивают практическую невозможность вычисления ключа электронной подписи из электронной подписи или из ключа ее проверки; [Фрагмент сокращен для включения в приложение.]

Комментарий по проверяемости: Пример является успешным ответом с несколькими цитатами. Ответ опирается на статьи 5 и 6 Федерального закона «Об электронной подписи», а также на связанные фрагменты о сертификате ключа проверки.

Пример Е.3. Вопрос q021

Таблица Е.5 – Параметры примера Е.3

параметр	значение
ID вопроса	q021
Вопрос пользователя	Что такое сделка по Гражданскому кодексу?
Метод извлечения	weighted_hybrid
Краткое содержание найденного контекста	Контекст содержит фрагменты нормативных актов, перечисленные в <i>retrieved_chunk_ids</i> .
Ответ системы	В переданном контексте недостаточно информации для ответа на вопрос.
<i>used_chunk_ids</i>	[]
<i>retrieved_chunk_ids</i>	[9500, 9338, 9560, 9578, 9576, 9334, 9570, 9398, 15658, 9545]
Статус	ok
Исключение	нет

Сформированные цитаты: цитаты отсутствуют.

Комментарий по проверяемости: Пример показывает безопасное сообщение о недостаточности контекста. Система не формирует неподтвержденные цитаты и оставляет *used_chunk_ids* пустым.

Пример Е.4. Вопрос *q050*

Таблица Е.6 – Параметры примера Е.4

параметр	значение
ID вопроса	q050
Вопрос пользователя	Нужно ли согласие пациента на медицинское вмешательство, если ситуация срочная?
Метод извлечения	не применен из-за ошибки
Краткое содержание найденного контекста	Найденный контекст отсутствует в сохраненном результате из-за ошибки генерации.
Ответ системы	Ответ не сформирован.
<i>used_chunk_ids</i>	[]
<i>retrieved_chunk_ids</i>	[]
Статус	error
Исключение	Модель вернула невалидный JSON: Expecting value: line 1 column 1 (char 0). Сырой ответ сохранен в eval/results/llm_debug/invalid_llm_response_1778701342086299000.txt

Сформированные цитаты: цитаты отсутствуют.

Комментарий по проверяемости: Пример проблемного случая: внешний ответ не был разобран как валидный JSON, поэтому ответ и цитаты не сформированы.

ПРИЛОЖЕНИЕ Ж. ЛИСТИНГИ КЛЮЧЕВЫХ ФРАГМЕНТОВ ПРОГРАММНОГО КОДА

В приложении приведены сокращенные фрагменты кода, отражающие основные этапы работы прототипа. Секреты, токены, пароли и содержимое `.env` не включены. Импорты и вспомогательные части сокращены, если они не требуются для понимания алгоритма.

Парсер является первым этапом подготовки корпуса. Он отбирает поддерживаемые входные файлы, пропускает временные и скрытые файлы, а при отсутствии валидных документов завершает работу с диагностическим сообщением. Это предотвращает появление пустых актов в корпусе.

Листинг Ж.1 – Отбор входных файлов и запуск парсинга нормативных актов

```
1 SUPPORTED_SUFFIXES = {".doc", ".html", ".htm"}
2 TEMP_FILE_PREFIXES = ("~$", ".")

3 def should_skip_input_file(path: Path) -> bool:
4     if path.suffix.lower() not in SUPPORTED_SUFFIXES:
5         return True
6     return path.name.startswith(TEMP_FILE_PREFIXES)

7 def parse_path(input_path: Path, output_dir: Path) -> None:
8     output_dir.mkdir(parents=True, exist_ok=True)
9     files, skipped = collect_input_files(input_path)

10    for path in skipped:
11        print(f"[SKIP] {path.name}")

12    if not files:
13        raise RuntimeError(
14            f"No supported files found in {input_path}. "
15            f"Supported suffixes: {'', '.join(sorted(SUPPORTED_SUFFIXES))}"
16        )

17    manifest: list[dict[str, Any]] = []
18    for path in files:
19        parsed = parse_document(path)
20        out_path = output_dir / f"{path.stem}.json"
21        write_json(out_path, parsed)
22        warnings = validate_parsed_document(parsed)
23        # фрагмент формирования manifest сокращен
```

Фрагмент показывает, что список входных файлов формируется до разбора HTML, а неподдерживаемые файлы исключаются из обработки. Результаты парсинга затем сохраняются в JSON и в *manifest.json*.

Разбиение на фрагменты выполняется с учетом юридической структуры документа. Текст фрагмента собирается из названия акта, структурного пути и текста нормы. Если статья помещается в лимит токенов, она сохраняется одним фрагментом.

Листинг Ж.2 – Формирование текста фрагмента и правило «одна статья — один фрагмент»

```
1 def make_chunk_text(  
2     act: dict[str, Any],  
3     group: dict[str, Any],  
4     nodes: list[dict[str, Any]],  
5     config: ChunkConfig,  
6 ) -> str:  
7     parts: list[str] = []  
8     if config.include_act_title:  
9         title = normalize_space(act.get("title") or "")  
10        if title:  
11            parts.append(title)  
12    if config.include_path:  
13        path_lines = build_path_lines(group, nodes)  
14        if path_lines:  
15            parts.append("\n".join(path_lines))  
  
16    body_lines: list[str] = []  
17    for node in nodes:  
18        if node.get("node_type") in STRUCTURAL_NODE_TYPES or node.get("node_type")  
19           == "article":  
20            continue  
21        text = node_to_text(node)  
22        if text:  
23            body_lines.append(text)  
  
24    if body_lines:  
25        parts.append("\n".join(body_lines))  
26    return "\n\n".join(parts).strip()  
  
27 def split_group_into_chunks(act: dict[str, Any], group: dict[str, Any], runtime:  
28     ChunkRuntime) -> list[dict[str, Any]]:  
29     full_chunk = make_chunk_record(act, group, group.get("nodes", []), runtime)  
30     if full_chunk["token_count"] <= runtime.config.chunk_size_tokens:  
31         return [full_chunk]  
32     # Если статья длинная, далее она разбивается по текстовым узлам.  
33     # фрагмент цикла разбиения по абзацам сокращен
```

Такой подход сохраняет ссылку на статью и уменьшает риск смешивания норм из разных статей. Для длинных статей в коде используется дополнительное разбиение по текстовым узлам.

Перед загрузкой корпуса в PostgreSQL выполняются проверки обязательных полей, дублей и ссылочной связности. Это снижает риск загрузки несогласованных фрагментов и некорректных ссылок на акты.

Листинг Ж.3 – Проверка фрагментов перед загрузкой корпуса

```
1 def validate_chunks(chunks: list[dict[str, Any]], act_keys: set[str]) -> None:
2     seen_pairs: set[tuple[str, int]] = set()
3     seen_hashes: set[str] = set()
4
5     for i, chunk in enumerate(chunks, start=1):
6         canonical_key = chunk.get("canonical_key")
7         chunk_index = chunk.get("chunk_index")
8         text = chunk.get("text")
9         token_count = chunk.get("token_count")
10        hash_value = chunk.get("hash")
11
12        if not canonical_key:
13            raise ValueError(f"chunks.jsonl row {i}: missing canonical_key")
14        if canonical_key not in act_keys:
15            raise ValueError(
16                f"chunks.jsonl row {i}: canonical_key not found in acts.jsonl: {canonical_key}"
17            )
18        if not isinstance(chunk_index, int) or chunk_index < 0:
19            raise ValueError(f"chunks.jsonl row {i}: invalid chunk_index {chunk_index}")
20        if not text or not str(text).strip():
21            raise ValueError(f"chunks.jsonl row {i}: empty text")
22        if not isinstance(token_count, int) or token_count <= 0:
23            raise ValueError(f"chunks.jsonl row {i}: invalid token_count {token_count}")
24        if not hash_value:
25            raise ValueError(f"chunks.jsonl row {i}: missing hash")
26
27        pair = (canonical_key, chunk_index)
28        if pair in seen_pairs:
29            raise ValueError(f"chunks.jsonl row {i}: duplicate chunk key {pair}")
30        seen_pairs.add(pair)
31
32        if hash_value in seen_hashes:
33            raise ValueError(f"chunks.jsonl row {i}: duplicate hash {hash_value}")
34        seen_hashes.add(hash_value)
```

Проверка выполняется до записи в базу данных. При обнаружении ошибки сценарий завершает работу с сообщением о конкретной строке `chunks.jsonl`.

Расчет векторных представлений выполняется только для фрагментов, у которых поле *embedding* еще не заполнено. Повторный запуск может продолжить обработку с места остановки. Размерность вектора проверяется перед записью в PostgreSQL.

Листинг Ж.4 – Выбор фрагментов без векторных представлений и проверка размерности

```

1 SELECT_CHUNKS_SQL = """
2 SELECT id, text
3 FROM chunks
4 WHERE embedding IS NULL
5 ORDER BY id
6 LIMIT %(limit)s;
7 """

8 UPDATE_EMBEDDING_SQL = """
9 UPDATE chunks
10 SET
11     embedding = %(embedding)s::vector,
12     embedding_model = %(embedding_model)s
13 WHERE id = %(chunk_id)s;
14 """

15 def encode_texts(model: SentenceTransformer, texts: list[str], batch_size: int, expected_dim:
    ↪ int) -> np.ndarray:
16     embeddings = model.encode(
17         texts,
18         batch_size=batch_size,
19         normalize_embeddings=True,
20         show_progress_bar=False,
21         convert_to_numpy=True,
22     )
23     if embeddings.ndim != 2 or embeddings.shape[1] != expected_dim:
24         raise RuntimeError(
25             f"Ожидалась размерность embeddings {expected_dim}, получено
                ↪ {embeddings.shape}"
26         )
27     return embeddings

```

Фрагмент отражает связь между моделью *BAAI/bge-m3*, полем *embedding* и типом *vector* в PostgreSQL. Запись имени модели позволяет фильтровать результаты векторного поиска.

Гибридное извлечение *weighted_hybrid* объединяет результаты полного текстового и векторного поиска. Итоговая оценка рассчитывается по рангам фрагмента в каждом списке с учетом весов из конфигурации.

Листинг Ж.5 – Объединение полнотекстового и векторного поиска методом *weighted_hybrid*

```
1 def weighted_rrf_fusion(  
2     sparse_rows: list[dict[str, Any]],  
3     dense_rows: list[dict[str, Any]],  
4     config: RetrievalConfig,  
5 ) -> list[RetrievedChunk]:  
6     combined: dict[int, dict[str, Any]] = {}  
  
7     for rank, row in enumerate(sparse_rows, start=1):  
8         chunk_id = int(row["chunk_id"])  
9         if chunk_id not in combined:  
10             combined[chunk_id] = dict(row)  
11             combined[chunk_id]["sparse_rank"] = rank  
12             combined[chunk_id]["retrieval_score"] = combined[chunk_id].get("retrieval_score",  
13                                     ↪ 0.0)  
13             combined[chunk_id]["retrieval_score"] += config.sparse_weight / (config.rrf_k + rank)  
  
14     for rank, row in enumerate(dense_rows, start=1):  
15         chunk_id = int(row["chunk_id"])  
16         if chunk_id not in combined:  
17             combined[chunk_id] = dict(row)  
18             combined[chunk_id]["dense_rank"] = rank  
19             combined[chunk_id]["retrieval_score"] = combined[chunk_id].get("retrieval_score",  
20                                     ↪ 0.0)  
20             combined[chunk_id]["retrieval_score"] += config.dense_weight / (config.rrf_k + rank)  
  
21     chunks = [row_to_chunk(row, row.get("sparse_rank"), row.get("dense_rank"),  
22                       ↪ float(row["retrieval_score"])) for row in combined.values()]  
22     return sorted(chunks, key=lambda item: item.retrieval_score, reverse=True)
```

Если фрагмент найден только одним методом, в итоговую оценку попадает только соответствующая часть формулы. После сортировки выбираются первые фрагменты для передачи в модуль генерации.

Ответ языковой модели принимается только в виде структурированного JSON. После разбора ответа система проверяет, что использованные фрагменты действительно были переданы в контекст, и только затем формирует цитаты.

Листинг Ж.6 – Проверка JSON-ответа и `used_chunk_ids`

```
1 class ModelAnswer(BaseModel):
2     answer: str
3     used_chunk_ids: list[int]
4     needs_clarification: bool

5 def parse_model_answer(raw_content: str) -> ModelAnswer:
6     content = _strip_json_fence(raw_content).strip()
7     # фрагмент подготовки кандидатов JSON сокращен
8     for candidate in candidates:
9         try:
10             answer = ModelAnswer.model_validate_json(candidate)
11             if not answer.used_chunk_ids:
12                 return answer.model_copy(update={
13                     "answer": "В переданном контексте недостаточно информации для
14                     ↳ ответа на вопрос.",
15                     "used_chunk_ids": [],
16                     "needs_clarification": True,
17                 })
18             return answer
19         except (ValidationError, ValueError, TypeError) as exc:
20             last_exc = exc
21             raise ValueError(f"Модель вернула невалидный JSON: {last_exc}")

21 def build_answer_citations(model_answer: ModelAnswer, selected_chunks:
22     ↳ list[RetrievedChunk]) -> list[AnswerCitation]:
23     chunks_by_id = {chunk.chunk_id: chunk for chunk in selected_chunks}
24     unknown_ids = [chunk_id for chunk_id in model_answer.used_chunk_ids if chunk_id not
25     ↳ in chunks_by_id]
26     if unknown_ids:
27         raise ValueError(f"Модель сослалась на неизвестные chunk_id: {unknown_ids}")
28     # фрагмент формирования AnswerCitation сокращен
```

Проверка `used_chunk_ids` является ключевым механизмом проверяемости. Модель не может сослаться на фрагмент, которого не было в переданном контексте.

После успешной генерации система сохраняет вопрос, ответ и служебные параметры извлечения. Цитаты сохраняются отдельно и связываются с ответом и фрагментами.

Листинг Ж.7 – Сохранение вопроса и ответа в базе данных

```
1 def save_answer_run_in_conn(  
2     conn: psycopg.Connection,  
3     question: str,  
4     result: GeneratedAnswer,  
5     external_uid: str = TECHNICAL_USER_UID,  
6     user_id: int | None = None,  
7 ) -> int:  
8     query_user_id = user_id if user_id is not None else ensure_technical_user(conn,  
9         ↪ external_uid)  
10    with conn.cursor() as cur:  
11        cur.execute(  
12            """  
13            INSERT INTO queries (user_id, question, normalized_question)  
14            VALUES %(user_id)s, %(question)s, %(normalized_question)s)  
15            RETURNING id;  
16            """,  
17            {  
18                "user_id": query_user_id, "question": question, "normalized_question":  
19                ↪ normalize_question(question)},  
20        )  
21        query_id = int(_row_value(cur.fetchone(), "id"))  
22        cur.execute(  
23            """  
24            INSERT INTO answers (  
25                query_id, answer_text, llm_model, prompt_version,  
26                needs_clarification, retrieval_method,  
27                retrieved_chunk_ids, dropped_chunk_ids, latency_ms  
28            )  
29            VALUES %(query_id)s, %(answer_text)s, %(llm_model)s, %(prompt_version)s,  
30                %(needs_clarification)s, %(retrieval_method)s,  
31                %(retrieved_chunk_ids)s, %(dropped_chunk_ids)s, %(latency_ms)s)  
32            RETURNING id;  
33            """,  
34            # фрагмент параметров сокращен  
35        )
```

Сохранение выполняется в одной транзакции соединения. Это обеспечивает согласованность вопроса, ответа и последующих цитат.

Локальная авторизация используется для разделения истории вопросов и ограничения доступа к сохраненным ответам. Пароль пользователя хешируется с солью, а при открытии ответа проверяется владелец записи.

Листинг Ж.8 – Хеширование пароля и проверка владельца ответа

```
1 def hash_password(password: str) -> str:
2     if not password:
3         raise ValueError("Пароль не должен быть пустым")
4     salt = secrets.token_bytes(PASSWORD_SALT_BYTES)
5     digest = hashlib.pbkdf2_hmac(
6         "sha256",
7         password.encode("utf-8"),
8         salt,
9         PASSWORD_HASH_ITERATIONS,
10    )
11    return f"{PASSWORD_HASH_ALGORITHM}${PASSWORD_HASH_ITERATIONS}${salt}
    ↪ .hex()}{digest.hex()}"

12 def _ensure_answer_owner(page: dict[str, Any], user: dict[str, Any]) -> None:
13     answer_user_id = page.get("answer", {}).get("user_id")
14     if answer_user_id is None or int(answer_user_id) != int(user["id"]):
15         raise HTTPException(status_code=403, detail="Нем доступна к этому ответу.")

16 @app.post("/answers/{answer_id}/feedback", response_model=None)
17 async def answer_feedback(request: Request, answer_id: int) -> RedirectResponse |
    ↪ HTMLResponse:
18     current_user = get_current_user(request)
19     if current_user is None:
20         return _redirect_to_auth(request, next_url=request.url_for("answer_page",
    ↪ answer_id=answer_id).path, login_required=True)
21     # далее проверяется владелец ответа и вызывается save_feedback(...)
```

Фрагмент показывает два связанных механизма: безопасное хранение пароля и запрет доступа к ответам другого пользователя. Обратная связь также сохраняется только после проверки владельца ответа.